Integrated Allergy Testing
Full Source Code Document

April 8, 2026 | Version 3.1.0
All TypeScript & TSX source files

Comments are rendered in green. Code in dark gray. Monospace font throughout.

■■ lib/ ■■

audit.ts

lib/audit.ts

```
/**
 * @file lib/audit.ts — HIPAA audit logging helper
 * @description Provides a non-fatal `log()` helper for recording HIPAA-required audit events.
 * All PHI access and mutations should call this function to ensure compliance.
 * Failures are swallowed (non-fatal) so audit issues never break core functionality.
 * @usage `import { log } from '@/lib/audit'`
 * `await log({ action: 'READ', entity: 'Patient', entityId: id, patientId: id })`
 */
import prisma from '@/lib/db'

export async function log(params: {
  action: string
  entity?: string
  entityId?: string
  patientId?: string
  details?: string
}): Promise<void> {
  try {
    await prisma.auditLog.create({ data: params })
  } catch {
    // non-fatal
  }
}
```

session.ts

lib/auth/session.ts

```
/**
 * @file lib/auth/session.ts — JWT session management
 * @description Handles JWT creation, verification, and cookie management for staff authentication.
 * - `signSession(payload)` — Creates a signed 8-hour JWT.
 * - `verifySession(req)` — Validates the iat_session cookie from a request.
 * - `setSessionCookie(token)` — Sets the HttpOnly session cookie.
 * - `clearSessionCookie()` — Deletes the session cookie on logout.
 * Secret is read from JWT_SECRET env var; falls back to a dev-only default.
 * @usage `import { verifySession } from '@/lib/auth/session'`
 */
import { SignJWT, jwtVerify } from 'jose'
import { cookies } from 'next/headers'
import { getSecret } from '@/lib/keyVault'

const COOKIE_NAME = 'iat_session'

// Lazy-loaded secret — fetched from Key Vault on first use, cached in memory
let _secretBytes: Uint8Array | null = null;

async function getSecretBytes(): Promise<Uint8Array> {
  if (_secretBytes) return _secretBytes;
  const secret = await getSecret('iat-jwt-secret', 'JWT_SECRET')
    ?? 'iat-dev-secret-change-in-production-32c';
  _secretBytes = new TextEncoder().encode(secret);
  return _secretBytes;
}

export async function signSession(payload: Record<string, unknown>) {
  const secret = await getSecretBytes();
  return new SignJWT(payload)
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('8h')
    .sign(secret)
}
```

```
export async function verifySession(req: Request): Promise<Record<string, unknown> | null> {
  try {
    const cookieHeader = req.headers.get('cookie') || ''
    const match = cookieHeader.match(new RegExp(`${COOKIE_NAME}=([^;]+)`))
    if (!match) return null
    const secret = await getSecretBytes();
    const { payload } = await jwtVerify(match[1], secret)
    return payload as Record<string, unknown>
  } catch { return null }
}
export async function setSessionCookie(token: string) {
  const cookieStore = await cookies()
  cookieStore.set(COOKIE_NAME, token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 60 * 60 * 8,
    path: '/',
  })
}
export async function clearSessionCookie() {
  const cookieStore = await cookies()
  cookieStore.delete(COOKIE_NAME)
}
```

db.ts

lib/db.ts

```
/**
 * @file lib/db.ts — Prisma client singleton for Turso/LibSQL
 *
 * @description
 * Exports a single shared PrismaClient instance that works with both local SQLite
 * (dev) and Turso remote databases (staging/production) via the libsql adapter.
 *
 * The singleton pattern (globalForPrisma) prevents new connections from being
 * created on every module load during Next.js hot reload in development, and
 * ensures warm serverless Lambda invocations reuse the existing connection pool
 * rather than creating a new one on each request.
 *
 * Configuration:
 * - DATABASE_URL: libsql:// URL for Turso (or file:./prisma/dev.db for local dev)
 * - DATABASE_AUTH_TOKEN: Turso auth token (not required for local file databases)
 *
 * @example
 * import prisma from '@/lib/db';
 * const patients = await prisma.patient.findMany({ where: { deletedAt: null } });
 */
import { PrismaClient } from '@/app/generated/prisma/client'
import { PrismaLibSql } from '@prisma/adapter-libsql'
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient }
function createPrismaClient() {
  const url = process.env.DATABASE_URL ?? 'file:./prisma/dev.db'
  const authToken = process.env.DATABASE_AUTH_TOKEN
  // Pass auth token to PrismaLibSql for Turso remote databases
  const adapterConfig = authToken ? { url, authToken } : { url }
  const adapter = new PrismaLibSql(adapterConfig)
  // log: [] suppresses verbose query/info logs in production
  return new PrismaClient({ adapter, log: [] })
}
// Singleton: reuse the client across warm serverless invocations in all environments.
// Without this, every cold start in production creates a new connection pool.
export const prisma = globalForPrisma.prisma ?? createPrismaClient()
```

```
// Always persist to global (not just dev) so Vercel warm lambdas reuse the connection.
globalForPrisma.prisma = prisma

export default prisma
```

hipaaHeaders.ts

lib/hipaaHeaders.ts

```
/**
 * @file lib/hipaaHeaders.ts — HIPAA-compliant HTTP response headers
 * @description Defines the standard set of security headers required on all API responses
 * that return Protected Health Information (PHI).
 * Add to NextResponse via: `NextResponse.json(data, { headers: HIPAA_HEADERS })`
 * For streaming/binary responses: spread into headers object `{ ...HIPAA_HEADERS }`
 * @usage `import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'`
 */
export const HIPAA_HEADERS = {
  'X-Content-Type-Options': 'nosniff',
  'X-Frame-Options': 'DENY',
  'Referrer-Policy': 'no-referrer',
  'Cache-Control': 'no-store, no-cache, must-revalidate',
  'Pragma': 'no-cache',
  'Strict-Transport-Security': 'max-age=63072000; includeSubDomains; preload',
} as const
```

keyVault.ts

lib/keyVault.ts

```
/**
 * @file lib/keyVault.ts — Azure Key Vault secret fetching
 * @description Fetches secrets from Azure Key Vault for production use.
 * Falls back to environment variables in development.
 * Caches secrets in memory for the lifetime of the lambda invocation.
 */
import { SecretClient } from '@azure/keyvault-secrets';
import { ClientSecretCredential } from '@azure/identity';

let client: SecretClient | null = null;
const secretCache = new Map<string, string>();

function getClient(): SecretClient | null {
  if (client) return client;

  const vaultUrl = process.env.AZURE_KEYVAULT_URL;
  const tenantId = process.env.AZURE_TENANT_ID;
  const clientId = process.env.AZURE_CLIENT_ID;
  const clientSecret = process.env.AZURE_CLIENT_SECRET;

  if (!vaultUrl || !tenantId || !clientId || !clientSecret) return null;

  const credential = new ClientSecretCredential(tenantId, clientId, clientSecret);
  client = new SecretClient(vaultUrl, credential);
  return client;
}
/**
 * Fetches a secret from Azure Key Vault, falling back to env var.
 * @param secretName - Key Vault secret name
 * @param envFallback - Environment variable name to use if Key Vault is unavailable
 * @returns Secret value or undefined
 */
export async function getSecret(secretName: string, envFallback?: string): Promise<string | undefined> {
  // Return from cache if available
  if (secretCache.has(secretName)) {
    return secretCache.get(secretName);
  }
  const kvClient = getClient();
  if (kvClient) {
    try {
      const secret = await kvClient.getSecret(secretName);
```

```
      const value = secret.value;
      if (value) {
        secretCache.set(secretName, value);
        return value;
      }
    } catch (err) {
      console.warn(`[KeyVault] Could not fetch ${secretName}, falling back to env:`, (err as Error).message);
    }
  }
  // Fall back to environment variable
  const fallback = envFallback ? process.env[envFallback] : undefined;
  if (fallback) {
    secretCache.set(secretName, fallback);
  }
  return fallback;
}
```

pdf.ts

lib/pdf.ts

```
/**
 * @file lib/pdf.ts — PDF generation utilities
 * @description Generates patient-facing PDF documents using jsPDF.
 * - `generateConsentPDF` — Patient consent form with optional signature image.
 * - `generateTestResultsPDF` — Tabular allergy test results report.
 * Both functions return a Blob suitable for streaming via NextResponse.
 * @usage `import { generateConsentPDF, generateTestResultsPDF } from '@/lib/pdf'`
 */
import { jsPDF } from 'jspdf'

export function generateConsentPDF(
  patient: {
    firstName: string
    lastName: string
    honorific?: string | null
    dob: Date | string
    email?: string | null
    patientId: string
  },
  signatureBase64?: string
): Blob {
  const doc = new jsPDF()

  // Header
  doc.setFontSize(18)
  doc.setTextColor('#0055A5')
  doc.text('Integrated Allergy Testing', 105, 20, { align: 'center' })
  doc.setFontSize(14)
  doc.setTextColor('#000000')
  doc.text('Patient Consent Form', 105, 30, { align: 'center' })

  // Patient info
  doc.setFontSize(11)
  const name = [patient.honorific, patient.firstName, patient.lastName].filter(Boolean).join(' ')
  doc.text(`Patient: ${name}`, 20, 50)
  doc.text(`Patient ID: ${patient.patientId}`, 20, 58)
  doc.text(`DOB: ${new Date(patient.dob).toLocaleDateString()}`, 20, 66)
  doc.text(`Date: ${new Date().toLocaleDateString()}`, 20, 74)

  // Consent text
  doc.setFontSize(10)
  const consentText = [
    'I hereby consent to allergy testing procedures performed by Integrated Allergy Testing staff.',
    'I understand that allergy testing involves exposure to allergens that may cause reactions.',
    'I have been informed of the risks and benefits of allergy testing.',
    'I authorize the release of my allergy test results to my referring physician.',
  ]
  let y = 90
```

```
  consentText.forEach((line) => {
    doc.text(line, 20, y, { maxWidth: 170 })
    y += 14
  })
  // Signature
  if (signatureBase64) {
    doc.text('Patient Signature:', 20, y + 10)
    doc.addImage(signatureBase64, 'PNG', 20, y + 15, 80, 30)
  } else {
    doc.line(20, y + 30, 100, y + 30)
    doc.text('Patient Signature', 20, y + 36)
  }
  doc.line(120, y + 30, 190, y + 30)
  doc.text('Date', 120, y + 36)
  return doc.output('blob')
}
export function generateTestResultsPDF(
  patient: {
    firstName: string
    lastName: string
    honorific?: string | null
    patientId: string
    dob: Date | string
  },
  results: Array<{
    allergenName: string
    testType: string
    reaction: number
    wheal?: string | null
    testedAt: Date | string
  }>
): Blob {
  const doc = new jsPDF()
  doc.setFontSize(18)
  doc.setTextColor('#0055A5')
  doc.text('Integrated Allergy Testing', 105, 20, { align: 'center' })
  doc.setFontSize(14)
  doc.setTextColor('#000000')
  doc.text('Allergy Test Results', 105, 30, { align: 'center' })
  const name = [patient.honorific, patient.firstName, patient.lastName].filter(Boolean).join(' ')
  doc.setFontSize(11)
  doc.text(`Patient: ${name}`, 20, 45)
  doc.text(`Patient ID: ${patient.patientId}`, 20, 53)
  doc.text(`Test Date: ${new Date().toLocaleDateString()}`, 20, 61)
  // Table header
  let y = 75
  doc.setFillColor('#0055A5')
  doc.rect(20, y, 170, 8, 'F')
  doc.setTextColor('#FFFFFF')
  doc.setFontSize(9)
  doc.text('Allergen', 22, y + 5.5)
  doc.text('Type', 90, y + 5.5)
  doc.text('Reaction (0-4)', 120, y + 5.5)
  doc.text('Wheal (mm)', 160, y + 5.5)
  y += 10
  doc.setTextColor('#000000')
  results.forEach((r, i) => {
    if (i % 2 === 0) {
      doc.setFillColor('#F0F9FF')
      doc.rect(20, y - 2, 170, 8, 'F')
    }
    doc.text(r.allergenName.slice(0, 35), 22, y + 4)
    doc.text(r.testType, 90, y + 4)
    doc.text(String(r.reaction), 135, y + 4)
```

```
    doc.text(r.wheal || '—', 165, y + 4)
    y += 8
    if (y > 270) {
      doc.addPage()
      y = 20
    }
  })
  return doc.output('blob')
}
```

■■ prisma/ ■■

schema.prisma

prisma/schema.prisma

```
generator client {
  provider = "prisma-client"
  output = "../app/generated/prisma"
}
datasource db {
  provider = "sqlite"
}
// ■■ Shared tables (match integrated-allergy IMS schema exactly) ■■■■■■■■■■■■■■■
model Location {
  id String @id @default(cuid())
  name String
  key String @unique
  suite String?
  street String
  city String
  state String
  zip String
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  dates LocationDate[]
}
model LocationDate {
  id String @id @default(cuid())
  locationId String
  type String
  date DateTime
  notes String?
  createdAt DateTime @default(now())
  location Location @relation(fields: [locationId], references: [id])
}
model Doctor {
  id String @id @default(cuid())
  name String
  title String @default("MD")
  specialty String @default("Allergy & Immunology")
  email String?
  phone String?
  clinicLocation String?
  npi String?
  photoUrl String?
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  entityId String?
  appUserId String?
```

```
  patients Patient[]
  dates DoctorDate[]
}
model DoctorDate {
  id String @id @default(cuid())
  doctorId String
  type String
  date DateTime
  notes String?
  createdAt DateTime @default(now())
  doctor Doctor @relation(fields: [doctorId], references: [id])
}
model Nurse {
  id String @id @default(cuid())
  name String
  title String @default("RN")
  email String?
  phone String?
  clinicLocation String?
  npi String?
  photoUrl String?
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  entityId String?
  appUserId String?
}
// Patient — matches IMS schema but with IAT-specific date relations
model Patient {
  id String @id @default(cuid())
  patientId String @unique
  name String
  dob DateTime
  physician String @default("")
  clinicLocation String @default("")
  diagnosis String @default("")
  startDate DateTime @default(now())
  phone String?
  homePhone String?
  email String?
  street String?
  apt String?
  city String?
  state String?
  zip String?
  insuranceId String?
  insuranceProvider String?
  insuranceGroup String?
  emergencyName String?
  emergencyPhone String?
  emergencyRelation String?
  notes String?
  status String @default("build-up")
  doctorId String?
  entityId String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  doctor Doctor? @relation(fields: [doctorId], references: [id])
  dates PatientDate[]
  testResults AllergyTestResult[]
  videoActivity VideoActivity[]
  formActivity FormActivity[]
```

```
  auditLogs AuditLog[]
  consentRecords ConsentRecord[]
  @@index([dob])
  @@index([deletedAt])
  @@index([status])
}
model PatientDate {
  id String @id @default(cuid())
  patientId String
  type String
  date DateTime
  notes String?
  createdAt DateTime @default(now())
  patient Patient @relation(fields: [patientId], references: [id])
}
// Allergen — matches IMS schema (no category column in shared table)
model Allergen {
  id String @id @default(cuid())
  name String
  type String @default("other")
  manufacturer String @default("")
  lotNumber String?
  stockConc String?
  expiresAt DateTime?
  createdAt DateTime @default(now())
  deletedAt DateTime?
  testResults AllergyTestResult[]
}
model AllergyTestResult {
  id String @id @default(cuid())
  patientId String
  allergenId String
  testType String // scratch | intradermal
  reaction Int @default(0)
  wheal String?
  notes String?
  nurseName String?
  testedAt DateTime @default(now())
  readAt DateTime?
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  patient Patient @relation(fields: [patientId], references: [id])
  allergen Allergen @relation(fields: [allergenId], references: [id])
  dates TestDate[]
  @@index([patientId])
  @@index([deletedAt])
  @@index([testedAt])
}
model TestDate {
  id String @id @default(cuid())
  testId String
  type String
  date DateTime
  notes String?
  createdAt DateTime @default(now())
  test AllergyTestResult @relation(fields: [testId], references: [id])
}
model Video {
  id String @id @default(cuid())
  title String
  description String?
```

```
  url String
  duration Int?
  category String?
  order Int @default(0)
  active Boolean @default(true)
  publishedAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  activity VideoActivity[]
}
model VideoActivity {
  id String @id @default(cuid())
  patientId String
  videoId String
  watchedAt DateTime @default(now())
  completed Boolean @default(false)
  acknowledged Boolean @default(false)
  createdAt DateTime @default(now())
  patient Patient @relation(fields: [patientId], references: [id])
  video Video @relation(fields: [videoId], references: [id])
}
model Form {
  id String @id @default(cuid())
  name String
  type String
  template String
  active Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  activity FormActivity[]
  consentRecords ConsentRecord[]
}
model FormActivity {
  id String @id @default(cuid())
  patientId String
  formId String
  signedAt DateTime?
  signature String?
  printedAt DateTime?
  emailedAt DateTime?
  pdfUrl String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  patient Patient @relation(fields: [patientId], references: [id])
  form Form @relation(fields: [formId], references: [id])
}
model AuditLog {
  id String @id @default(cuid())
  action String
  entity String @default("")
  entityId String?
  patientId String?
  details String?
  createdAt DateTime @default(now())
  patient Patient? @relation(fields: [patientId], references: [id])
}
model ConsentRecord {
  id String @id @default(cuid())
  patientId String
  formId String
  signedAt DateTime @default(now())
  signature String?
  pdfUrl String?
  ipAddress String?
```

```
  userAgent String?
  version String @default("1.0")
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  patient Patient @relation(fields: [patientId], references: [id])
  form Form @relation(fields: [formId], references: [id])
}
model WaitingRoom {
  id String @id @default(cuid())
  patientId String
  patientName String
  status String @default("waiting") // waiting | in-service | complete
  checkedInAt DateTime @default(now())
  calledAt DateTime?
  completedAt DateTime?
  nurseId String?
  nurseName String?
  notes String?
  videosWatched Int @default(0)
  videoAckBy String?
  videoAckAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
model StaffUser {
  id String @id @default(cuid())
  email String @unique
  passwordHash String
  name String
  role String @default("staff")
  active Boolean @default(true)
  mfaEnabled Boolean @default(false)
  mfaSecret String?
  tempToken String?
  tempTokenExpiry DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
model IATAppointment {
  id String @id @default(cuid())
  title String
  patientId String // Required — must have a patient
  patientName String?
  reasonId String?
  reasonName String?
  startTime DateTime
  endTime DateTime
  type String @default("allergy-test")
  notes String?
  status String @default("scheduled")
  createdBy String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  @@index([startTime])
  @@index([deletedAt])
  @@index([patientId])
}
model Encounter {
  id String @id @default(cuid())
  patientId String
  encounterDate DateTime @default(now())
  doctorId String?
  doctorName String?
```

```
  nurseId String?
  nurseName String?
  appointmentId String?
  chiefComplaint String
  subjectiveNotes String?
  objectiveNotes String?
  assessment String?
  plan String?
  followUpDays Int?
  status String @default("open")
  createdBy String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?

  @@index([patientId])
  @@index([encounterDate])
  @@index([deletedAt])
}

model AppointmentReason {
  id String @id @default(cuid())
  name String
  color String @default("#0d9488")
  duration Int @default(30)
  active Boolean @default(true)
  sortOrder Int @default(0)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

■■ app/api/ ■■

route.ts

app/api/allergens/route.ts

```
/**
 * @file /api/allergens — Allergen reference list
 * @description Manages the allergen catalog used for allergy testing panels.
 * GET — Return all active allergens sorted by type and name.
 * POST — Create a new allergen entry (name required, type optional).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'

export const dynamic = 'force-dynamic'

export async function GET() {
  try {
    const allergens = await prisma.allergen.findMany({
      where: { deletedAt: null },
      orderBy: [{ type: 'asc' }, { name: 'asc' }],
    })
    return NextResponse.json(allergens)
  } catch (error) {
    console.error('GET /api/allergens error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}

export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as { name?: string; type?: string }
    const { name, type } = body

    if (!name) {
      return NextResponse.json({ error: 'name is required' }, { status: 400 })
```

```
    }
    const allergen = await prisma.allergen.create({
      data: { name, ...(type ? { type } : {}) },
    })
    return NextResponse.json(allergen, { status: 201 })
  } catch (error) {
    console.error('POST /api/allergens error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/allergens/seed/route.ts

```
/**
 * @file /api/allergens/seed — Allergen reference data seeder
 * @description Seeds the allergen catalog with the standard IAT panel (25 allergens across 5 categories).
 * POST — Idempotent: skips if allergens already exist.
 * @security Public route — used during initial database setup
 */
import { NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
const ALLERGENS: { name: string; category: string }[] = [
  // Pollen
  { name: 'Timothy Grass', category: 'pollen' },
  { name: 'Bermuda Grass', category: 'pollen' },
  { name: 'Kentucky Blue Grass', category: 'pollen' },
  { name: 'Ragweed', category: 'pollen' },
  { name: 'Mountain Cedar', category: 'pollen' },
  { name: 'Elm', category: 'pollen' },
  { name: 'Oak', category: 'pollen' },
  { name: 'Birch', category: 'pollen' },
  { name: 'Maple', category: 'pollen' },
  { name: 'Ash', category: 'pollen' },
  // Mold
  { name: 'Alternaria', category: 'mold' },
  { name: 'Aspergillus', category: 'mold' },
  { name: 'Cladosporium', category: 'mold' },
  { name: 'Penicillium', category: 'mold' },
  // Dust
  { name: 'Dust Mite (Dermatophagoides farinae)', category: 'dust' },
  { name: 'Dust Mite (Dermatophagoides pteronyssinus)', category: 'dust' },
  { name: 'Cockroach', category: 'dust' },
  // Animal
  { name: 'Cat', category: 'animal' },
  { name: 'Dog', category: 'animal' },
  { name: 'Horse', category: 'animal' },
  { name: 'Mouse', category: 'animal' },
  // Food
  { name: 'Peanut', category: 'food' },
  { name: 'Tree Nut Mix', category: 'food' },
  { name: 'Shellfish Mix', category: 'food' },
  { name: 'Egg White', category: 'food' },
  { name: 'Milk', category: 'food' },
]
export async function POST() {
  // Block in production — seed endpoints must not be accessible publicly
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json({ error: 'Not available in production' }, { status: 403 })
  }
  try {
    const count = await prisma.allergen.count()
```

```
    if (count > 0) {
      return NextResponse.json({
        message: 'Allergens already seeded',
        skipped: true,
        count,
      })
    }
    const created = await prisma.allergen.createMany({
      data: ALLERGENS,
    })
    return NextResponse.json({
      message: 'Allergens seeded successfully',
      count: created.count,
    }, { status: 201 })
  } catch (error) {
    console.error('POST /api/allergens/seed error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/appointment-reasons/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function PUT(req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
    const { id } = await params
    const body = await req.json() as { name?: string; color?: string; duration?: number; active?: boolean;
sortOrder?: number }
    const reason = await prisma.appointmentReason.update({
      where: { id },
      data: {
        ...(body.name !== undefined && { name: body.name }),
        ...(body.color !== undefined && { color: body.color }),
        ...(body.duration !== undefined && { duration: body.duration }),
        ...(body.active !== undefined && { active: body.active }),
        ...(body.sortOrder !== undefined && { sortOrder: body.sortOrder }),
      },
    })
    return NextResponse.json({ reason })
  } catch (err) {
    console.error('PUT /api/appointment-reasons/[id] error:', err)
    return NextResponse.json({ error: 'Failed to update reason' }, { status: 500 })
  }
}
export async function DELETE(_req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
    const { id } = await params
    await prisma.appointmentReason.update({ where: { id }, data: { active: false } })
    return NextResponse.json({ ok: true })
  } catch (err) {
    console.error('DELETE /api/appointment-reasons/[id] error:', err)
    return NextResponse.json({ error: 'Failed to delete reason' }, { status: 500 })
  }
}
```

route.ts

app/api/appointment-reasons/route.ts

```
/**
 * @file /api/appointment-reasons — Appointment reason/type management
 * @description Manages the list of appointment reasons (Shot, Testing, Intake etc.)
```

```
 * GET — list all active reasons ordered by sortOrder
 * POST — create new reason
 * @security Requires authenticated session
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'

export const dynamic = 'force-dynamic'

export async function GET() {
  try {
    const reasons = await prisma.appointmentReason.findMany({
      where: { active: true },
      orderBy: { sortOrder: 'asc' },
    })
    return NextResponse.json({ reasons })
  } catch (err) {
    console.error('GET /api/appointment-reasons error:', err)
    return NextResponse.json({ reasons: [] })
  }
}

export async function POST(req: NextRequest) {
  try {
    const body = await req.json() as { name: string; color?: string; duration?: number }
    if (!body.name?.trim()) return NextResponse.json({ error: 'Name is required' }, { status: 400 })

    const maxOrder = await prisma.appointmentReason.aggregate({ _max: { sortOrder: true } })
    const reason = await prisma.appointmentReason.create({
      data: {
        name: body.name.trim(),
        color: body.color || '#0d9488',
        duration: body.duration || 30,
        sortOrder: (maxOrder._max.sortOrder || 0) + 1,
      },
    })
    return NextResponse.json({ reason }, { status: 201 })
  } catch (err) {
    console.error('POST /api/appointment-reasons error:', err)
    return NextResponse.json({ error: 'Failed to create reason' }, { status: 500 })
  }
}
```

route.ts

app/api/audit/route.ts

```
/**
 * @file /api/audit — HIPAA audit log viewer
 * @description Read-only access to the AuditLog table for compliance review.
 * GET — Return up to 50 recent audit events; supports ?patientId= filter.
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const patientId = searchParams.get('patientId')
    const limitParam = searchParams.get('limit')
    const limit = limitParam ? Math.min(parseInt(limitParam, 10) || 50, 500) : 50

    const logs = await prisma.auditLog.findMany({
      where: {
        ...(patientId ? { patientId } : {}),
      },
      include: {
```

```
        patient: {
          select: {
            id: true,
            patientId: true,
            name: true,
          },
        },
      },
      orderBy: { createdAt: 'desc' },
      take: limit,
    })
    return NextResponse.json(logs, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('GET /api/audit error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/auth/[...nextauth]/route.ts

```
import NextAuth from 'next-auth'
import AzureADProvider from 'next-auth/providers/azure-ad'

// Allowed email domains - add clinic domains here
const ALLOWED_DOMAINS = [
  'tipinc.ai',
  // Add more clinic domains as they onboard
]
const handler = NextAuth({
  providers: [
    AzureADProvider({
      clientId: process.env.AZURE_AD_CLIENT_ID!,
      clientSecret: process.env.AZURE_AD_CLIENT_SECRET!,
      tenantId: process.env.AZURE_AD_TENANT_ID || 'common', // use specific tenant for single-org, 'common' for
multi
    }),
  ],
  callbacks: {
    async signIn({ user }) {
      const email = user.email || ''
      const domain = email.split('@')[1]?.toLowerCase()
      // Check domain allowlist — return false to block (NextAuth will redirect to error page)
      if (!domain || !ALLOWED_DOMAINS.includes(domain)) {
        return false
      }
      return true
    },
    async jwt({ token, user, account }) {
      if (account && user) {
        token.email = user.email
        token.name = user.name
        token.provider = account.provider
      }
      return token
    },
    async session({ session, token }) {
      if (session.user) {
        session.user.email = token.email as string
      }
      return session
    },
  },
  pages: {
    signIn: '/login',
```

```
    error: '/login',
  },
  debug: process.env.NODE_ENV === 'development',
  secret: process.env.NEXTAUTH_SECRET,
})
export { handler as GET, handler as POST }
```

route.ts

app/api/auth/azure-callback/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { getServerSession } from 'next-auth'
import prisma from '@/lib/db'
import { signSession } from '@/lib/auth/session'

export const dynamic = 'force-dynamic'

const ALLOWED_DOMAINS = ['tipinc.ai']

export async function GET(req: NextRequest) {
  try {
    const session = await getServerSession()
    if (!session?.user?.email) {
      return NextResponse.redirect(new URL('/login?error=no_session', req.url))
    }

    const email = session.user.email
    const domain = email.split('@')[1]?.toLowerCase()

    if (!domain || !ALLOWED_DOMAINS.includes(domain)) {
      return NextResponse.redirect(new URL('/login?error=unauthorized_domain', req.url))
    }
    // Find or create staff user
    let staffUser = await prisma.staffUser.findUnique({ where: { email } })
    if (!staffUser) {
      staffUser = await prisma.staffUser.create({
        data: {
          email,
          name: session.user.name || email.split('@')[0],
          passwordHash: 'sso-no-password', // SSO users don't use password
          role: 'staff',
          mfaEnabled: true, // Azure AD handles MFA
          active: true,
        },
      })
    }
    if (!staffUser.active) {
      return NextResponse.redirect(new URL('/login?error=account_disabled', req.url))
    }
    // Issue our app JWT
    const token = await signSession({
      userId: staffUser.id,
      email: staffUser.email,
      role: staffUser.role,
      name: staffUser.name,
      provider: 'azure',
    })
    const response = NextResponse.redirect(new URL('/', req.url))
    response.cookies.set('iat_session', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 60 * 60 * 8,
      path: '/',
    })
    // Log SSO login (fire-and-forget)
    prisma.auditLog.create({
      data: {
```

```
        action: 'SSO_LOGIN',
        entity: 'StaffUser',
        entityId: staffUser.id,
        details: `Azure AD SSO login: ${email}`,
      },
    }).catch(() => {})
    return response
  } catch (err) {
    console.error('Azure callback error:', err)
    return NextResponse.redirect(new URL('/login?error=sso_failed', req.url))
  }
}
```

route.ts

app/api/auth/login/route.ts

```
/**
 * @file /api/auth/login — Staff authentication
 * @description Authenticates staff users with email/password, then issues either a
 * temp token for MFA verification/setup or a direct session (super_admin only).
 * MFA is enforced for all staff roles.
 * @security Public route — no session required
 */
import { NextRequest, NextResponse } from 'next/server'
import bcrypt from 'bcryptjs'
import { randomUUID } from 'crypto'
import prisma from '@/lib/db'

export const dynamic = 'force-dynamic'

export async function POST(req: NextRequest) {
  try {
    const body = await req.json() as { email?: string; password?: string }
    const { email, password } = body
    if (!email || !password) {
      return NextResponse.json({ error: 'Email and password required' }, { status: 400 })
    }
    // Find staff user
    const user = await prisma.staffUser.findUnique({ where: { email } })
    if (!user || !user.active) {
      return NextResponse.json({ error: 'Invalid email or password' }, { status: 401 })
    }
    // Verify password
    const valid = await bcrypt.compare(password, user.passwordHash)
    if (!valid) {
      return NextResponse.json({ error: 'Invalid email or password' }, { status: 401 })
    }
    // Issue temp token for MFA flow
    const tempToken = randomUUID()
    const tempTokenExpiry = new Date(Date.now() + 10 * 60 * 1000) // 10 min
    await prisma.staffUser.update({
      where: { id: user.id },
      data: { tempToken, tempTokenExpiry },
    })
    if (!user.mfaEnabled || !user.mfaSecret) {
      // MFA not yet set up — send to setup flow
      return NextResponse.json({ requiresMfaSetup: true, tempToken })
    }
    // MFA enabled — require TOTP verification
    return NextResponse.json({ requiresMfa: true, tempToken })
  } catch (error) {
    console.error('POST /api/auth/login error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/auth/logout/route.ts

```
import { NextResponse } from 'next/server'
export const dynamic = 'force-dynamic'
export async function POST() {
  const response = NextResponse.json({ ok: true })
  response.cookies.set('iat_session', '', { maxAge: 0, path: '/' })
  return response
}
```

route.ts

app/api/auth/me/route.ts

```
/**
 * @file /api/auth/me — Current session user info
 * @description Returns the authenticated staff user's profile.
 * GET — Verify session and return id, email, name, role for the logged-in user.
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { verifySession } from '@/lib/auth/session'

export const dynamic = 'force-dynamic'

export async function GET(req: NextRequest) {
  const session = await verifySession(req)
  if (!session) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }
  try {
    const user = await prisma.staffUser.findUnique({
      where: { id: session.userId as string },
      select: { id: true, email: true, name: true, role: true, active: true },
    })
    if (!user || !user.active) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }
    return NextResponse.json(user)
  } catch (error) {
    console.error('GET /api/auth/me error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/auth/mfa-setup/route.ts

```
/**
 * @file /api/auth/mfa-setup — TOTP MFA setup
 * @description Two-phase MFA enrollment for staff users.
 * GET — Generates a new TOTP secret and returns the QR code data URL.
 * Requires x-temp-token header (issued by /api/auth/login).
 * POST — Verifies a TOTP code against the provided secret and, if valid,
 * persists the secret, enables MFA, and issues a full session cookie.
 * @security Requires valid tempToken (not expired, active user)
 */
import { NextRequest, NextResponse } from 'next/server'
import speakeasy from 'speakeasy'
import QRCode from 'qrcode'
import prisma from '@/lib/db'
import { signSession } from '@/lib/auth/session'

export const dynamic = 'force-dynamic'
```

```
export async function GET(req: NextRequest) {
  const tempToken = req.headers.get('x-temp-token')
  if (!tempToken) {
    return NextResponse.json({ error: 'Missing temp token' }, { status: 401 })
  }
  const user = await prisma.staffUser.findFirst({
    where: {
      tempToken,
      tempTokenExpiry: { gte: new Date() },
      active: true,
    },
  })
  if (!user) {
    return NextResponse.json({ error: 'Invalid or expired token' }, { status: 401 })
  }
  const secret = speakeasy.generateSecret({
    name: `IAT (${user.email})`,
    length: 32,
  })
  const qrCode = await QRCode.toDataURL(secret.otpauth_url!)
  return NextResponse.json({ secret: secret.base32, qrCode })
}
export async function POST(req: NextRequest) {
  try {
    const body = await req.json() as { tempToken?: string; secret?: string; code?: string }
    const { tempToken, secret, code } = body
    if (!tempToken || !secret || !code) {
      return NextResponse.json({ error: 'tempToken, secret, and code are required' }, { status: 400 })
    }
    const user = await prisma.staffUser.findFirst({
      where: {
        tempToken,
        tempTokenExpiry: { gte: new Date() },
        active: true,
      },
    })
    if (!user) {
      return NextResponse.json({ error: 'Invalid or expired token' }, { status: 401 })
    }
    // Verify the TOTP code against the provided secret
    const valid = speakeasy.totp.verify({
      secret,
      encoding: 'base32',
      token: code,
      window: 1,
    })
    if (!valid) {
      return NextResponse.json({ error: 'Invalid verification code' }, { status: 400 })
    }
    // Save MFA secret, enable MFA, clear temp token
    await prisma.staffUser.update({
      where: { id: user.id },
      data: {
        mfaSecret: secret,
        mfaEnabled: true,
        tempToken: null,
        tempTokenExpiry: null,
      },
    })
    // Issue session JWT
    const token = await signSession({
      userId: user.id,
```

```
      email: user.email,
      role: user.role,
      name: user.name,
    })
    const response = NextResponse.json({ success: true, role: user.role, name: user.name })
    response.cookies.set('iat_session', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 60 * 60 * 8,
      path: '/',
    })
    return response
  } catch (error) {
    console.error('POST /api/auth/mfa-setup error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/auth/mfa-verify/route.ts

```
/**
 * @file /api/auth/mfa-verify — TOTP verification during login
 * @description Verifies a TOTP code for a user who has MFA already enabled.
 * On success, issues a full session JWT cookie and clears the temp token.
 * @security Requires valid tempToken (issued by /api/auth/login)
 */
import { NextRequest, NextResponse } from 'next/server'
import speakeasy from 'speakeasy'
import prisma from '@/lib/db'
import { signSession } from '@/lib/auth/session'
export const dynamic = 'force-dynamic'
export async function POST(req: NextRequest) {
  try {
    const body = await req.json() as { tempToken?: string; code?: string }
    const { tempToken, code } = body
    if (!tempToken || !code) {
      return NextResponse.json({ error: 'tempToken and code are required' }, { status: 400 })
    }
    const user = await prisma.staffUser.findFirst({
      where: {
        tempToken,
        tempTokenExpiry: { gte: new Date() },
        active: true,
      },
    })
    if (!user) {
      return NextResponse.json({ error: 'Invalid or expired token' }, { status: 401 })
    }
    if (!user.mfaSecret) {
      return NextResponse.json({ error: 'MFA not configured for this account' }, { status: 400 })
    }
    // Verify TOTP code
    const valid = speakeasy.totp.verify({
      secret: user.mfaSecret,
      encoding: 'base32',
      token: code,
      window: 1,
    })
    if (!valid) {
      return NextResponse.json({ error: 'Invalid code' }, { status: 400 })
    }
```

```
    // Clear temp token
    await prisma.staffUser.update({
      where: { id: user.id },
      data: {
        tempToken: null,
        tempTokenExpiry: null,
      },
    })
    // Issue session JWT
    const token = await signSession({
      userId: user.id,
      email: user.email,
      role: user.role,
      name: user.name,
    })
    const response = NextResponse.json({ success: true, role: user.role, name: user.name })
    response.cookies.set('iat_session', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 60 * 60 * 8,
      path: '/',
    })
    return response
  } catch (error) {
    console.error('POST /api/auth/mfa-verify error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/auth/refresh/route.ts

```
/**
 * @file /api/auth/refresh — Session token refresh
 * @description Rotates the JWT session token before it expires.
 * POST — Verifies current session, issues a fresh 8-hour token.
 * Called automatically by the client when token age > 6 hours.
 * @security Requires valid existing session cookie
 */
import { NextRequest, NextResponse } from 'next/server'
import { verifySession, signSession, setSessionCookie } from '@/lib/auth/session'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function POST(req: NextRequest) {
  try {
    const session = await verifySession(req)
    if (!session) {
      return NextResponse.json({ error: 'No valid session' }, { status: 401 })
    }
    // Verify user still exists and is active
    const user = await prisma.staffUser.findUnique({
      where: { id: session.userId as string },
      select: { id: true, email: true, role: true, name: true, active: true },
    })
    if (!user || !user.active) {
      return NextResponse.json({ error: 'Session invalid' }, { status: 401 })
    }
    // Issue fresh token
    const newToken = await signSession({
      userId: user.id,
      email: user.email,
      role: user.role,
```

```
      name: user.name,
    })
    const response = NextResponse.json({ ok: true, refreshed: true })
    response.cookies.set('iat_session', newToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 60 * 60 * 8,
      path: '/',
    })
    return response
  } catch (error) {
    console.error('POST /api/auth/refresh error:', error)
    return NextResponse.json({ error: 'Refresh failed' }, { status: 500 })
  }
}
```

route.ts

app/api/clinic-hours/route.ts

```
/**
 * @file /api/clinic-hours — Clinic hours of operation
 * @description GET returns hours for all days. PUT updates a specific day.
 * Used by the calendar to block bookings outside operating hours.
 * @security Requires authenticated session
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'
const DAYS = ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun']
const DAY_LABELS: Record<string, string> = {
  mon: 'Monday', tue: 'Tuesday', wed: 'Wednesday', thu: 'Thursday',
  fri: 'Friday', sat: 'Saturday', sun: 'Sunday'
}
export async function GET() {
  try {
    const rows = await prisma.$queryRaw<{ key: string; value: string }[]>`
      SELECT key, value FROM Settings WHERE key LIKE 'clinic_hours_%' OR key = 'clinic_appointment_duration'
    `
    const hours: Record<string, { label: string; hours: string; open: boolean; start?: string; end?: string }> =
{}
    let appointmentDuration = 30
    for (const row of rows) {
      if (row.key === 'clinic_appointment_duration') {
        appointmentDuration = parseInt(row.value) || 30
        continue
      }
      const day = row.key.replace('clinic_hours_', '')
      const isOpen = row.value !== 'closed'
      const [start, end] = isOpen ? row.value.split('-') : [undefined, undefined]
      hours[day] = { label: DAY_LABELS[day] || day, hours: row.value, open: isOpen, start, end }
    }
    // Fill missing days with defaults
    for (const day of DAYS) {
      if (!hours[day]) {
        hours[day] = { label: DAY_LABELS[day] || day, hours: 'closed', open: false }
      }
    }
    return NextResponse.json({ hours, appointmentDuration }, { headers: HIPAA_HEADERS })
  } catch (err) {
    console.error('GET /api/clinic-hours error:', err)
    return NextResponse.json({ error: 'Failed to load hours' }, { status: 500 })
```

```
  }
}
export async function PUT(req: NextRequest) {
  try {
    const body = await req.json() as { day: string; hours: string }
    const { day, hours } = body
    if (!DAYS.includes(day)) {
      return NextResponse.json({ error: 'Invalid day' }, { status: 400 })
    }
    // Validate format: HH:MM-HH:MM or "closed"
    if (hours !== 'closed' && !/^\d{2}:\d{2}-\d{2}:\d{2}$/.test(hours)) {
      return NextResponse.json({ error: 'Invalid format. Use HH:MM-HH:MM or "closed"' }, { status: 400 })
    }
    await prisma.$executeRaw`
      INSERT INTO Settings (key, value) VALUES (${`clinic_hours_${day}`}, ${hours})
      ON CONFLICT(key) DO UPDATE SET value = excluded.value
    `
    return NextResponse.json({ ok: true })
  } catch (err) {
    console.error('PUT /api/clinic-hours error:', err)
    return NextResponse.json({ error: 'Failed to update hours' }, { status: 500 })
  }
}
```

route.ts

app/api/consent/check/route.ts

```
/**
 * @file /api/consent/check — Check consent form signing status for a patient
 * @description
 * GET — Returns the signing status of all required consent forms for a given patientId.
 * Returns { allSigned: boolean, forms: ConsentFormStatus[] }.
 * Kiosk-facing and staff-facing endpoint.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'
const CONSENT_FORMS = [
  { formId: 'form-consent-001', name: 'Patient Consent for Allergy Testing' },
  { formId: 'form-consent-002', name: 'Authorization for Release of Medical Information' },
]
export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url)
  const patientId = searchParams.get('patientId')
  if (!patientId) {
    return NextResponse.json({ error: 'patientId is required' }, { status: 400 })
  }
  try {
    const records = await prisma.consentRecord.findMany({
      where: {
        patientId,
        formId: { in: CONSENT_FORMS.map(f => f.formId) },
      },
      orderBy: { signedAt: 'desc' },
    })
    const forms = CONSENT_FORMS.map(f => {
      const record = records.find((r: { formId: string; signedAt: Date }) => r.formId === f.formId)
      return {
        formId: f.formId,
        name: f.name,
        signed: !!record,
        signedAt: record?.signedAt?.toISOString(),
```

```
      }
    })
    const allSigned = forms.every(f => f.signed)
    return NextResponse.json({ allSigned, forms }, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('Consent check error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/consent/pdf/route.ts

```
/**
 * @file /api/consent/pdf — Generate a signed consent PDF for a patient
 * @description
 * GET — Generates and returns a PDF for the specified patient + form combination.
 * Includes consent text, patient info, and embedded signature image.
 * Staff-facing endpoint for downloading signed consent records.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { jsPDF } from 'jspdf'
export const dynamic = 'force-dynamic'
export async function GET(req: NextRequest) {
  try {
    const { searchParams } = new URL(req.url)
    const patientId = searchParams.get('patientId')
    const formId = searchParams.get('formId')
    if (!patientId || !formId) return NextResponse.json({ error: 'Missing params' }, { status: 400 })
    const [patient, form, record] = await Promise.all([
      prisma.patient.findUnique({ where: { id: patientId } }),
      prisma.form.findUnique({ where: { id: formId } }),
      prisma.consentRecord.findFirst({
        where: { patientId, formId },
        orderBy: { signedAt: 'desc' },
      }),
    ])
    if (!patient || !form) return NextResponse.json({ error: 'Not found' }, { status: 404 })
    const doc = new jsPDF()
    // Header
    doc.setFontSize(18)
    doc.setTextColor('#0055A5')
    doc.text('Integrated Allergy Testing', 105, 20, { align: 'center' })
    doc.setFontSize(14)
    doc.setTextColor('#000000')
    doc.text(form.name, 105, 30, { align: 'center' })
    // Divider
    doc.setDrawColor('#e2e8f0')
    doc.line(20, 36, 190, 36)
    // Patient info
    doc.setFontSize(11)
    doc.setTextColor('#374151')
    doc.text(`Patient: ${patient.name}`, 20, 48)
    doc.text(`Patient ID: ${patient.patientId ?? patient.id.slice(0, 8).toUpperCase()}`, 20, 56)
    doc.text(`DOB: ${patient.dob ? new Date(patient.dob).toLocaleDateString('en-US', { month: 'long', day: '
numeric', year: 'numeric' }) : '—'}`, 20, 64)
    doc.text(
      `Date Signed: ${record ? new Date(record.signedAt).toLocaleDateString('en-US', { month: 'long', day: '
numeric', year: 'numeric' }) : 'Not signed'}`,
      20, 72
    )
    // Divider
```

```
    doc.line(20, 78, 190, 78)
    // Consent text (strip HTML)
    const plainText = (form.template || '')
      .replace(/<[^>]+>/g, ' ')
      .replace(/ /g, ' ')
      .replace(/&amp;/g, '&')
      .replace(/&lt;/g, '<')
      .replace(/&gt;/g, '>')
      .replace(/&quot;/g, '"')
      .replace(/\s+/g, ' ')
      .trim()
    const lines = doc.splitTextToSize(plainText, 170)
    let y = 88
    doc.setFontSize(10)
    doc.setTextColor('#1a2233')
    for (const line of lines) {
      if (y > 250) {
        doc.addPage()
        y = 20
      }
      doc.text(line, 20, y)
      y += 5
    }
    // Signature section
    y += 10
    if (y > 240) {
      doc.addPage()
      y = 20
    }
    doc.setDrawColor('#e2e8f0')
    doc.line(20, y - 4, 190, y - 4)
    doc.setFontSize(11)
    doc.setTextColor('#374151')
    doc.text('Patient Signature:', 20, y + 6)
    if (record?.signature) {
      try {
        doc.addImage(record.signature, 'PNG', 20, y + 10, 80, 30)
        y += 45
      } catch {
        // fallback to blank line if image fails
        doc.line(20, y + 20, 120, y + 20)
        y += 28
      }
    } else {
      doc.line(20, y + 20, 120, y + 20)
      y += 28
    }
    // Footer
    doc.setFontSize(9)
    doc.setTextColor('#94a3b8')
    doc.text(
      'This document is a legally binding consent. HIPAA Compliant. Integrated Allergy Testing.',
      105,
      285,
      { align: 'center' }
    )
    const pdfBytes = doc.output('arraybuffer')
    return new NextResponse(pdfBytes, {
      headers: {
        'Content-Type': 'application/pdf',
        'Content-Disposition': `attachment; filename="consent-${patient.patientId ?? patient.id.slice(0,
8)}-${formId}.pdf"`,
      },
```

```
    })
  } catch (err) {
    console.error('Consent PDF error:', err)
    return NextResponse.json({ error: 'PDF generation failed' }, { status: 500 })
  }
}
```

route.ts

app/api/consent/sign/route.ts

```
/**
 * @file /api/consent/sign — Record a patient's consent form signature
 * @description
 * POST — Creates a ConsentRecord with the patient's signature (base64 PNG).
 * Also creates an audit log entry for HIPAA compliance.
 * Kiosk-facing endpoint; requires patientId and formId.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

const VALID_CONSENT_FORM_IDS = ['form-consent-001', 'form-consent-002']
const MAX_SIGNATURE_BYTES = 500_000 // ~375KB base64-encoded PNG — reject absurdly large payloads

export async function POST(req: NextRequest) {
  try {
    const body = await req.json()
    const { patientId, formId, signature, userAgent } = body

    if (!patientId || !formId) {
      return NextResponse.json({ error: 'patientId and formId are required' }, { status: 400 })
    }

    // Validate formId is a known consent form — prevents spoofed formIds
    if (!VALID_CONSENT_FORM_IDS.includes(formId)) {
      return NextResponse.json({ error: 'Invalid formId' }, { status: 400 })
    }

    // Validate patientId exists
    const patient = await prisma.patient.findFirst({
      where: { id: patientId, deletedAt: null },
      select: { id: true },
    })
    if (!patient) {
      return NextResponse.json({ error: 'Patient not found' }, { status: 404 })
    }

    // Limit signature size to prevent DoS via enormous base64 payloads
    if (signature && typeof signature === 'string' && signature.length > MAX_SIGNATURE_BYTES) {
      return NextResponse.json({ error: 'Signature data too large' }, { status: 413 })
    }

    const ipAddress =
      req.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ||
      req.headers.get('x-real-ip') ||
      undefined

    const record = await prisma.consentRecord.create({
      data: {
        patientId,
        formId,
        signature: signature || null,
        ipAddress: ipAddress || null,
        userAgent: userAgent || req.headers.get('user-agent') || null,
        version: '1.0',
      },
    })

    await prisma.auditLog.create({
      data: {
        action: 'CONSENT_SIGNED',
```

```
        entity: 'ConsentRecord',
        entityId: record.id,
        patientId,
        details: `Consent signed for formId: ${formId}`,
      },
    }).catch(() => { /* non-blocking */ })
    return NextResponse.json({ ok: true, recordId: record.id }, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('Consent sign error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/doctors/[id]/route.ts

```
/**
 * @file /api/doctors/[id] — Single doctor operations
 * @description
 * GET — Fetch a single doctor by ID.
 * PUT — Update doctor fields.
 * DELETE — Soft delete (set deletedAt).
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function GET(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const doctor = await prisma.doctor.findFirst({
      where: { id, deletedAt: null },
    })
    if (!doctor) {
      return NextResponse.json({ error: 'Doctor not found' }, { status: 404 })
    }
    return NextResponse.json(doctor)
  } catch (error) {
    console.error('GET /api/doctors/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const body = await request.json() as {
      name?: string
      title?: string
      specialty?: string
      email?: string
      phone?: string
      clinicLocation?: string
      active?: boolean
    }
    const existing = await prisma.doctor.findFirst({ where: { id, deletedAt: null } })
    if (!existing) {
      return NextResponse.json({ error: 'Doctor not found' }, { status: 404 })
    }
    const doctor = await prisma.doctor.update({
```

```
      where: { id },
      data: {
        ...(body.name !== undefined ? { name: body.name } : {}),
        ...(body.title !== undefined ? { title: body.title } : {}),
        ...(body.specialty !== undefined ? { specialty: body.specialty } : {}),
        ...(body.email !== undefined ? { email: body.email } : {}),
        ...(body.phone !== undefined ? { phone: body.phone } : {}),
        ...(body.clinicLocation !== undefined ? { clinicLocation: body.clinicLocation } : {}),
        ...(body.active !== undefined ? { active: body.active } : {}),
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'DOCTOR_UPDATED',
        entity: 'Doctor',
        entityId: doctor.id,
        patientId: null,
        details: `Doctor updated: ${doctor.name}`,
      },
    }).catch(() => {})
    return NextResponse.json(doctor)
  } catch (error) {
    console.error('PUT /api/doctors/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function DELETE(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const existing = await prisma.doctor.findFirst({ where: { id, deletedAt: null } })
    if (!existing) {
      return NextResponse.json({ error: 'Doctor not found' }, { status: 404 })
    }
    await prisma.doctor.update({
      where: { id },
      data: { deletedAt: new Date(), active: false },
    })
    prisma.auditLog.create({
      data: {
        action: 'DOCTOR_DELETED',
        entity: 'Doctor',
        entityId: id,
        patientId: null,
        details: `Doctor deleted: ${existing.name}`,
      },
    }).catch(() => {})
    return NextResponse.json({ success: true })
  } catch (error) {
    console.error('DELETE /api/doctors/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/doctors/route.ts

```
/**
 * @file /api/doctors — Doctor/physician directory
 * @description Manages the referring physician list for patient assignments.
 * GET — List all active doctors.
 * POST — Create a new doctor record (name required).
```

```
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
const createDoctorSchema = z.object({
  name: z.string().min(1).max(200),
  title: z.string().max(50).optional(),
  specialty: z.string().max(200).optional(),
  email: z.string().email().optional().or(z.literal('')),
  phone: z.string().max(20).optional(),
  clinicLocation: z.string().max(200).optional(),
})
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const all = searchParams.get('all') === '1'
    const doctors = await prisma.doctor.findMany({
      where: {
        deletedAt: null,
        ...(all ? {} : { active: true }),
      },
      orderBy: [{ name: 'asc' }],
    })
    return NextResponse.json(doctors)
  } catch (error) {
    console.error('GET /api/doctors error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const result = createDoctorSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const { name, title, specialty, email, phone, clinicLocation } = result.data
    const doctor = await prisma.doctor.create({
      data: {
        name,
        ...(title ? { title } : {}),
        ...(specialty ? { specialty } : {}),
        ...(email ? { email } : {}),
        ...(phone ? { phone } : {}),
        ...(clinicLocation ? { clinicLocation } : {}),
        ...(((body as { npi?: string }).npi) ? { npi: (body as { npi?: string }).npi } : {}),
      },
    })
    return NextResponse.json(doctor, { status: 201 })
  } catch (error) {
    console.error('POST /api/doctors error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/encounters/[id]/pdf/route.ts

```
/**
 * @file /api/encounters/[id]/pdf — Clinical encounter summary PDF
 * @description
```

```
 * GET — Generates and returns a clinical encounter summary PDF using jsPDF.
 * Includes patient info, encounter details, SOAP notes, and HIPAA footer.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { jsPDF } from 'jspdf'

export const dynamic = 'force-dynamic'

export async function GET(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const encounter = await prisma.encounter.findUnique({
      where: { id: (await params).id },
    })
    if (!encounter || encounter.deletedAt) {
      return NextResponse.json({ error: 'Encounter not found' }, { status: 404 })
    }
    const patient = await prisma.patient.findUnique({
      where: { id: encounter.patientId },
    })
    if (!patient) {
      return NextResponse.json({ error: 'Patient not found' }, { status: 404 })
    }
    const doc = new jsPDF()
    const pageW = doc.internal.pageSize.getWidth()
    // ■■ Header ■■■
```

```
doc.text(`Date:`, 110, 54)
doc.text(`Physician:`, 110, 62)
doc.text(`Nurse:`, 110, 70)

doc.setTextColor('#111827')
doc.text(encDate, 140, 54)
doc.text(encounter.doctorName ?? '—', 140, 62)
doc.text(encounter.nurseName ?? '—', 140, 70)

doc.setDrawColor('#e2e8f0')
doc.line(20, 76, 190, 76)

// ■■ SOAP Sections ■■■
```

```
    const totalPages = (doc.internal as unknown as { getNumberOfPages: () => number }).getNumberOfPages()
    const datePrinted = new Date().toLocaleDateString('en-US', {
      month: 'long', day: 'numeric', year: 'numeric',
    })
    for (let i = 1; i <= totalPages; i++) {
      doc.setPage(i)
      doc.setFontSize(8)
      doc.setFont('helvetica', 'normal')
      doc.setTextColor('#94a3b8')
      doc.text(
        `HIPAA Compliant — Confidential Patient Record — Integrated Allergy Testing`,
        pageW / 2, 285, { align: 'center' }
      )
      doc.text(
        `Date Printed: ${datePrinted} | Page ${i} of ${totalPages}`,
        pageW / 2, 290, { align: 'center' }
      )
    }
    const pdfBytes = doc.output('arraybuffer')
    const safeId = patientDisplayId.replace(/[^a-zA-Z0-9]/g, '')
    const encDateStr = new Date(encounter.encounterDate).toISOString().slice(0, 10)
    return new NextResponse(pdfBytes, {
      headers: {
        'Content-Type': 'application/pdf',
        'Content-Disposition': `attachment; filename="encounter-${safeId}-${encDateStr}.pdf"`,
      },
    })
  } catch (err) {
    console.error('Encounter PDF error:', err)
    return NextResponse.json({ error: 'PDF generation failed' }, { status: 500 })
  }
}
```

route.ts

app/api/encounters/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'
export async function GET(_req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
    const { id } = await params
    const encounter = await prisma.encounter.findFirst({ where: { id, deletedAt: null } })
    if (!encounter) return NextResponse.json({ error: 'Not found' }, { status: 404 })
    return NextResponse.json(encounter, { headers: HIPAA_HEADERS })
  } catch (err) { console.error(err); return NextResponse.json({ error: 'Failed' }, { status: 500 }) }
}
export async function PUT(req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
    const { id } = await params
    const body = await req.json() as Record<string, unknown>
    const data: Record<string, unknown> = {}
    const fields = ['chiefComplaint','subjectiveNotes','objectiveNotes','assessment','plan','status','doctorId','d
octorName','nurseId','nurseName','appointmentId','followUpDays']
    for (const f of fields) if (body[f] !== undefined) data[f] = body[f]
    if (body.encounterDate) data.encounterDate = new Date(String(body.encounterDate))
    const encounter = await prisma.encounter.update({ where: { id }, data })
    prisma.auditLog.create({ data: { action: 'ENCOUNTER_UPDATED', entity: 'Encounter', entityId: id, patientId:
encounter.patientId }}).catch(()=>{})
    return NextResponse.json({ encounter }, { headers: HIPAA_HEADERS })
  } catch (err) { console.error(err); return NextResponse.json({ error: 'Failed' }, { status: 500 }) }
}
export async function DELETE(_req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
```

```
  try {
    const { id } = await params
    const encounter = await prisma.encounter.update({ where: { id }, data: { deletedAt: new Date() } })
    prisma.auditLog.create({ data: { action: 'ENCOUNTER_DELETED', entity: 'Encounter', entityId: id, patientId:
encounter.patientId }}).catch(()=>{})
    return NextResponse.json({ ok: true })
  } catch (err) { console.error(err); return NextResponse.json({ error: 'Failed' }, { status: 500 }) }
}
```

route.ts

app/api/encounters/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'

export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url)
  const patientId = searchParams.get('patientId')
  const limit = Math.min(parseInt(searchParams.get('limit') || '50'), 200)
  try {
    const encounters = await prisma.encounter.findMany({
      where: { ...(patientId ? { patientId } : {}), deletedAt: null },
      orderBy: { encounterDate: 'desc' },
      take: limit,
    })
    return NextResponse.json({ encounters }, { headers: HIPAA_HEADERS })
  } catch (err) { console.error(err); return NextResponse.json({ encounters: [] }) }
}

export async function POST(req: NextRequest) {
  try {
    const body = await req.json() as Record<string, unknown>
    if (!body.patientId || !body.chiefComplaint) return NextResponse.json({ error: 'patientId and chiefComplaint
required' }, { status: 400 })
    const encounter = await prisma.encounter.create({ data: {
      patientId: String(body.patientId),
      chiefComplaint: String(body.chiefComplaint),
      encounterDate: body.encounterDate ? new Date(String(body.encounterDate)) : new Date(),
      doctorId: body.doctorId ? String(body.doctorId) : null,
      doctorName: body.doctorName ? String(body.doctorName) : null,
      nurseId: body.nurseId ? String(body.nurseId) : null,
      nurseName: body.nurseName ? String(body.nurseName) : null,
      appointmentId: body.appointmentId ? String(body.appointmentId) : null,
      subjectiveNotes: body.subjectiveNotes ? String(body.subjectiveNotes) : null,
      objectiveNotes: body.objectiveNotes ? String(body.objectiveNotes) : null,
      assessment: body.assessment ? String(body.assessment) : null,
      plan: body.plan ? String(body.plan) : null,
      followUpDays: body.followUpDays ? Number(body.followUpDays) : null,
      status: body.status ? String(body.status) : 'open',
    }})
    prisma.auditLog.create({ data: { action: 'ENCOUNTER_CREATED', entity: 'Encounter', entityId: encounter.id,
patientId: encounter.patientId, details: `Chief complaint: ${encounter.chiefComplaint}` }}).catch(()=>{})
    return NextResponse.json({ encounter }, { status: 201, headers: HIPAA_HEADERS })
  } catch (err) { console.error(err); return NextResponse.json({ error: 'Failed to create encounter' }, { status:
500 }) }
}
```

route.ts

app/api/forms/activity/route.ts

```
/**
 * @file /api/forms/activity — Patient form activity tracking
 * @description Records form interactions (signing, printing, emailing) for a patient.
 * POST — Create a new form activity record (patientId and formId required).
```

```
 * Logs a CONSENT_SIGNED audit event when signedAt is provided.
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'

export const dynamic = 'force-dynamic'

export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as {
      patientId?: string
      formId?: string
      signedAt?: string
      signature?: string
      printedAt?: string
      emailedAt?: string
    }
    const { patientId, formId } = body
    if (!patientId || !formId) {
      return NextResponse.json(
        { error: 'patientId and formId are required' },
        { status: 400 }
      )
    }
    const activity = await prisma.formActivity.create({
      data: {
        patientId,
        formId,
        signedAt: body.signedAt ? new Date(body.signedAt) : null,
        signature: body.signature,
        printedAt: body.printedAt ? new Date(body.printedAt) : null,
        emailedAt: body.emailedAt ? new Date(body.emailedAt) : null,
      },
      include: {
        form: true,
        patient: {
          select: {
            id: true,
            patientId: true,
            name: true,
          },
        },
      },
    })
    // Audit consent signing — critical PHI mutation
    const action = body.signedAt ? 'CONSENT_SIGNED' : 'FORM_ACTIVITY_CREATED'
    await prisma.auditLog.create({
      data: {
        action,
        entity: 'FormActivity',
        entityId: activity.id,
        patientId,
        details: `Form ${formId} activity recorded for patient ${patientId}`,
      },
    })
    return NextResponse.json(activity, { status: 201 })
  } catch (error) {
    console.error('POST /api/forms/activity error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/forms/pdf/route.ts

```
/**
 * @file /api/forms/pdf — PDF document generation for patient forms
 * @description Generates and streams PDF files for patient consent or test results.
 * GET — Generate a PDF given ?patientId= and ?type= ('consent' | 'results').
 * Consent PDFs include the patient's stored signature if available.
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { generateConsentPDF, generateTestResultsPDF } from '@/lib/pdf'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url)
  const patientId = searchParams.get('patientId')
  const type = searchParams.get('type') // 'consent' | 'results'

  if (!patientId || !type) {
    return NextResponse.json({ error: 'patientId and type are required' }, { status: 400 })
  }
  if (!['consent', 'results'].includes(type)) {
    return NextResponse.json({ error: 'type must be consent or results' }, { status: 400 })
  }
  try {
    const patient = await prisma.patient.findFirst({
      where: {
        OR: [{ id: patientId }, { patientId }],
      },
    })
    if (!patient) {
      return NextResponse.json({ error: 'Patient not found' }, { status: 404 })
    }
    // Parse name into firstName/lastName for PDF compatibility
    const [firstName, ...rest] = patient.name.split(' ')
    const lastName = rest.join(' ')

    let pdfBlob: Blob
    let filename: string

    if (type === 'consent') {
      // Get the most recent signature if available
      const formActivity = await prisma.formActivity.findFirst({
        where: { patientId: patient.id, signedAt: { not: null } },
        orderBy: { signedAt: 'desc' },
      })
      pdfBlob = generateConsentPDF(
        {
          firstName,
          lastName,
          dob: patient.dob,
          email: patient.email,
          patientId: patient.patientId,
        },
        formActivity?.signature ?? undefined
      )
      filename = `consent-${patient.patientId}.pdf`
    } else {
      // Fetch test results
      const testResults = await prisma.allergyTestResult.findMany({
        where: { patientId: patient.id, active: true },
        include: { allergen: true },
        orderBy: { testedAt: 'desc' },
      })
      const results = testResults.map((r) => ({
        allergenName: r.allergen.name,
        testType: r.testType,
```

```
        reaction: r.reaction,
        wheal: r.wheal,
        testedAt: r.testedAt,
      }))
      pdfBlob = generateTestResultsPDF(
        {
          firstName,
          lastName,
          patientId: patient.patientId,
          dob: patient.dob,
        },
        results
      )
      filename = `results-${patient.patientId}.pdf`
    }
    const arrayBuffer = await pdfBlob.arrayBuffer()
    const buffer = Buffer.from(arrayBuffer)
    await prisma.auditLog.create({
      data: {
        action: 'PDF_GENERATED',
        entity: 'Patient',
        entityId: patient.id,
        patientId: patient.id,
        details: `Generated ${type} PDF for patient ${patient.patientId}`,
      },
    })
    return new NextResponse(buffer, {
      status: 200,
      headers: {
        'Content-Type': 'application/pdf',
        'Content-Disposition': `attachment; filename="${filename}"`,
        'Content-Length': String(buffer.length),
        ...HIPAA_HEADERS,
      },
    })
  } catch (error) {
    console.error('GET /api/forms/pdf error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/forms/route.ts

```
/**
 * @file /api/forms — Form template management
 * @description Manages form templates used for patient consent and intake.
 * GET — Return all active form templates.
 * POST — Create a new form template (name, type, and template required).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function GET() {
  try {
    const forms = await prisma.form.findMany({
      where: { active: true },
      orderBy: { name: 'asc' },
    })
    return NextResponse.json(forms)
  } catch (error) {
    console.error('GET /api/forms error:', error)
```

```
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as {
      name?: string
      type?: string
      template?: string
    }
    const { name, type, template } = body
    if (!name || !type || !template) {
      return NextResponse.json(
        { error: 'name, type, and template are required' },
        { status: 400 }
      )
    }
    // Sanitize template: strip <script>, <iframe>, on* attributes, and javascript: hrefs
    // to reduce XSS risk when rendered via dangerouslySetInnerHTML in consent pages.
    const sanitizedTemplate = template
      .replace(/<script\b[^<]*(?:(?!<\/script>)<[^<]*)*<\/script>/gi, '')
      .replace(/<iframe\b[^<]*(?:(?!<\/iframe>)<[^<]*)*<\/iframe>/gi, '')
      .replace(/<object\b[^<]*(?:(?!<\/object>)<[^<]*)*<\/object>/gi, '')
      .replace(/<embed\b[^>]*>/gi, '')
      .replace(/\bon\w+\s*=/gi, 'data-blocked=')
      .replace(/javascript\s*:/gi, 'blocked:')
      .replace(/vbscript\s*:/gi, 'blocked:')
      .replace(/data\s*:\s*text\s*\/\s*html/gi, 'blocked:')
    const form = await prisma.form.create({
      data: { name, type, template: sanitizedTemplate },
    })
    return NextResponse.json(form, { status: 201 })
  } catch (error) {
    console.error('POST /api/forms error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/health/route.ts

```
import { NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function GET() {
  try {
    await prisma.staffUser.count()
    return NextResponse.json({ ok: true })
  } catch (e) {
    const err = e as Error
    // Never expose DB URLs or credentials in health check responses
    console.error('Health check failed:', err.message)
    return NextResponse.json({ ok: false }, { status: 500 })
  }
}
```

route.ts

app/api/iat-appointments/[id]/route.ts

```
/**
 * @file /api/iat-appointments/[id] — Update or soft-delete a single appointment
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
```

```
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
const updateSchema = z.object({
  title: z.string().min(1).max(200).optional(),
  patientId: z.string().optional(),
  patientName: z.string().max(200).optional().nullable(),
  startTime: z.string().datetime().optional(),
  endTime: z.string().datetime().optional(),
  type: z.string().optional(),
  notes: z.string().max(1000).optional().nullable(),
  status: z.enum(['scheduled', 'in-progress', 'complete', 'cancelled']).optional(),
})
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const body = await request.json()
    const result = updateSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const existing = await prisma.iATAppointment.findFirst({
      where: { id, deletedAt: null },
    })
    if (!existing) {
      return NextResponse.json({ error: 'Appointment not found' }, { status: 404 })
    }
    const data = result.data
    // Validate time order if both provided
    const start = data.startTime ? new Date(data.startTime) : existing.startTime
    const end = data.endTime ? new Date(data.endTime) : existing.endTime
    if (start >= end) {
      return NextResponse.json({ error: 'endTime must be after startTime' }, { status: 400 })
    }
    const updated = await prisma.iATAppointment.update({
      where: { id },
      data: {
        ...(data.title !== undefined ? { title: data.title } : {}),
        ...(data.patientId !== undefined ? { patientId: data.patientId } : {}),
        ...(data.patientName !== undefined ? { patientName: data.patientName } : {}),
        ...(data.startTime !== undefined ? { startTime: new Date(data.startTime) } : {}),
        ...(data.endTime !== undefined ? { endTime: new Date(data.endTime) } : {}),
        ...(data.type !== undefined ? { type: data.type } : {}),
        ...(data.notes !== undefined ? { notes: data.notes } : {}),
        ...(data.status !== undefined ? { status: data.status } : {}),
        updatedAt: new Date(),
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'APPOINTMENT_UPDATED',
        entity: 'IATAppointment',
        entityId: updated.id,
        patientId: updated.patientId || null,
        details: `Appointment updated: ${updated.title} at ${updated.startTime}`,
      },
    }).catch(() => {})
    return NextResponse.json(updated)
  } catch (error) {
    console.error('PUT /api/iat-appointments/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
```

```
  }
}
export async function DELETE(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const existing = await prisma.iATAppointment.findFirst({
      where: { id, deletedAt: null },
    })
    if (!existing) {
      return NextResponse.json({ error: 'Appointment not found' }, { status: 404 })
    }
    await prisma.iATAppointment.update({
      where: { id },
      data: { deletedAt: new Date(), updatedAt: new Date() },
    })
    prisma.auditLog.create({
      data: {
        action: 'APPOINTMENT_DELETED',
        entity: 'IATAppointment',
        entityId: id,
        patientId: existing.patientId || null,
        details: `Appointment deleted: ${existing.title} at ${existing.startTime}`,
      },
    }).catch(() => {})
    return NextResponse.json({ ok: true })
  } catch (error) {
    console.error('DELETE /api/iat-appointments/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/iat-appointments/route.ts

```
/**
 * @file /api/iat-appointments — Appointment booking for IAT calendar
 * @description
 * GET ?date=YYYY-MM-DD — returns all appointments for that date's week (Mon-Sun),
 * or current week if no date supplied.
 * POST — create new appointment.
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
const createSchema = z.object({
  title: z.string().min(1).max(200),
  patientId: z.string().optional(),
  patientName: z.string().max(200).optional(),
  startTime: z.string().datetime(),
  endTime: z.string().datetime(),
  type: z.string().optional(),
  notes: z.string().max(1000).optional(),
  createdBy: z.string().max(200).optional(),
})
function getWeekRange(date: Date) {
  const d = new Date(date)
  const day = d.getDay() // 0=Sun
  const diffToMon = (day === 0 ? -6 : 1 - day)
  const monday = new Date(d)
```

```
  monday.setDate(d.getDate() + diffToMon)
  monday.setHours(0, 0, 0, 0)
  const sunday = new Date(monday)
  sunday.setDate(monday.getDate() + 6)
  sunday.setHours(23, 59, 59, 999)
  return { start: monday, end: sunday }
}
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const dateParam = searchParams.get('date')
    const base = dateParam ? new Date(dateParam) : new Date()
    const { start, end } = getWeekRange(base)
    const appointments = await prisma.iATAppointment.findMany({
      where: {
        deletedAt: null,
        startTime: { gte: start, lte: end },
      },
      orderBy: { startTime: 'asc' },
    })
    return NextResponse.json(appointments)
  } catch (error) {
    console.error('GET /api/iat-appointments error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const result = createSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const { title, patientId, patientName, startTime, endTime, type, notes, createdBy } = result.data
    // Validate time order
    if (new Date(startTime) >= new Date(endTime)) {
      return NextResponse.json({ error: 'endTime must be after startTime' }, { status: 400 })
    }
    const appointment = await prisma.iATAppointment.create({
      data: {
        title,
        patientId: patientId || '',
        patientName: patientName ?? null,
        startTime: new Date(startTime),
        endTime: new Date(endTime),
        type: type ?? 'allergy-test',
        notes: notes ?? null,
        createdBy: createdBy ?? null,
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'APPOINTMENT_CREATED',
        entity: 'IATAppointment',
        entityId: appointment.id,
        patientId: appointment.patientId || null,
        details: `Appointment: ${appointment.title} at ${appointment.startTime}`,
      },
    }).catch(() => {})
    return NextResponse.json(appointment, { status: 201 })
  } catch (error) {
    console.error('POST /api/iat-appointments error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
```

```
  }
}
```

route.ts

app/api/kiosk/lookup/route.ts

```
/**
 * @file /api/kiosk/lookup — Patient DOB lookup for kiosk check-in
 * @description
 * POST — Look up patients by date of birth.
 * Returns found=true with patient list (id + first initial only, no full PII) if found.
 * Returns found=false if no matching patients.
 * No auth required — kiosk-facing endpoint (runs on localhost touch screen).
 */
import { NextRequest, NextResponse } from 'next/server';
import prisma from '@/lib/db';
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders';

export const dynamic = 'force-dynamic';

const RATE_LIMIT_MAX = 10;
const RATE_LIMIT_WINDOW_MS = 60_000;

export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as { dob?: string };
    const { dob } = body;

    // Rate limiting: max 10 DOB lookups per IP per 60 seconds
    const ip =
      request.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ||
      request.headers.get('x-real-ip') ||
      'unknown';

    const windowStart = new Date(Date.now() - RATE_LIMIT_WINDOW_MS);
    const recentCount = await prisma.auditLog.count({
      where: {
        action: 'Kiosk DOB Lookup',
        details: ip,
        createdAt: { gte: windowStart },
      },
    });

    // Log this attempt (IP only — never log the DOB)
    await prisma.auditLog.create({
      data: {
        action: 'Kiosk DOB Lookup',
        entity: 'Patient',
        details: ip,
      },
    }).catch(() => { /* non-blocking */ });

    if (recentCount >= RATE_LIMIT_MAX) {
      return NextResponse.json(
        { error: 'Too many requests. Please wait before trying again.' },
        { status: 429, headers: HIPAA_HEADERS }
      );
    }

    if (!dob) {
      return NextResponse.json({ error: 'dob is required' }, { status: 400 });
    }

    // Parse the DOB — input is YYYY-MM-DD from the date picker
    const dobDate = new Date(dob + 'T00:00:00.000Z');
    if (isNaN(dobDate.getTime())) {
      return NextResponse.json({ error: 'Invalid date format' }, { status: 400 });
    }

    // Search ±1 day to handle timezone offset issues (SQLite stores UTC, input is local)
    const dayBefore = new Date(dobDate.getTime() - 86400000);
    const dayAfter = new Date(dobDate.getTime() + 86400000);

    const patients = await prisma.patient.findMany({
```

```
      where: {
        deletedAt: null,
        dob: { gte: dayBefore, lte: dayAfter },
      },
      select: {
        id: true,
        name: true,
        dob: true,
      },
    });
    if (patients.length === 0) {
      return NextResponse.json({ found: false }, { headers: HIPAA_HEADERS });
    }
    // Return minimal info — only first name for privacy (no last name, no DOB)
    const safePatients = patients.map(p => ({
      id: p.id,
      firstName: p.name.split(' ')[0],
      // DOB intentionally omitted — it was used only for lookup, not returned
    }));
    return NextResponse.json(
      { found: true, patients: safePatients },
      { headers: HIPAA_HEADERS }
    );
  } catch (error) {
    console.error('POST /api/kiosk/lookup error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

route.ts

app/api/kiosk/register/route.ts

```
/**
 * @file /api/kiosk/register — Register a new patient via kiosk
 * @description
 * POST — Creates a new patient record from the kiosk self-registration wizard.
 * Requires firstName, lastName, and dob.
 * No auth required — kiosk-facing endpoint.
 */
import { NextRequest, NextResponse } from 'next/server';
import prisma from '@/lib/db';
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders';
export const dynamic = 'force-dynamic';
export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as {
      firstName?: string;
      lastName?: string;
      phone?: string;
      email?: string;
      street?: string;
      apt?: string;
      city?: string;
      state?: string;
      zip?: string;
      insuranceProvider?: string;
      insuranceId?: string;
      dob?: string;
    };
    const { firstName, lastName, dob } = body;
    if (!firstName || !lastName || !dob) {
      return NextResponse.json(
        { error: 'firstName, lastName, and dob are required' },
        { status: 400 }
```

```
  );
}
// Input length validation
if (typeof firstName !== 'string' || firstName.trim().length < 1 || firstName.trim().length > 100) {
  return NextResponse.json({ error: 'firstName must be 1-100 characters' }, { status: 400 });
}
if (typeof lastName !== 'string' || lastName.trim().length < 1 || lastName.trim().length > 100) {
  return NextResponse.json({ error: 'lastName must be 1-100 characters' }, { status: 400 });
}
const dobDate = new Date(dob + 'T00:00:00.000Z');
if (isNaN(dobDate.getTime())) {
  return NextResponse.json({ error: 'Invalid date format' }, { status: 400 });
}
// Validate DOB is in a reasonable range (born 1900-present)
const minDate = new Date('1900-01-01')
const today = new Date()
if (dobDate < minDate || dobDate > today) {
  return NextResponse.json({ error: 'Invalid date of birth' }, { status: 400 });
}
const fullName = `${firstName.trim()} ${lastName.trim()}`;
const patientId = `PAT-${Date.now().toString(36).toUpperCase()}`;
const patient = await prisma.patient.create({
  data: {
    patientId,
    name: fullName,
    dob: dobDate,
    phone: body.phone?.trim() || null,
    email: body.email?.trim() || null,
    street: body.street?.trim() || null,
    apt: body.apt?.trim() || null,
    city: body.city?.trim() || null,
    state: body.state?.trim() || null,
    zip: body.zip?.trim() || null,
    insuranceProvider: body.insuranceProvider?.trim() || null,
    insuranceId: body.insuranceId?.trim() || null,
  },
  select: {
    id: true,
    patientId: true,
    name: true,
    dob: true,
  },
});
await prisma.auditLog.create({
  data: {
    action: 'KIOSK_REGISTER',
    entity: 'Patient',
    entityId: patient.id,
    patientId: patient.id,
    details: `New patient self-registered via kiosk: ${patientId} — ${fullName}`,
  },
}).catch(() => { /* non-blocking */ });
return NextResponse.json(
  {
    patient: {
      id: patient.id,
      patientId: patient.patientId,
      firstName: firstName.trim(),
      lastName: lastName.trim(),
      name: patient.name,
      dob: patient.dob,
    },
  },
  { status: 201, headers: HIPAA_HEADERS }
```

```
    );
  } catch (error) {
    console.error('POST /api/kiosk/register error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

route.ts

app/api/kiosk/update-patient/route.ts

```
/**
 * @file /api/kiosk/update-patient — Kiosk patient info update
 * @description Allows kiosk to update limited patient fields (phone, email, address, insurance).
 * Only updates specific safe fields — cannot change name, DOB, status, physician, etc.
 * No auth required (kiosk-facing) but validates patientId exists.
 * @security Public kiosk endpoint — field allowlist prevents privilege escalation
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

// Only these fields can be updated from the kiosk (strict allowlist)
const ALLOWED_FIELDS = ['phone', 'email', 'insuranceId', 'notes'] as const
type AllowedField = typeof ALLOWED_FIELDS[number]

export async function PUT(req: NextRequest) {
  try {
    const body = await req.json() as { patientId?: string } & Partial<Record<AllowedField, string>>
    const { patientId, ...updates } = body

    if (!patientId) {
      return NextResponse.json({ error: 'patientId is required' }, { status: 400 })
    }

    // Verify patient exists
    const patient = await prisma.patient.findFirst({
      where: { id: patientId, deletedAt: null },
      select: { id: true },
    })
    if (!patient) {
      return NextResponse.json({ error: 'Patient not found' }, { status: 404 })
    }

    // Only apply allowed fields
    const safeUpdates: Partial<Record<AllowedField, string>> = {}
    for (const field of ALLOWED_FIELDS) {
      if (field in updates && typeof updates[field] === 'string') {
        safeUpdates[field] = String(updates[field]).slice(0, 500) // cap length
      }
    }

    if (Object.keys(safeUpdates).length === 0) {
      return NextResponse.json({ ok: true, message: 'Nothing to update' })
    }

    await prisma.patient.update({
      where: { id: patientId },
      data: safeUpdates,
    })

    // Audit log (non-blocking)
    prisma.auditLog.create({
      data: {
        action: 'KIOSK_INFO_UPDATE',
        entity: 'Patient',
        entityId: patientId,
        patientId,
        details: `Kiosk updated fields: ${Object.keys(safeUpdates).join(', ')}`,
      },
    }).catch(() => {})
```

```
    return NextResponse.json({ ok: true }, { headers: HIPAA_HEADERS })
  } catch (err) {
    console.error('PUT /api/kiosk/update-patient error:', err)
    return NextResponse.json({ error: 'Failed to update patient' }, { status: 500 })
  }
}
```

route.ts

app/api/kiosk/verify/route.ts

```
/**
 * @file /api/kiosk/verify — Verify patient identity by first name
 * @description
 * POST — Verifies a patient's identity by comparing the entered first name
 * against the stored name (case-insensitive).
 * No auth required — kiosk-facing endpoint.
 */
import { NextRequest, NextResponse } from 'next/server';
import prisma from '@/lib/db';
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders';

export const dynamic = 'force-dynamic';

const VERIFY_RATE_LIMIT_MAX = 5;
const VERIFY_RATE_LIMIT_WINDOW_MS = 60_000;

export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as { patientId?: string; firstName?: string };
    const { patientId, firstName } = body;

    if (!patientId || !firstName) {
      return NextResponse.json({ error: 'patientId and firstName are required' }, { status: 400 });
    }

    // Rate limit verify attempts per IP to prevent name enumeration
    const ip =
      request.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ||
      request.headers.get('x-real-ip') ||
      'unknown';

    const windowStart = new Date(Date.now() - VERIFY_RATE_LIMIT_WINDOW_MS);
    const recentCount = await prisma.auditLog.count({
      where: {
        action: 'KIOSK_VERIFY_ATTEMPT',
        details: ip,
        createdAt: { gte: windowStart },
      },
    });

    // Log this attempt
    await prisma.auditLog.create({
      data: {
        action: 'KIOSK_VERIFY_ATTEMPT',
        entity: 'Patient',
        details: ip,
      },
    }).catch(() => { /* non-blocking */ });

    if (recentCount >= VERIFY_RATE_LIMIT_MAX) {
      return NextResponse.json(
        { error: 'Too many attempts. Please wait before trying again.' },
        { status: 429, headers: HIPAA_HEADERS }
      );
    }

    const patient = await prisma.patient.findFirst({
      where: { id: patientId, deletedAt: null },
      select: { id: true, name: true, patientId: true },
    });

    if (!patient) {
      return NextResponse.json({ verified: false, error: 'Patient not found' }, { headers: HIPAA_HEADERS });
```

```
    }
    // Case-insensitive first-name match
    const storedFirstName = patient.name.split(' ')[0].toLowerCase().trim();
    const enteredFirstName = firstName.toLowerCase().trim();
    if (storedFirstName !== enteredFirstName) {
      return NextResponse.json({ verified: false }, { headers: HIPAA_HEADERS });
    }
    // Log the kiosk check-in
    await prisma.auditLog.create({
      data: {
        action: 'KIOSK_CHECKIN',
        entity: 'Patient',
        entityId: patient.id,
        patientId: patient.id,
        details: `Kiosk identity verified for patient ${patient.patientId}`,
      },
    }).catch(() => { /* non-blocking */ });
    return NextResponse.json(
      {
        verified: true,
        patient: {
          id: patient.id,
          patientId: patient.patientId,
          firstName: patient.name.split(' ')[0],
          // Full name intentionally omitted from kiosk response — first name only for privacy
        },
      },
      { headers: HIPAA_HEADERS }
    );
  } catch (error) {
    console.error('POST /api/kiosk/verify error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

route.ts

app/api/kiosk/video-watched/route.ts

```
/**
 * @file /api/kiosk/video-watched — Record video completion from kiosk
 * @description
 * POST — Records that a patient watched a video during kiosk check-in.
 * Creates a VideoActivity record (upsert by patientId+videoId).
 * No auth required — kiosk-facing endpoint.
 */
import { NextRequest, NextResponse } from 'next/server';
import prisma from '@/lib/db';
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders';
export const dynamic = 'force-dynamic';
export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as {
      patientId?: string;
      videoId?: string;
      completed?: boolean;
    };
    const { patientId, videoId, completed = true } = body;
    if (!patientId || !videoId) {
      return NextResponse.json({ error: 'patientId and videoId are required' }, { status: 400 });
    }
    // Upsert: if they already have an activity record, update it; otherwise create
    const existing = await prisma.videoActivity.findFirst({
      where: { patientId, videoId },
```

```
    });
    let activity;
    if (existing) {
      activity = await prisma.videoActivity.update({
        where: { id: existing.id },
        data: { completed, watchedAt: new Date() },
      });
    } else {
      activity = await prisma.videoActivity.create({
        data: { patientId, videoId, completed, watchedAt: new Date() },
      });
    }
    return NextResponse.json({ success: true, activity }, { headers: HIPAA_HEADERS });
  } catch (error) {
    console.error('POST /api/kiosk/video-watched error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

route.ts

app/api/kiosk/videos-watched/route.ts

```
/**
 * @file /api/kiosk/videos-watched — Query which videos a patient has completed
 * @description
 * GET — Returns an array of videoIds the patient has marked completed.
 * Used by the verify step to determine if the patient can skip the videos screen.
 * No auth required — kiosk-facing endpoint.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url)
  const patientId = searchParams.get('patientId')
  if (!patientId) return NextResponse.json({ watchedIds: [] })

  try {
    const activities = await prisma.videoActivity.findMany({
      where: { patientId, completed: true },
      select: { videoId: true },
    })
    return NextResponse.json({ watchedIds: activities.map(a => a.videoId) }, { headers: HIPAA_HEADERS })
  } catch {
    return NextResponse.json({ watchedIds: [] })
  }
}
```

route.ts

app/api/locations/[id]/route.ts

```
/**
 * @file /api/locations/[id] — Single location operations
 * @description
 * GET — Fetch a single location by ID.
 * PUT — Update location fields (including active/deletedAt for deactivate).
 * DELETE — Soft delete (set deletedAt).
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'

export const dynamic = 'force-dynamic'

export async function GET(
  _request: NextRequest,
```

```
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const location = await prisma.location.findFirst({
      where: { id, deletedAt: null },
    })
    if (!location) {
      return NextResponse.json({ error: 'Location not found' }, { status: 404 })
    }
    return NextResponse.json(location)
  } catch (error) {
    console.error('GET /api/locations/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const body = await request.json() as {
      name?: string
      key?: string
      suite?: string | null
      street?: string
      city?: string
      state?: string
      zip?: string
      active?: boolean
    }
    const existing = await prisma.location.findFirst({ where: { id, deletedAt: null } })
    if (!existing) {
      return NextResponse.json({ error: 'Location not found' }, { status: 404 })
    }
    const location = await prisma.location.update({
      where: { id },
      data: {
        ...(body.name !== undefined ? { name: body.name } : {}),
        ...(body.key !== undefined ? { key: body.key } : {}),
        ...(body.suite !== undefined ? { suite: body.suite } : {}),
        ...(body.street !== undefined ? { street: body.street } : {}),
        ...(body.city !== undefined ? { city: body.city } : {}),
        ...(body.state !== undefined ? { state: body.state } : {}),
        ...(body.zip !== undefined ? { zip: body.zip } : {}),
        ...(body.active !== undefined ? {
          active: body.active,
          ...(body.active ? { deletedAt: null } : {}),
        } : {}),
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'LOCATION_UPDATED',
        entity: 'Location',
        entityId: location.id,
        patientId: null,
        details: `Location updated: ${location.name} (${location.key})`,
      },
    }).catch(() => {})
    return NextResponse.json(location)
  } catch (error) {
    console.error('PUT /api/locations/[id] error:', error)
```

```
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function DELETE(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const existing = await prisma.location.findFirst({ where: { id, deletedAt: null } })
    if (!existing) {
      return NextResponse.json({ error: 'Location not found' }, { status: 404 })
    }
    const location = await prisma.location.update({
      where: { id },
      data: { deletedAt: new Date(), active: false },
    })
    return NextResponse.json(location)
  } catch (error) {
    console.error('DELETE /api/locations/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/locations/route.ts

```
/**
 * @file /api/locations — Clinical location management
 * @description Manages clinic/office locations used for doctor and patient assignments.
 * GET — List all active locations sorted by name.
 * POST — Create a new location (name, key, street, city, state, zip required).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
const createLocationSchema = z.object({
  name: z.string().min(1).max(200),
  key: z.string().min(1).max(50).regex(/^[A-Z0-9-]+$/, 'Key must be uppercase alphanumeric'),
  suite: z.string().max(50).optional(),
  street: z.string().min(1).max(300),
  city: z.string().min(1).max(100),
  state: z.string().length(2, 'State must be 2 letters').toUpperCase(),
  zip: z.string().regex(/^\d{5}(-\d{4})?$/, 'Invalid ZIP code'),
})
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const all = searchParams.get('all') === '1'
    const locations = await prisma.location.findMany({
      where: {
        deletedAt: null,
        ...(all ? {} : { active: true }),
      },
      orderBy: { name: 'asc' },
    })
    return NextResponse.json(locations)
  } catch (error) {
    console.error('GET /api/locations error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
```

```
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const result = createLocationSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const { name, key, suite, street, city, state, zip } = result.data
    const location = await prisma.location.create({
      data: {
        name,
        key,
        suite,
        street,
        city,
        state,
        zip,
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'LOCATION_CREATED',
        entity: 'Location',
        entityId: location.id,
        patientId: null,
        details: `Location created: ${location.name} (${location.key})`,
      },
    }).catch(() => {})
    return NextResponse.json(location, { status: 201 })
  } catch (error) {
    console.error('POST /api/locations error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/nurses/[id]/route.ts

```
/**
 * @file /api/nurses/[id] — Single nurse operations
 * @description
 * GET — Fetch a single nurse by ID.
 * PUT — Update nurse fields.
 * DELETE — Soft delete (set deletedAt).
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function GET(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const nurse = await prisma.nurse.findFirst({
      where: { id, deletedAt: null },
    })
    if (!nurse) {
      return NextResponse.json({ error: 'Nurse not found' }, { status: 404 })
    }
    return NextResponse.json(nurse)
  } catch (error) {
```

```
    console.error('GET /api/nurses/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const body = await request.json() as {
      name?: string
      title?: string
      email?: string
      phone?: string
      clinicLocation?: string
      active?: boolean
    }
    const existing = await prisma.nurse.findFirst({ where: { id, deletedAt: null } })
    if (!existing) {
      return NextResponse.json({ error: 'Nurse not found' }, { status: 404 })
    }
    const nurse = await prisma.nurse.update({
      where: { id },
      data: {
        ...(body.name !== undefined ? { name: body.name } : {}),
        ...(body.title !== undefined ? { title: body.title } : {}),
        ...(body.email !== undefined ? { email: body.email } : {}),
        ...(body.phone !== undefined ? { phone: body.phone } : {}),
        ...(body.clinicLocation !== undefined ? { clinicLocation: body.clinicLocation } : {}),
        ...(body.active !== undefined ? { active: body.active } : {}),
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'NURSE_UPDATED',
        entity: 'Nurse',
        entityId: nurse.id,
        patientId: null,
        details: `Nurse updated: ${nurse.name}`,
      },
    }).catch(() => {})
    return NextResponse.json(nurse)
  } catch (error) {
    console.error('PUT /api/nurses/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function DELETE(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const existing = await prisma.nurse.findFirst({ where: { id, deletedAt: null } })
    if (!existing) {
      return NextResponse.json({ error: 'Nurse not found' }, { status: 404 })
    }
    await prisma.nurse.update({
      where: { id },
      data: { deletedAt: new Date(), active: false },
    })
    prisma.auditLog.create({
```

```
      data: {
        action: 'NURSE_DELETED',
        entity: 'Nurse',
        entityId: id,
        patientId: null,
        details: `Nurse deleted: ${existing.name}`,
      },
    }).catch(() => {})
    return NextResponse.json({ success: true })
  } catch (error) {
    console.error('DELETE /api/nurses/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/nurses/route.ts

```
/**
 * @file /api/nurses — Nurse directory
 * @description
 * GET — List nurses. Pass ?all=1 to include inactive.
 * POST — Create a new nurse record (name required).
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
const createNurseSchema = z.object({
  name: z.string().min(1).max(200),
  title: z.string().max(50).optional(),
  specialty: z.string().max(200).optional(),
  email: z.string().email().optional().or(z.literal('')),
  phone: z.string().max(20).optional(),
  clinicLocation: z.string().max(200).optional(),
})
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const all = searchParams.get('all') === '1'
    const nurses = await prisma.nurse.findMany({
      where: {
        deletedAt: null,
        ...(all ? {} : { active: true }),
      },
      orderBy: [{ name: 'asc' }],
    })
    return NextResponse.json(nurses)
  } catch (error) {
    console.error('GET /api/nurses error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const result = createNurseSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const { name, title, email, phone, clinicLocation } = result.data
    const nurse = await prisma.nurse.create({
      data: {
```

```
        name,
        ...(title ? { title } : {}),
        ...(email ? { email } : {}),
        ...(phone ? { phone } : {}),
        ...(clinicLocation ? { clinicLocation } : {}),
        ...(((body as { npi?: string }).npi) ? { npi: (body as { npi?: string }).npi } : {}),
      },
    })
    return NextResponse.json(nurse, { status: 201 })
  } catch (error) {
    console.error('POST /api/nurses error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/patients/[id]/route.ts

```
/**
 * @file /api/patients/[id] — Single patient record operations
 * @description Retrieves and updates a specific patient by internal ID.
 * GET — Fetch full patient record including test results, forms, videos, and audit logs.
 * PUT — Update patient demographic, clinical, or status fields (partial update supported).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

export async function GET(
  _request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params

    const patient = await prisma.patient.findUnique({
      where: { id },
      include: {
        doctor: true,
        dates: true,
        testResults: {
          include: { allergen: true, dates: true },
        },
        videoActivity: {
          include: { video: true },
        },
        formActivity: {
          include: { form: true },
        },
        auditLogs: {
          orderBy: { createdAt: 'desc' },
          take: 20,
        },
      },
    })
    if (!patient) {
      return NextResponse.json({ error: 'Patient not found', id }, { status: 404 })
    }
    return NextResponse.json(patient, { headers: HIPAA_HEADERS })
  } catch (error) {
    const err = error as Error
    console.error('GET /api/patients/[id] error:', err.message)
    return NextResponse.json({ error: err.message, id: 'unknown' }, { status: 500 })
  }
```

```
}
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const body = await request.json() as Record<string, string | number | boolean | null | undefined>
    const {
      name,
      dob,
      email,
      phone,
      physician,
      clinicLocation,
      diagnosis,
      notes,
      doctorId,
      status,
      insuranceId,
    } = body
    const patient = await prisma.patient.update({
      where: { id },
      data: {
        ...(name !== undefined ? { name: name as string } : {}),
        ...(dob !== undefined ? { dob: new Date(dob as string) } : {}),
        ...(email !== undefined ? { email: email as string } : {}),
        ...(phone !== undefined ? { phone: phone as string } : {}),
        ...(physician !== undefined ? { physician: physician as string } : {}),
        ...(clinicLocation !== undefined ? { clinicLocation: clinicLocation as string } : {}),
        ...(diagnosis !== undefined ? { diagnosis: diagnosis as string } : {}),
        ...(notes !== undefined ? { notes: notes as string } : {}),
        ...(doctorId !== undefined ? { doctorId: doctorId as string } : {}),
        ...(status !== undefined ? { status: status as string } : {}),
        ...(insuranceId !== undefined ? { insuranceId: insuranceId as string } : {}),
        ...(body.homePhone !== undefined ? { homePhone: body.homePhone as string } : {}),
        ...(body.street !== undefined ? { street: body.street as string } : {}),
        ...(body.apt !== undefined ? { apt: body.apt as string } : {}),
        ...(body.city !== undefined ? { city: body.city as string } : {}),
        ...(body.state !== undefined ? { state: body.state as string } : {}),
        ...(body.zip !== undefined ? { zip: body.zip as string } : {}),
        ...(body.insuranceProvider !== undefined ? { insuranceProvider: body.insuranceProvider as string } : {}),
        ...(body.insuranceGroup !== undefined ? { insuranceGroup: body.insuranceGroup as string } : {}),
        ...(body.emergencyName !== undefined ? { emergencyName: body.emergencyName as string } : {}),
        ...(body.emergencyPhone !== undefined ? { emergencyPhone: body.emergencyPhone as string } : {}),
        ...(body.emergencyRelation !== undefined ? { emergencyRelation: body.emergencyRelation as string } : {}),
      },
    })
    await prisma.auditLog.create({
      data: {
        action: 'UPDATE',
        entity: 'Patient',
        entityId: patient.id,
        patientId: patient.id,
        details: `Updated patient ${patient.patientId}`,
      },
    })
    return NextResponse.json(patient, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('PUT /api/patients/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/patients/route.ts

```
/**
 * @file /api/patients — Patient list and creation
 * @description Manages the patient roster.
 * GET — List all active patients; supports ?search= query param.
 * POST — Create a new patient record (name, dob required).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'

export const dynamic = 'force-dynamic'

const createPatientSchema = z.object({
  name: z.string().min(1, 'Name is required').max(200),
  dob: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'DOB must be YYYY-MM-DD'),
  email: z.string().email().optional().or(z.literal('')),
  phone: z.string().max(20).optional(),
  physician: z.string().max(200).optional(),
  clinicLocation: z.string().max(200).optional(),
  diagnosis: z.string().max(500).optional(),
  insuranceId: z.string().max(100).optional(),
  notes: z.string().max(2000).optional(),
})

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const search = searchParams.get('search')

    const patients = await prisma.patient.findMany({
      where: {
        deletedAt: null,
        // Only show IAT patients (patientId starts with PAT-)
        patientId: { startsWith: 'PAT-' },
        ...(search
          ? {
              OR: [
                { name: { contains: search } },
                { patientId: { contains: search } },
                { email: { contains: search } },
              ],
            }
          : {}),
      },
      select: {
        id: true,
        patientId: true,
        name: true,
        dob: true,
        status: true,
        doctorId: true,
        clinicLocation: true,
        physician: true,
      },
      orderBy: [{ name: 'asc' }],
    })

    return NextResponse.json(patients, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('GET /api/patients error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}

export async function POST(request: NextRequest) {
```

```
  try {
    const body = await request.json()
    const result = createPatientSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const { name, dob } = result.data
    const patientId = `PAT-${Date.now().toString(36).toUpperCase()}`
    const patient = await prisma.patient.create({
      data: {
        patientId,
        name,
        dob: new Date(dob),
        email: result.data.email,
        phone: result.data.phone,
        homePhone: (body as Record<string, string>).homePhone || null,
        street: (body as Record<string, string>).street || null,
        city: (body as Record<string, string>).city || null,
        state: (body as Record<string, string>).state || null,
        zip: (body as Record<string, string>).zip || null,
        physician: result.data.physician,
        clinicLocation: result.data.clinicLocation,
        diagnosis: result.data.diagnosis,
        insuranceId: result.data.insuranceId,
        insuranceProvider: (body as Record<string, string>).insuranceProvider || null,
        insuranceGroup: (body as Record<string, string>).insuranceGroup || null,
        emergencyName: (body as Record<string, string>).emergencyName || null,
        emergencyPhone: (body as Record<string, string>).emergencyPhone || null,
        emergencyRelation: (body as Record<string, string>).emergencyRelation || null,
        notes: result.data.notes,
        doctorId: (body as { doctorId?: string }).doctorId,
      },
    })
    await prisma.auditLog.create({
      data: {
        action: 'CREATE',
        entity: 'Patient',
        entityId: patient.id,
        patientId: patient.id,
        details: `Created patient ${patientId}: ${name}`,
      },
    })
    return NextResponse.json(patient, { status: 201, headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('POST /api/patients error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/seed/route.ts

```
/**
 * @file /api/seed — Full database seeder
 * @description Seeds the database with initial reference data for a fresh deployment.
 * POST — Creates location, doctor, sample videos, form templates, and allergens. Idempotent.
 * @security Public route — used during initial setup only
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
// GET is disabled in production — only allow in development
export async function GET(request: NextRequest) {
```

```
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json({ error: 'Not available in production' }, { status: 403 })
  }
  return POST(request)
}
export async function POST(request: NextRequest) {
  // Block in production — seed endpoints must not be accessible publicly
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json({ error: 'Not available in production' }, { status: 403 })
  }
  try {
    // Check if DB is already seeded
    const locationCount = await prisma.location.count()
    if (locationCount > 0) {
      return NextResponse.json({
        message: 'Database already seeded',
        skipped: true,
      })
    }
    // 1. Create location
    const location = await prisma.location.create({
      data: {
        name: 'Integrated Allergy Testing',
        key: 'IAT-001',
        street: '123 Medical Drive',
        city: 'Dumfries',
        state: 'VA',
        zip: '22026',
      },
    })
    // 2. Create doctor
    await prisma.doctor.create({
      data: {
        name: 'Dr. Robert Sikora',
        clinicLocation: location.name,
      },
    })
    // 3. Create videos
    await prisma.video.createMany({
      data: [
        {
          title: 'Welcome to Integrated Allergy',
          description: 'An introduction to Integrated Allergy Testing and our services.',
          url: 'https://example.com/videos/welcome',
          category: 'what',
          order: 1,
        },
        {
          title: 'Understanding Allergy Testing',
          description: 'Learn about the science behind allergy testing and what to expect.',
          url: 'https://example.com/videos/understanding',
          category: 'why',
          order: 2,
        },
        {
          title: 'What to Expect During Testing',
          description: 'A step-by-step walkthrough of the allergy testing process.',
          url: 'https://example.com/videos/expect',
          category: 'how',
          order: 3,
        },
      ],
    })
    // 4. Create forms
```

```
    await prisma.form.createMany({
      data: [
        {
          name: 'Patient Consent Form',
          type: 'consent',
          template: '<h1>Patient Consent Form</h1><p>I hereby consent to allergy testing procedures as
recommended by my physician...</p>',
        },
        {
          name: 'Allergy Testing Form',
          type: 'testing',
          template: '<h1>Allergy Testing Form</h1><p>Please complete all sections of this form prior to your
allergy testing appointment...</p>',
        },
      ],
    })
    // 5. Seed allergens via the allergens/seed route
    const baseUrl = request.nextUrl.origin
    const allergenSeedResponse = await fetch(`${baseUrl}/api/allergens/seed`, {
      method: 'POST',
    })
    const allergenResult = await allergenSeedResponse.json() as { count?: number; message?: string }
    return NextResponse.json({
      message: 'Database seeded successfully',
      location: location.name,
      allergens: allergenResult,
    }, { status: 201 })
  } catch (error) {
    console.error('POST /api/seed error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/staff/[id]/reset-mfa/route.ts

```
/**
 * @file /api/staff/[id]/reset-mfa — Admin MFA reset
 * @description Allows an admin to reset MFA for a staff user, forcing them to re-enroll
 * on next login.
 * @security Requires authenticated session with admin role
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { verifySession } from '@/lib/auth/session'
export const dynamic = 'force-dynamic'
export async function POST(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  const session = await verifySession(req)
  if (!session) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }
  if (session.role !== 'admin') {
    return NextResponse.json({ error: 'Forbidden: admin only' }, { status: 403 })
  }
  const { id } = await params
  try {
    const user = await prisma.staffUser.findUnique({ where: { id } })
    if (!user) {
      return NextResponse.json({ error: 'User not found' }, { status: 404 })
    }
```

```
    await prisma.staffUser.update({
      where: { id },
      data: {
        mfaEnabled: false,
        mfaSecret: null,
        tempToken: null,
        tempTokenExpiry: null,
      },
    })
    await prisma.auditLog.create({
      data: {
        action: 'MFA_RESET',
        entity: 'StaffUser',
        entityId: id,
        details: `MFA reset by admin ${session.email as string} for user ${user.email}`,
      },
    })
    return NextResponse.json({ success: true, message: `MFA reset for ${user.email}` })
  } catch (error) {
    console.error('POST /api/staff/[id]/reset-mfa error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/staff/route.ts

```
/**
 * @file /api/staff — Staff user administration
 * @description Admin-only endpoints for managing staff accounts.
 * GET — List all staff users (admin only).
 * POST — Create a new staff user with email, name, password, and optional role (admin only).
 * @security Requires authenticated session with admin role (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import bcrypt from 'bcryptjs'
import prisma from '@/lib/db'
import { verifySession } from '@/lib/auth/session'
export const dynamic = 'force-dynamic'
export async function POST(req: NextRequest) {
  // Verify session and admin role
  const session = await verifySession(req)
  if (!session) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }
  if (session.role !== 'admin') {
    return NextResponse.json({ error: 'Forbidden: admin only' }, { status: 403 })
  }
  try {
    const body = await req.json() as { email?: string; name?: string; password?: string; role?: string }
    const { email, name, password, role } = body
    if (!email || !name || !password) {
      return NextResponse.json({ error: 'email, name, and password are required' }, { status: 400 })
    }
    // Check for existing user
    const existing = await prisma.staffUser.findUnique({ where: { email } })
    if (existing) {
      return NextResponse.json({ error: 'A user with that email already exists' }, { status: 409 })
    }
    const passwordHash = await bcrypt.hash(password, 12)
    const user = await prisma.staffUser.create({
      data: {
        email,
```

```
        name,
        passwordHash,
        role: role === 'admin' ? 'admin' : 'staff',
      },
      select: { id: true, email: true, name: true, role: true, createdAt: true },
    })
    await prisma.auditLog.create({
      data: {
        action: 'STAFF_CREATED',
        entity: 'StaffUser',
        entityId: user.id,
        details: `Created by admin ${session.email as string}`,
      },
    })
    return NextResponse.json(user, { status: 201 })
  } catch (error) {
    console.error('POST /api/staff error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function GET(req: NextRequest) {
  const session = await verifySession(req)
  if (!session) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }
  if (session.role !== 'admin') {
    return NextResponse.json({ error: 'Forbidden: admin only' }, { status: 403 })
  }
  try {
    const users = await prisma.staffUser.findMany({
      select: { id: true, email: true, name: true, role: true, active: true, createdAt: true },
      orderBy: { createdAt: 'desc' },
    })
    return NextResponse.json(users)
  } catch (error) {
    console.error('GET /api/staff error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/test-results/[id]/route.ts

```
/**
 * @file /api/test-results/[id] — Update a single allergy test result
 * @description Partial update of a test result record (reaction, wheal, notes, readAt).
 * PUT — Update test result fields by internal ID.
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params
    const body = await request.json() as {
      reaction?: number
      wheal?: string
      notes?: string
      nurseName?: string
```

```
      readAt?: string
    }
    const { reaction, wheal, notes, nurseName, readAt } = body
    const testResult = await prisma.allergyTestResult.update({
      where: { id },
      data: {
        ...(reaction !== undefined ? { reaction } : {}),
        ...(wheal !== undefined ? { wheal } : {}),
        ...(notes !== undefined ? { notes } : {}),
        ...(nurseName !== undefined ? { nurseName } : {}),
        ...(readAt !== undefined ? { readAt: new Date(readAt) } : {}),
      },
      include: { allergen: true },
    })
    await prisma.auditLog.create({
      data: {
        action: 'UPDATE',
        entity: 'AllergyTestResult',
        entityId: testResult.id,
        patientId: testResult.patientId,
        details: `Updated test result for allergen ${testResult.allergenId}`,
      },
    })
    return NextResponse.json(testResult, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('PUT /api/test-results/[id] error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/test-results/route.ts

```
/**
 * @file /api/test-results — Allergy test result list and creation
 * @description Manages allergy test results across patients.
 * GET — List test results; supports ?patientId= filter.
 * POST — Record a new test result (patientId, allergenId, testType, reaction required).
 * testType must be 'scratch' or 'intradermal'; reaction must be 0–5.
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'
const createTestResultSchema = z.object({
  patientId: z.string().min(1),
  allergenId: z.string().min(1),
  testType: z.enum(['scratch', 'intradermal']),
  reaction: z.number().int().min(0).max(5),
  wheal: z.string().max(20).optional(),
  notes: z.string().max(1000).optional(),
  nurseName: z.string().max(200).optional(),
})
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const patientId = searchParams.get('patientId')
    const testResults = await prisma.allergyTestResult.findMany({
      where: {
        active: true,
        ...(patientId ? { patientId } : {}),
```

```
      },
      include: {
        allergen: true,
        patient: {
          select: {
            id: true,
            patientId: true,
            name: true,
          },
        },
      },
      orderBy: { testedAt: 'desc' },
    })
    return NextResponse.json(testResults, { headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('GET /api/test-results error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const result = createTestResultSchema.safeParse(body)
    if (!result.success) {
      return NextResponse.json({ error: result.error.issues[0].message }, { status: 400 })
    }
    const { patientId, allergenId, testType, reaction, wheal, notes, nurseName } = result.data
    const testResult = await prisma.allergyTestResult.create({
      data: {
        patientId,
        allergenId,
        testType,
        reaction,
        wheal,
        notes,
        nurseName,
      },
      include: { allergen: true },
    })
    await prisma.auditLog.create({
      data: {
        action: 'CREATE',
        entity: 'AllergyTestResult',
        entityId: testResult.id,
        patientId,
        details: `Created ${testType} test result for allergen ${allergenId}, reaction: ${reaction}`,
      },
    })
    return NextResponse.json(testResult, { status: 201, headers: HIPAA_HEADERS })
  } catch (error) {
    console.error('POST /api/test-results error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/videos/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function PUT(req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
```

```
    const { id } = await params
    const body = await req.json() as { title?: string; url?: string | null; description?: string | null;
category?: string | null; duration?: string | null }
    const video = await prisma.video.update({
      where: { id },
      data: {
        ...(body.title !== undefined && { title: body.title }),
        ...(body.url !== undefined && { url: body.url ?? '' }),
        ...(body.description !== undefined && { description: body.description }),
        ...(body.category !== undefined && { category: body.category }),
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'VIDEO_UPDATED',
        entity: 'Video',
        entityId: video.id,
        patientId: null,
        details: `Video updated: ${video.title}`,
      },
    }).catch(() => {})
    return NextResponse.json({ video })
  } catch (error) {
    console.error('PUT /api/videos/[id] error:', error)
    return NextResponse.json({ error: 'Failed to update video' }, { status: 500 })
  }
}
```

route.ts

app/api/videos/activity/route.ts

```
/**
 * @file /api/videos/activity — Patient video watch activity tracking
 * @description Records video interactions (watched, completed, acknowledged) for a patient.
 * POST — Create a video activity record (patientId and videoId required).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as {
      patientId?: string
      videoId?: string
      completed?: boolean
      acknowledged?: boolean
    }
    const { patientId, videoId } = body
    if (!patientId || !videoId) {
      return NextResponse.json(
        { error: 'patientId and videoId are required' },
        { status: 400 }
      )
    }
    const activity = await prisma.videoActivity.create({
      data: {
        patientId,
        videoId,
        completed: body.completed ?? false,
        acknowledged: body.acknowledged ?? false,
      },
      include: {
```

```
        video: true,
        patient: {
          select: {
            id: true,
            patientId: true,
            name: true,
          },
        },
      },
    })
    await prisma.auditLog.create({
      data: {
        action: 'VIDEO_ACTIVITY',
        entity: 'VideoActivity',
        entityId: activity.id,
        patientId,
        details: `Patient ${patientId} watched video ${videoId}; completed=${activity.completed};
acknowledged=${activity.acknowledged}`,
      },
    })
    return NextResponse.json(activity, { status: 201 })
  } catch (error) {
    console.error('POST /api/videos/activity error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/videos/route.ts

```
/**
 * @file /api/videos — Patient education video library
 * @description Manages the video library used in the patient kiosk education workflow.
 * GET — Return all active videos ordered by display order.
 * POST — Create a new video entry (title and url required).
 * @security Requires authenticated session (iat_session cookie via proxy.ts)
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
export const dynamic = 'force-dynamic'
export async function GET() {
  try {
    const videos = await prisma.video.findMany({
      where: { active: true },
      orderBy: { order: 'asc' },
    })
    return NextResponse.json(videos)
  } catch (error) {
    console.error('GET /api/videos error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
export async function POST(request: NextRequest) {
  try {
    const body = await request.json() as {
      title?: string
      description?: string
      url?: string
      duration?: number
      category?: string
      order?: number
    }
    const { title, url } = body
```

```
    if (!title || !url) {
      return NextResponse.json(
        { error: 'title and url are required' },
        { status: 400 }
      )
    }
    const video = await prisma.video.create({
      data: {
        title,
        description: body.description,
        url,
        duration: body.duration,
        category: body.category,
        order: body.order ?? 0,
      },
    })
    prisma.auditLog.create({
      data: {
        action: 'VIDEO_CREATED',
        entity: 'Video',
        entityId: video.id,
        patientId: null,
        details: `Video created: ${video.title}`,
      },
    }).catch(() => {})
    return NextResponse.json(video, { status: 201 })
  } catch (error) {
    console.error('POST /api/videos error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

route.ts

app/api/waiting-room/[id]/route.ts

```
/**
 * @file /api/waiting-room/[id] — Individual waiting-room entry management
 * @description
 * PUT — Updates a waiting-room entry status, nurse assignment, or video ack.
 * DELETE — Removes an entry from the waiting room (discharge/dismiss).
 * Staff-facing endpoint.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
export const dynamic = 'force-dynamic'
const VALID_STATUSES = ['waiting', 'in-service', 'complete', 'cancelled']
export async function PUT(req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
    const { id } = await params
    const body = await req.json() as { status?: string; nurseName?: string; nurseId?: string; videoAckBy?: string
}
    // Validate status if provided
    if (body.status !== undefined && !VALID_STATUSES.includes(body.status)) {
      return NextResponse.json({ error: `status must be one of: ${VALID_STATUSES.join(', ')}` }, { status: 400,
headers: HIPAA_HEADERS })
    }
    const data: Record<string, unknown> = {}
    if (body.status) data.status = body.status
    if (body.nurseName) data.nurseName = String(body.nurseName).slice(0, 200)
    if (body.nurseId) data.nurseId = body.nurseId
    if (body.videoAckBy) { data.videoAckBy = body.videoAckBy; data.videoAckAt = new Date() }
    if (body.status === 'in-service') data.calledAt = new Date()
```

```
    if (body.status === 'complete') data.completedAt = new Date()
    const entry = await prisma.waitingRoom.update({ where: { id }, data })
    return NextResponse.json({ entry }, { headers: HIPAA_HEADERS })
  } catch (err) {
    console.error('PUT /api/waiting-room/[id] error:', err)
    return NextResponse.json({ error: 'Failed to update' }, { status: 500 })
  }
}
export async function DELETE(_req: NextRequest, { params }: { params: Promise<{ id: string }> }) {
  try {
    const { id } = await params
    // Soft delete — mark as complete rather than removing
    await prisma.waitingRoom.update({ where: { id }, data: { status: 'complete', completedAt: new Date() } })
    return NextResponse.json({ ok: true }, { headers: HIPAA_HEADERS })
  } catch (err) {
    console.error('DELETE /api/waiting-room/[id] error:', err)
    return NextResponse.json({ error: 'Failed to delete' }, { status: 500 })
  }
}
```

route.ts

app/api/waiting-room/route.ts

```
/**
 * @file /api/waiting-room — Waiting room management
 * @description
 * GET — Returns all active waiting-room entries (status: waiting | in-service).
 * POST — Adds a patient to the waiting room after kiosk check-in completes.
 * Staff-facing (GET) and kiosk-facing (POST) endpoint.
 */
import { NextRequest, NextResponse } from 'next/server'
import prisma from '@/lib/db'
import { HIPAA_HEADERS } from '@/lib/hipaaHeaders'
import { verifySession } from '@/lib/auth/session'
export const dynamic = 'force-dynamic'
export async function GET(req: NextRequest) {
  // Staff-only: requires authenticated session
  const session = await verifySession(req)
  if (!session) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401, headers: HIPAA_HEADERS })
  }
  try {
    const entries = await prisma.waitingRoom.findMany({
      where: { status: { in: ['waiting', 'in-service'] } },
      orderBy: { checkedInAt: 'asc' },
    })
    return NextResponse.json({ entries }, { headers: HIPAA_HEADERS })
  } catch (err) {
    console.error('GET /api/waiting-room error:', err)
    return NextResponse.json({ entries: [] })
  }
}
export async function POST(req: NextRequest) {
  try {
    const body = await req.json() as { patientId?: string; patientName?: string; notes?: string; videosWatched?:
number }
    // Validate required fields
    if (!body.patientId || !body.patientName) {
      return NextResponse.json({ error: 'patientId and patientName are required' }, { status: 400, headers:
HIPAA_HEADERS })
    }
    // Validate that the patientId actually exists in the database to prevent spoofing
    const patient = await prisma.patient.findFirst({
```

```
      where: { id: body.patientId, deletedAt: null },
      select: { id: true, name: true },
    })
    if (!patient) {
      return NextResponse.json({ error: 'Patient not found' }, { status: 404, headers: HIPAA_HEADERS })
    }
    // Dedup: if patient already has an active (waiting/in-service) entry, return it
    const existing = await prisma.waitingRoom.findFirst({
      where: { patientId: patient.id, status: { in: ['waiting', 'in-service'] } },
    })
    if (existing) {
      return NextResponse.json({ entry: existing, deduplicated: true }, { headers: HIPAA_HEADERS })
    }
    // Sanitize inputs and use verified name from DB, not user-provided name
    const videosWatched = typeof body.videosWatched === 'number' && body.videosWatched >= 0
      ? Math.floor(body.videosWatched)
      : 0
    const entry = await prisma.waitingRoom.create({
      data: {
        patientId: patient.id,
        patientName: patient.name, // Always use DB name, not user-supplied name
        notes: body.notes ? String(body.notes).slice(0, 500) : undefined,
        videosWatched,
        status: 'waiting',
      },
    })
    return NextResponse.json({ entry }, { status: 201, headers: HIPAA_HEADERS })
  } catch (err) {
    console.error('POST /api/waiting-room error:', err)
    return NextResponse.json({ error: 'Failed to add to waiting room' }, { status: 500 })
  }
}
```

■■ app/ (pages & layouts) ■■

layout.tsx

app/layout.tsx

```
'use client';
import './globals.css';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { useState, useEffect } from 'react';
const navItems = [
  { href: '/', label: 'Dashboard', icon: '■' },
  { href: '/patients', label: 'Patients', icon: '■' },
  { href: '/testing', label: 'Testing', icon: '■' },
  { href: '/calendar', label: 'Calendar', icon: '■' },
  { href: '/encounters', label: 'Encounters', icon: '■' },
  { href: '/doctors', label: 'Doctors', icon: '■■■■' },
  { href: '/nurses', label: 'Nurses', icon: '■■■■' },
  { href: '/locations', label: 'Locations', icon: '■' },
  { href: '/videos', label: 'Videos', icon: '■' },
  { href: '/kiosk', label: 'Kiosk', icon: '■' },
  { href: '/settings', label: 'Settings', icon: '■■' },
];
function UserCard() {
  const [user, setUser] = useState<{ name: string; email: string; role: string } | null>(null);
  useEffect(() => {
    fetch('/api/auth/me').then(r => r.ok ? r.json() : null).then(d => {
      if (d?.user) setUser(d.user);
    }).catch(() => {});
```

```
  }, []);
  if (!user) return null;
  return (
    <div style={{ margin: '0 8px 8px', padding: '10px 12px', background: 'linear-gradient(135deg, #e8f9f7,
#d0f4ef)', border: '1.5px solid #2ec4b6', borderRadius: 12 }}>
      <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
        <div style={{ width: 32, height: 32, borderRadius: '50%', background: '#2ec4b6', display: 'flex',
alignItems: 'center', justifyContent: 'center', color: '#fff', fontWeight: 700, fontSize: 14 }}>
          {user.name.charAt(0).toUpperCase()}
        </div>
        <div style={{ flex: 1, minWidth: 0 }}>
          <div style={{ fontWeight: 700, fontSize: 13, color: '#0d9488', overflow: 'hidden', textOverflow: '
ellipsis', whiteSpace: 'nowrap' }}>{user.name}</div>
          <div style={{ fontSize: 11, color: '#6b7280', textTransform: 'capitalize' }}>{user.role}</div>
        </div>
      </div>
      <button onClick={async () => {
          await fetch('/api/auth/logout', { method: 'POST' }).catch(() => {});
          // Also sign out of NextAuth (SSO sessions)
          window.location.href = '/api/auth/signout?callbackUrl=/login';
        }}
        style={{ marginTop: 8, width: '100%', padding: '4px 0', fontSize: 11, color: '#dc2626', background: '
transparent', border: '1px solid #fca5a5', borderRadius: 6, cursor: 'pointer', fontWeight: 600 }}>
        Sign Out
      </button>
    </div>
  );
}
function Sidebar({ open, onClose }: { open: boolean; onClose: () => void }) {
  const pathname = usePathname();
  return (
    <>
      <div className={`sidebar-overlay ${open ? 'open' : ''}`} onClick={onClose} />
      <nav className={`sidebar ${open ? 'open' : ''}`}>
        <div className="sidebar-logo">
          {/* eslint-disable-next-line @next/next/no-img-element */}
          <img
            src="/integrated-allergy-logo.jpg"
            alt="Integrated Allergy Testing"
            style={{ height: 55, width: 'auto', display: 'block' }}
          />
          <div style={{ color: '#9ca3af', fontSize: 10, marginTop: 4 }}>Testing Suite v3.1.0</div>
        </div>
        <div className="sidebar-nav">
          <div className="nav-label">Navigation</div>
          {navItems.map((item) => {
            const isActive =
              item.href === '/'
                ? pathname === '/'
                : pathname.startsWith(item.href);
            // Kiosk opens in a new tab
            if (item.href === '/kiosk') {
              return (
                <button
                  key={item.href}
                  className="nav-link"
                  style={{ width: '100%', background: 'none', border: 'none', textAlign: 'left', cursor: '
pointer', padding: 0 }}
                  onClick={() => {
                    onClose();
                    // Open kiosk as a popup with no toolbar/controls
                    const w = window.open(
                      '/kiosk',
                      'kiosk',
```

```
                      `width=${screen.width},height=${screen.height},left=0,top=0,toolbar=no,menubar=no,scrollbars
=no,status=no,location=no`
                    );
                    if (w) {
                      w.moveTo(0, 0);
                      w.resizeTo(screen.width, screen.height);
                      w.addEventListener('load', () => {
                        w.document.documentElement.requestFullscreen?.().catch(() => {});
                      });
                    }
                  }}
                >
                  <span className="nav-icon">{item.icon}</span>
                  {item.label}
                  <span style={{ fontSize: 10, opacity: 0.6, marginLeft: 4, background: '#0d9488', color: '#fff',
padding: '1px 5px', borderRadius: 4 }}>LAUNCH</span>
                </button>
              );
            }
            return (
              <Link
                key={item.href}
                href={item.href}
                className={`nav-link ${isActive ? 'active' : ''}`}
                onClick={onClose}
              >
                <span className="nav-icon">{item.icon}</span>
                {item.label}
              </Link>
            );
          })}
        </div>
        <UserCard />
        {/* Sign Out — above the footer line */}
        <div style={{ padding: '0 12px 10px' }}>
          <button
            onClick={async () => {
              await fetch('/api/auth/logout', { method: 'POST' }).catch(() => {});
              window.location.href = '/api/auth/signout?callbackUrl=/login';
            }}
            style={{
              width: '100%',
              padding: '8px 0',
              background: 'rgba(255,255,255,0.1)',
              color: '#ffffff',
              border: '1px solid rgba(255,255,255,0.25)',
              borderRadius: 8,
              fontSize: 12,
              fontWeight: 600,
              cursor: 'pointer',
            }}
          >
            ■ Sign Out
          </button>
        </div>
        <div className="sidebar-footer">
          © 2026 Integrated Allergy Testing
        </div>
      </nav>
    </>
  );
}
function AppShell({ children }: { children: React.ReactNode }) {
  const [sidebarOpen, setSidebarOpen] = useState(false);
```

```
  const pathname = usePathname();
  const isAuthPage = pathname === '/login' || pathname?.startsWith('/login') || pathname === '/consent' ||
pathname?.startsWith('/consent') || pathname?.startsWith('/kiosk');
  // Close sidebar on route change — deferred to avoid synchronous setState-in-effect
  useEffect(() => { Promise.resolve().then(() => setSidebarOpen(false)); }, [pathname]);
  // Auto-refresh JWT every 6 hours (expires at 8h) — silent token rotation
  useEffect(() => {
    if (isAuthPage) return;
    const interval = setInterval(() => {
      fetch('/api/auth/refresh', { method: 'POST' }).catch(() => {});
    }, 6 * 60 * 60 * 1000);
    return () => clearInterval(interval);
  }, [isAuthPage]);
  if (isAuthPage) {
    return <>{children}</>;
  }
  return (
    <div className="app-shell">
      <button className="sidebar-toggle" onClick={() => setSidebarOpen((v) => !v)} aria-label="Toggle
navigation">■</button>
      <Sidebar open={sidebarOpen} onClose={() => setSidebarOpen(false)} />
      <div className="main-content">{children}</div>
    </div>
  );
}
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <head>
        <meta name="viewport" content="width=device-width, initial-scale=1" />
        <title>Integrated Allergy Testing</title>
      </head>
      <body>
        <AppShell>{children}</AppShell>
      </body>
    </html>
  );
}
```

page.tsx

app/page.tsx

```
'use client';
import { useEffect, useState, useCallback } from 'react';
import Link from 'next/link';
interface TodayAppointment {
  id: string;
  title: string;
  patientName?: string | null;
  patientId?: string | null;
  startTime: string;
  endTime: string;
  type: string;
  status: string;
  notes?: string | null;
  reasonName?: string | null;
}
const APPT_TYPE_COLORS: Record<string, { bg: string; text: string }> = {
  'allergy-test': { bg: '#e8f9f7', text: '#0d9488' },
  'consultation': { bg: '#eff6ff', text: '#1d4ed8' },
  'follow-up': { bg: '#f5f3ff', text: '#7c3aed' },
};
function formatApptTime(iso: string) {
```

```
  const d = new Date(iso);
  const h = d.getHours() % 12 || 12;
  const m = d.getMinutes();
  const ampm = d.getHours() >= 12 ? 'pm' : 'am';
  return m > 0 ? `${h}:${String(m).padStart(2,'0')}${ampm}` : `${h}${ampm}`;
}
interface WaitingEntry {
  id: string;
  patientId: string;
  patientName: string;
  status: 'waiting' | 'in-service' | 'complete';
  checkedInAt: string;
  calledAt?: string;
  nurseName?: string;
  notes?: string;
  videosWatched?: number;
  videoAckBy?: string;
  videoAckAt?: string;
}
interface Nurse { id: string; name: string; title?: string; }
export default function DashboardPage() {
  const [patientCount, setPatientCount] = useState<number | null>(null);
  const [nurseCount, setNurseCount] = useState<number | null>(null);
  const [userName, setUserName] = useState('');
  const [loading, setLoading] = useState(true);
  const [waiting, setWaiting] = useState<WaitingEntry[]>([]);
  const [nurses, setNurses] = useState<Nurse[]>([]);
  const [updatingId, setUpdatingId] = useState<string | null>(null);
  const [todayAppts, setTodayAppts] = useState<TodayAppointment[]>([]);
  const [selectedAppt, setSelectedAppt] = useState<TodayAppointment | null>(null);
  const [editingAppt, setEditingAppt] = useState<TodayAppointment | null>(null);
  const [editApptTitle, setEditApptTitle] = useState('');
  const [editApptDate, setEditApptDate] = useState('');
  const [editApptStartHour, setEditApptStartHour] = useState('09');
  const [editApptStartMin, setEditApptStartMin] = useState('00');
  const [editApptEndHour, setEditApptEndHour] = useState('10');
  const [editApptEndMin, setEditApptEndMin] = useState('00');
  const [editApptPatientId, setEditApptPatientId] = useState('');
  const [editApptPatientName, setEditApptPatientName] = useState('');
  const [editApptReasonId, setEditApptReasonId] = useState('');
  const [editApptNotes, setEditApptNotes] = useState('');
  const [editApptPatients, setEditApptPatients] = useState<{id: string; name: string}[]>([]);
  const [editApptReasons, setEditApptReasons] = useState<{id: string; name: string; color: string}[]>([]);
  const [savingAppt, setSavingAppt] = useState(false);
  const [deletingApptId, setDeletingApptId] = useState<string | null>(null);
  const [showAddApptModal, setShowAddApptModal] = useState(false);
  const [newApptTitle, setNewApptTitle] = useState('');
  const [newApptTime, setNewApptTime] = useState('09:00');
  const [newApptEndTime, setNewApptEndTime] = useState('10:00');
  const [addingAppt, setAddingAppt] = useState(false);
  const loadWaiting = useCallback(async () => {
    try {
      const r = await fetch('/api/waiting-room');
      const d = await r.json();
      setWaiting(d.entries ?? []);
    } catch {}
  }, []);
  useEffect(() => {
    async function loadData() {
      try {
        const [patientsRes, , nursesRes, meRes] = await Promise.allSettled([
          fetch('/api/patients'), fetch('/api/doctors'), fetch('/api/nurses'), fetch('/api/auth/me'),
        ]);
        if (patientsRes.status === 'fulfilled' && patientsRes.value.ok) {
```

```
          const d = await patientsRes.value.json();
          setPatientCount((Array.isArray(d) ? d : d.patients ?? []).length);
        }
        if (nursesRes.status === 'fulfilled' && nursesRes.value.ok) {
          const d = await nursesRes.value.json();
          const list = Array.isArray(d) ? d : d.nurses ?? [];
          setNurses(list);
          setNurseCount(list.filter((n: { active?: boolean }) => n.active !== false).length);
        }
        if (meRes.status === 'fulfilled' && meRes.value.ok) {
          const d = await meRes.value.json();
          setUserName(d?.user?.name ?? d?.name ?? '');
        }
      } catch {}
      finally { setLoading(false); }
    }
    loadData();
    loadWaiting();
    // Auto-refresh waiting room every 10s
    const interval = setInterval(loadWaiting, 10000);
    return () => clearInterval(interval);
  }, [loadWaiting]);
  // Load today's appointments
  useEffect(() => {
    const today = new Date().toISOString().slice(0, 10);
    fetch(`/api/iat-appointments?date=${today}`)
      .then(r => r.ok ? r.json() : [])
      .then(data => {
        const arr: TodayAppointment[] = Array.isArray(data) ? data : [];
        const todayStr = new Date().toDateString();
        setTodayAppts(arr.filter(a => new Date(a.startTime).toDateString() === todayStr));
      })
      .catch(() => {});
  }, []);
  async function handleQuickAddAppt() {
    if (!newApptTitle.trim()) return;
    setAddingAppt(true);
    const today = new Date().toISOString().slice(0, 10);
    const startIso = new Date(`${today}T${newApptTime}:00`).toISOString();
    const endIso = new Date(`${today}T${newApptEndTime}:00`).toISOString();
    try {
      const res = await fetch('/api/iat-appointments', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title: newApptTitle, startTime: startIso, endTime: endIso }),
      });
      if (res.ok) {
        const todayStr = new Date().toDateString();
        const newAppt = await res.json();
        if (new Date(newAppt.startTime).toDateString() === todayStr) {
          setTodayAppts(prev => [...prev, newAppt].sort((a, b) => new Date(a.startTime).getTime() - new
Date(b.startTime).getTime()));
        }
        setShowAddApptModal(false);
        setNewApptTitle('');
        setNewApptTime('09:00');
        setNewApptEndTime('10:00');
      }
    } catch {}
    setAddingAppt(false);
  }
  async function updateStatus(id: string, status: string, nurseName?: string) {
    setUpdatingId(id);
    await fetch(`/api/waiting-room/${id}`, {
```

```
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status, nurseName }),
    });
    await loadWaiting();
    setUpdatingId(null);
  }
  function waitTime(checkedInAt: string) {
    const mins = Math.floor((Date.now() - new Date(checkedInAt).getTime()) / 60000);
    if (mins < 1) return 'Just arrived';
    if (mins === 1) return '1 min';
    return `${mins} mins`;
  }
  const today = new Date().toLocaleDateString('en-US', { weekday: 'long', month: 'long', day: 'numeric', year: '
numeric' });
  const waitingCount = waiting.filter(e => e.status === 'waiting').length;
  const inServiceCount = waiting.filter(e => e.status === 'in-service').length;
  return (
    <>
      <div className="page-header">
        <div>
          <div className="page-title">{userName ? `Welcome, ${userName}` : 'Dashboard'}</div>
          <div className="page-subtitle">{today}</div>
        </div>
        <div style={{ display: 'flex', gap: 8 }}>
          <Link href="/patients/new" className="btn-secondary btn-sm btn">+ Register Patient</Link>
          <Link href="/testing" className="btn btn-sm">■ Start Testing</Link>
          <Link href="/calendar?action=new" className="btn btn-sm" style={{ background: '#7c3aed', color: '#fff'
}}>■ Book Appointment</Link>
        </div>
      </div>
      <div className="page-body">
        {/* KPI Cards */}
        <div className="kpi-grid">
          <div className="kpi-card">
            <div className="kpi-icon">■</div>
            <div className="kpi-label">Total Patients</div>
            {loading ? <div className="spinner" /> : <div className="kpi-value">{patientCount ?? 0}</div>}
          </div>
          <div className="kpi-card" style={{ borderTop: `4px solid #f59e0b` }}>
            <div className="kpi-icon">■</div>
            <div className="kpi-label">Waiting</div>
            <div className="kpi-value" style={{ color: waitingCount > 0 ? '#b45309' : '#64748b'
}}>{waitingCount}</div>
          </div>
          <div className="kpi-card" style={{ borderTop: `4px solid #0d9488` }}>
            <div className="kpi-icon">■</div>
            <div className="kpi-label">In Service</div>
            <div className="kpi-value" style={{ color: inServiceCount > 0 ? '#0d9488' : '#64748b'
}}>{inServiceCount}</div>
          </div>
          <div className="kpi-card">
            <div className="kpi-icon">■■■■</div>
            <div className="kpi-label">Nurses</div>
            {loading ? <div className="spinner" /> : <div className="kpi-value">{nurseCount ?? 0}</div>}
          </div>
        </div>
        {/* Waiting Room Board */}
        <div className="card" style={{ marginBottom: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: 16
}}>
            <div style={{ display: 'flex', alignItems: 'center', gap: 10 }}>
              <div className="card-title" style={{ margin: 0 }}>■ Waiting Room</div>
```

```
              <span style={{ fontSize: 11, color: '#0d9488', fontWeight: 600, display: 'flex', alignItems: '
center', gap: 4 }}>
                <span style={{ width: 8, height: 8, borderRadius: '50%', background: '#0d9488', display: '
inline-block', animation: 'pulse 2s infinite' }} />
                Live · refreshes every 10s
              </span>
            </div>
            <button onClick={loadWaiting} style={{ padding: '6px 14px', borderRadius: 8, border: '1px solid
#e2e8f0', background: '#fff', color: '#374151', fontSize: 13, fontWeight: 600, cursor: 'pointer' }}>
              ■ Now
            </button>
          </div>
          {waiting.length === 0 ? (
            <div style={{ textAlign: 'center', padding: '32px 0', color: '#94a3b8' }}>
              <div style={{ fontSize: 40, marginBottom: 8 }}>■</div>
              <div style={{ fontSize: 16, fontWeight: 600 }}>Waiting room is clear</div>
              <div style={{ fontSize: 13, marginTop: 4 }}>Patients checked in via kiosk will appear here</div>
            </div>
          ) : (
            <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 14 }}>
              <thead>
                <tr style={{ background: '#f8fafc' }}>
                  {['Patient', 'Wait Time', 'Videos', 'Status', 'Nurse', 'Actions'].map(h => (
                    <th key={h} style={{ padding: '10px 14px', textAlign: 'left', fontSize: 12, fontWeight: 700,
color: '#374151', textTransform: 'uppercase', borderBottom: '2px solid #e2e8f0' }}>{h}</th>
                  ))}
                </tr>
              </thead>
              <tbody>
                {waiting.map(e => (
                  <tr key={e.id} style={{ borderBottom: '1px solid #f1f5f9', background: e.status === '
in-service' ? '#e8f9f7' : 'white' }}>
                    <td style={{ padding: '12px 14px' }}>
                      <div style={{ fontWeight: 700 }}>{e.patientName}</div>
                      {e.notes && <div style={{ fontSize: 12, color: '#64748b' }}>{e.notes}</div>}
                    </td>
                    <td style={{ padding: '12px 14px', color: '#64748b', fontSize: 13
}}>{waitTime(e.checkedInAt)}</td>
                    <td style={{ padding: '12px 14px' }}>
                      {e.videoAckBy ? (
                        <div>
                          <div style={{ fontSize: 12, fontWeight: 700, color: '#15803d' }}>■ {e.videosWatched ??
0} watched</div>
                          <div style={{ fontSize: 11, color: '#64748b' }}>Ack: {e.videoAckBy}</div>
                        </div>
                      ) : (
                        <div>
                          <div style={{ fontSize: 12, fontWeight: 600, color: (e.videosWatched ?? 0) > 0 ? '
#b45309' : '#94a3b8', marginBottom: 4 }}>
                            {(e.videosWatched ?? 0) > 0 ? `■ ${e.videosWatched} video${(e.videosWatched ?? 0) !
== 1 ? 's' : ''} watched` : '—'}
                          </div>
                          {(e.videosWatched ?? 0) > 0 && (
                            <select
                              defaultValue=""
                              onChange={ev => ev.target.value && (async () => {
                                await fetch(`/api/waiting-room/${e.id}`, { method: 'PUT', headers: { '
Content-Type': 'application/json' }, body: JSON.stringify({ videoAckBy: ev.target.value }) });
                                loadWaiting();
                              })()}
                              style={{ fontSize: 11, padding: '2px 6px', borderRadius: 6, border: '1px solid
#fde68a', background: '#fefce8', cursor: 'pointer', color: '#92400e' }}
                            >
                              <option value="">✓ Acknowledge</option>
                              {nurses.map(n => <option key={n.id} value={n.name}>{n.name}</option>)}
```

```
                          </select>
                        )}
                      </div>
                    )}
                  </td>
                  <td style={{ padding: '12px 14px' }}>
                    <span style={{
                      fontWeight: 700, fontSize: 12, padding: '3px 10px', borderRadius: 999,
                      background: e.status === 'waiting' ? '#fef9c3' : '#d1fae5',
                      color: e.status === 'waiting' ? '#b45309' : '#065f46',
                    }}>
                      {e.status === 'waiting' ? '■ Waiting' : '■ In Service'}
                    </span>
                  </td>
                  <td style={{ padding: '12px 14px', color: '#64748b', fontSize: 13 }}>
                    {e.status === 'waiting' ? (
                      <select onChange={ev => ev.target.value && updateStatus(e.id, 'in-service',
ev.target.value)}
                        defaultValue=""
                        style={{ fontSize: 13, padding: '4px 8px', borderRadius: 6, border: '1px solid
#e2e8f0', cursor: 'pointer' }}>
                        <option value="">— Call Patient —</option>
                        {nurses.map(n => <option key={n.id} value={n.name}>{n.title ? `${n.title} ${n.name}` :
n.name}</option>)}
                      </select>
                    ) : (
                      <span style={{ fontWeight: 600, color: '#0d9488' }}>{e.nurseName ?? '—'}</span>
                    )}
                  </td>
                  <td style={{ padding: '12px 14px' }}>
                    {e.status === 'in-service' && (
                      <button onClick={() => updateStatus(e.id, 'complete')}
                        disabled={updatingId === e.id}
                        style={{ padding: '6px 14px', borderRadius: 8, border: 'none', background: '#0055A5',
color: '#fff', fontSize: 13, fontWeight: 700, cursor: 'pointer' }}>
                        {updatingId === e.id ? '■' : '■ Complete'}
                      </button>
                    )}
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        )}
      </div>
      {/* Today's Schedule (mini calendar widget) */}
      <div className="card" style={{ marginBottom: 24 }}>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: 16
}}>
          <div className="card-title" style={{ margin: 0 }}>■ Today&apos;s Schedule</div>
          <div style={{ display: 'flex', gap: 8, alignItems: 'center' }}>
            <button onClick={() => setShowAddApptModal(true)}
              style={{ padding: '5px 14px', borderRadius: 8, border: 'none', background: '#0d9488', color: '
#fff', fontWeight: 700, fontSize: 13, cursor: 'pointer' }}>
              + Add
            </button>
            <Link href="/calendar"
              style={{ padding: '5px 14px', borderRadius: 8, border: '1px solid #0d9488', color: '#0d9488',
fontWeight: 600, fontSize: 13, textDecoration: 'none' }}>
              View Full Calendar →
            </Link>
          </div>
        </div>
        {todayAppts.length === 0 ? (
```

```
            <div style={{ textAlign: 'center', padding: '20px 0', color: '#94a3b8' }}>
              <div style={{ fontSize: 28, marginBottom: 6 }}>■</div>
              <div style={{ fontSize: 14, fontWeight: 600 }}>No appointments scheduled for today</div>
              <div style={{ fontSize: 12, marginTop: 4 }}>
                <button onClick={() => setShowAddApptModal(true)} style={{ background: 'none', border: 'none',
color: '#0d9488', cursor: 'pointer', fontWeight: 600, textDecoration: 'underline', fontSize: 12 }}>
                  Book one now
                </button>
              </div>
            </div>
          ) : (
            <div style={{ display: 'flex', flexDirection: 'column', gap: 8 }}>
              {todayAppts.map(appt => {
                const c = APPT_TYPE_COLORS[appt.type] ?? APPT_TYPE_COLORS['allergy-test'];
                return (
                  <div key={appt.id}
                    onClick={() => setSelectedAppt(appt)}
                    style={{ display: 'flex', alignItems: 'center', gap: 12, padding: '10px 14px', background:
c.bg, borderRadius: 10, border: `1.5px solid ${c.text}20`, cursor: 'pointer', transition: 'box-shadow 0.15s' }}
                    onMouseEnter={e => (e.currentTarget.style.boxShadow = '0 2px 8px rgba(0,0,0,0.12)')}
                    onMouseLeave={e => (e.currentTarget.style.boxShadow = 'none')}
                  >
                    <div style={{ fontWeight: 700, fontSize: 13, color: c.text, minWidth: 56 }}>
                      {formatApptTime(appt.startTime)}
                    </div>
                    <div style={{ flex: 1 }}>
                      <div style={{ fontWeight: 700, fontSize: 14, color: '#111827' }}>{appt.title}</div>
                      {appt.patientName && <div style={{ fontSize: 12, color: '#64748b'
}}>{appt.patientName}</div>}
                    </div>
                    <span style={{ fontSize: 11, fontWeight: 700, padding: '2px 10px', borderRadius: 999,
background: '#fff', color: c.text, border: `1px solid ${c.text}` }}>
                      {appt.status}
                    </span>
                  </div>
                );
              })}
            </div>
          )}
          {/* Quick-add modal */}
          {showAddApptModal && (
            <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.35)', zIndex: 1000, display: '
flex', alignItems: 'center', justifyContent: 'center', padding: 16 }}>
              <div style={{ background: '#fff', borderRadius: 14, width: '100%', maxWidth: 400, padding: 24,
boxShadow: '0 10px 40px rgba(0,0,0,0.15)' }}>
                <div style={{ fontWeight: 700, fontSize: 18, marginBottom: 16, color: '#111827' }}>+ Quick Add
Appointment</div>
                <div style={{ marginBottom: 12 }}>
                  <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>Title *</label>
                  <input value={newApptTitle} onChange={e => setNewApptTitle(e.target.value)}
                    placeholder="e.g. Follow-up visit"
                    style={{ width: '100%', padding: '8px 12px', borderRadius: 8, border: '1.5px solid #e2e8f0',
fontSize: 14, boxSizing: 'border-box' }} />
                </div>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 10, marginBottom: 20 }}>
                  <div>
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>Start</label>
                    <input type="time" value={newApptTime} onChange={e => setNewApptTime(e.target.value)}
                      style={{ width: '100%', padding: '8px 10px', borderRadius: 8, border: '1.5px solid
#e2e8f0', fontSize: 14, boxSizing: 'border-box' }} />
                  </div>
                  <div>
```

```
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>End</label>
                    <input type="time" value={newApptEndTime} onChange={e => setNewApptEndTime(e.target.value)}
                      style={{ width: '100%', padding: '8px 10px', borderRadius: 8, border: '1.5px solid
#e2e8f0', fontSize: 14, boxSizing: 'border-box' }} />
                  </div>
                </div>
                <div style={{ display: 'flex', gap: 10 }}>
                  <button onClick={() => setShowAddApptModal(false)}
                    style={{ flex: 1, padding: '10px 0', borderRadius: 10, border: '1.5px solid #e2e8f0',
background: '#fff', color: '#374151', fontWeight: 700, cursor: 'pointer', fontSize: 14 }}>
                    Cancel
                  </button>
                  <button onClick={handleQuickAddAppt} disabled={addingAppt || !newApptTitle.trim()}
                    style={{ flex: 2, padding: '10px 0', borderRadius: 10, border: 'none', background: '#0d9488',
color: '#fff', fontWeight: 700, cursor: 'pointer', fontSize: 14, opacity: addingAppt ? 0.7 : 1 }}>
                    {addingAppt ? 'Saving…' : 'Book for Today'}
                  </button>
                </div>
              </div>
            </div>
          )}
        </div>
        {/* Appointment Detail/Edit/Delete Modal */}
        {selectedAppt && (
          <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.4)', zIndex: 1000, display: '
flex', alignItems: 'center', justifyContent: 'center', padding: 16 }}>
            <div style={{ background: '#fff', borderRadius: 16, width: '100%', maxWidth: 420, boxShadow: '0 20px
60px rgba(0,0,0,0.2)', overflow: 'hidden' }}>
              {/* Header */}
              <div style={{ background: APPT_TYPE_COLORS[selectedAppt.type]?.bg ?? '#e8f9f7', padding: '16px
20px', borderBottom: '1px solid #e2e8f0', display: 'flex', justifyContent: 'space-between', alignItems: 'center'
}}>
                <div style={{ fontWeight: 800, fontSize: 16, color: APPT_TYPE_COLORS[selectedAppt.type]?.text ?? '
#0d9488' }}>{selectedAppt.title}</div>
                <button onClick={() => setSelectedAppt(null)} style={{ background: 'none', border: 'none',
fontSize: 20, cursor: 'pointer', color: '#64748b' }}>✕</button>
              </div>
              {/* Details */}
              <div style={{ padding: '16px 20px' }}>
                {[
                  { label: 'Patient', value: selectedAppt.patientName || '—' },
                  { label: 'Time', value: `${formatApptTime(selectedAppt.startTime)} –
${formatApptTime(selectedAppt.endTime)}` },
                  { label: 'Type', value: selectedAppt.type },
                  { label: 'Status', value: selectedAppt.status },
                  { label: 'Notes', value: selectedAppt.notes || '—' },
                ].map(r => (
                  <div key={r.label} style={{ display: 'flex', gap: 12, padding: '8px 0', borderBottom: '1px
solid #f1f5f9' }}>
                    <div style={{ width: 70, fontSize: 12, fontWeight: 700, color: '#64748b', textTransform: '
uppercase' }}>{r.label}</div>
                    <div style={{ fontSize: 14, color: '#111827' }}>{r.value}</div>
                  </div>
                ))}
              </div>
              {/* Actions */}
              <div style={{ padding: '12px 20px', borderTop: '1px solid #e2e8f0', display: 'flex', gap: 8,
justifyContent: 'flex-end' }}>
                <button
                  onClick={async () => {
                    const t = selectedAppt;
                    const start = t.startTime ? new Date(t.startTime) : new Date();
                    const end = t.endTime ? new Date(t.endTime) : new Date();
                    setEditApptTitle(t.title);
```

```
                    setEditApptDate(start.toISOString().split('T')[0]);
                    setEditApptStartHour(String(start.getHours()).padStart(2,'0'));
                    setEditApptStartMin(String(start.getMinutes()).padStart(2,'0'));
                    setEditApptEndHour(String(end.getHours()).padStart(2,'0'));
                    setEditApptEndMin(String(end.getMinutes()).padStart(2,'0'));
                    setEditApptPatientId(t.patientId || '');
                    setEditApptPatientName(t.patientName || '');
                    setEditApptReasonId(t.type || '');
                    setEditApptNotes(t.notes || '');
                    setEditingAppt(t);
                    setSelectedAppt(null);
                    // Load patients and reasons
                    const [pRes, rRes] = await Promise.allSettled([
                      fetch('/api/patients').then(r => r.json()),
                      fetch('/api/appointment-reasons').then(r => r.json()),
                    ]);
                    if (pRes.status === 'fulfilled') setEditApptPatients(Array.isArray(pRes.value) ? pRes.value :
pRes.value.patients ?? []);
                    if (rRes.status === 'fulfilled') setEditApptReasons(rRes.value.reasons ?? []);
                  }}
                  style={{ padding: '8px 16px', borderRadius: 8, border: '1px solid #e2e8f0', background: '#fff',
color: '#374151', fontSize: 13, fontWeight: 600, cursor: 'pointer' }}>
                  ✏ Edit
                </button>
                <button
                  disabled={deletingApptId === selectedAppt.id}
                  onClick={async () => {
                    if (!confirm('Delete this appointment?')) return;
                    setDeletingApptId(selectedAppt.id);
                    await fetch(`/api/iat-appointments/${selectedAppt.id}`, { method: 'DELETE' });
                    setTodayAppts(prev => prev.filter(a => a.id !== selectedAppt.id));
                    setSelectedAppt(null);
                    setDeletingApptId(null);
                  }}
                  style={{ padding: '8px 16px', borderRadius: 8, border: 'none', background: '#fef2f2', color: '
#b91c1c', fontSize: 13, fontWeight: 600, cursor: 'pointer' }}>
                  {deletingApptId === selectedAppt.id ? '⏳' : '🗑 Delete'}
                </button>
              </div>
            </div>
          </div>
        )}
        {/* Inline Edit Appointment Modal */}
        {editingAppt && (
          <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.4)', zIndex: 1000, display: '
flex', alignItems: 'center', justifyContent: 'center', padding: 16 }}>
            <div style={{ background: '#fff', borderRadius: 16, width: '100%', maxWidth: 440, boxShadow: '0 20px
60px rgba(0,0,0,0.2)', overflow: 'hidden' }}>
              <div style={{ padding: '16px 20px', borderBottom: '1px solid #e2e8f0', display: 'flex',
justifyContent: 'space-between', alignItems: 'center' }}>
                <div style={{ fontWeight: 700, fontSize: 16 }}>✏ Edit Appointment</div>
                <button onClick={() => setEditingAppt(null)} style={{ background: 'none', border: 'none',
fontSize: 20, cursor: 'pointer', color: '#64748b' }}>✕</button>
              </div>
              <div style={{ padding: '16px 20px', display: 'flex', flexDirection: 'column', gap: 12 }}>
                {/* Patient */}
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Patient *</label>
                  <select className="form-input" value={editApptPatientId}
                    onChange={e => { const p = editApptPatients.find(x => x.id === e.target.value);
setEditApptPatientId(e.target.value); setEditApptPatientName(p?.name ?? ''); }}>
                    <option value="">— Select Patient —</option>
                    {editApptPatients.map(p => <option key={p.id} value={p.id}>{p.name}</option>)}
                  </select>
```

```
                </div>
                {/* Reason */}
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Reason</label>
                  <select className="form-input" value={editApptReasonId} onChange={e =>
setEditApptReasonId(e.target.value)}>
                    <option value="">— Select Reason —</option>
                    {editApptReasons.map(r => <option key={r.id} value={r.id}>{r.name}</option>)}
                  </select>
                </div>
                {/* Title */}
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Title</label>
                  <input className="form-input" value={editApptTitle} onChange={e =>
setEditApptTitle(e.target.value)} />
                </div>
                {/* Date + Time */}
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Date</label>
                  <input className="form-input" type="date" value={editApptDate} onChange={e =>
setEditApptDate(e.target.value)} />
                </div>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 10 }}>
                  <div>
                    <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Start Time</label>
                    <div style={{ display: 'flex', gap: 6 }}>
                      <select className="form-input" value={editApptStartHour} onChange={e =>
setEditApptStartHour(e.target.value)}>
                        {Array.from({length: 12}, (_, i) => i + 7).map(h => <option key={h}
value={String(h).padStart(2,'0')}>{h > 12 ? `${h-12}PM` : h === 12 ? '12PM' : `${h}AM`}</option>)}
                      </select>
                      <select className="form-input" value={editApptStartMin} onChange={e =>
setEditApptStartMin(e.target.value)}>
                        {['00','15','30','45'].map(m => <option key={m} value={m}>:{m}</option>)}
                      </select>
                    </div>
                  </div>
                  <div>
                    <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>End Time</label>
                    <div style={{ display: 'flex', gap: 6 }}>
                      <select className="form-input" value={editApptEndHour} onChange={e =>
setEditApptEndHour(e.target.value)}>
                        {Array.from({length: 12}, (_, i) => i + 7).map(h => <option key={h}
value={String(h).padStart(2,'0')}>{h > 12 ? `${h-12}PM` : h === 12 ? '12PM' : `${h}AM`}</option>)}
                      </select>
                      <select className="form-input" value={editApptEndMin} onChange={e =>
setEditApptEndMin(e.target.value)}>
                        {['00','15','30','45'].map(m => <option key={m} value={m}>:{m}</option>)}
                      </select>
                    </div>
                  </div>
                </div>
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Notes</label>
                  <textarea className="form-input" rows={2} value={editApptNotes} onChange={e =>
setEditApptNotes(e.target.value)} style={{ resize: 'vertical' }} />
                </div>
              </div>
```

```
              <div style={{ padding: '12px 20px', borderTop: '1px solid #e2e8f0', display: 'flex', gap: 8,
justifyContent: 'flex-end' }}>
                <button onClick={() => setEditingAppt(null)} style={{ padding: '8px 16px', borderRadius: 8,
border: '1px solid #e2e8f0', background: '#fff', cursor: 'pointer', fontWeight: 600, fontSize: 13
}}>Cancel</button>
                <button disabled={savingAppt} onClick={async () => {
                  setSavingAppt(true);
                  const startIso = `${editApptDate}T${editApptStartHour}:${editApptStartMin}:00`;
                  const endIso = `${editApptDate}T${editApptEndHour}:${editApptEndMin}:00`;
                  const reason = editApptReasons.find(r => r.id === editApptReasonId);
                  await fetch(`/api/iat-appointments/${editingAppt.id}`, {
                    method: 'PUT', headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({
                      title: editApptTitle, startTime: startIso, endTime: endIso,
                      patientId: editApptPatientId, patientName: editApptPatientName,
                      type: reason?.name || editApptReasonId, notes: editApptNotes,
                    }),
                  });
                  setSavingAppt(false); setEditingAppt(null);
                  const today = new Date().toISOString().split('T')[0];
                  fetch(`/api/iat-appointments?date=${today}`).then(r => r.json()).then(d =>
setTodayAppts(d.appointments ?? d ?? []));
                }}
                  style={{ padding: '8px 16px', borderRadius: 8, border: 'none', background: '#0d9488', color: '
#fff', cursor: 'pointer', fontWeight: 700, fontSize: 13 }}>
                  {savingAppt ? '■' : '■ Save'}
                </button>
              </div>
            </div>
          </div>
        )}
        {/* Quick Actions */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 24 }}>
          <div className="card">
            <div className="card-title">Quick Actions</div>
            <div style={{ display: 'flex', flexDirection: 'column', gap: 10 }}>
              {[
                { href: '/patients/new', label: '■ Register New Patient' },
                { href: '/testing', label: '■ Start Testing' },
                { href: '/patients', label: '■ View All Patients' },
                { href: '/kiosk', label: '■ Open Patient Kiosk', target: '_blank' },
                { href: '/doctors', label: '■■■■ Manage Doctors' },
                { href: '/nurses', label: '■■■■ Manage Nurses' },
              ].map(a => (
                <a key={a.href} href={a.href} target={a.target}
                  style={{ display: 'flex', alignItems: 'center', gap: 8, padding: '10px 16px', borderRadius: 8,
border: '1px solid #e2e8f0', background: '#fff', color: '#374151', fontSize: 15, fontWeight: 600, textDecoration: '
none' }}>
                  {a.label}
                </a>
              ))}
            </div>
          </div>
          <div className="card">
            <div className="card-title">System Status</div>
            <div style={{ display: 'flex', flexDirection: 'column', gap: 14 }}>
              {[
                { label: 'Patient Kiosk', status: 'Online', icon: '■' },
                { label: 'API Server', status: 'Operational', icon: '■' },
                { label: 'Database', status: 'Operational', icon: '■' },
                { label: 'Waiting Room', status: `${waitingCount + inServiceCount} active`, icon: '■' },
                { label: 'Auth Service', status: 'Operational', icon: '■' },
                { label: 'HIPAA Compliance', status: 'Active', icon: '■' },
              ].map(s => (
```

```
                <div key={s.label} style={{ display: 'flex', justifyContent: 'space-between', alignItems: '
center', padding: '8px 0', borderBottom: '1px solid #f1f5f9' }}>
                  <span style={{ fontSize: 14, color: '#374151' }}>{s.icon} {s.label}</span>
                  <span style={{ fontSize: 12, fontWeight: 700, padding: '2px 10px', borderRadius: 999,
background: '#e8f9f7', color: '#0d9488' }}>{s.status}</span>
                </div>
              ))}
            </div>
          </div>
        </div>
      </div>
    </>
  );
}
```

page.tsx

app/patients/page.tsx

```
'use client';
import { useEffect, useState } from 'react';
import Link from 'next/link';
import { useRouter } from 'next/navigation';
interface Patient {
  id: string;
  patientId?: string;
  name: string;
  dob?: string;
  status: string;
  clinicLocation?: string;
  physician?: string;
  email?: string;
}
const STATUS_BADGE: Record<string, string> = {
  registered: 'badge-blue',
  tested: 'badge-yellow',
  consented: 'badge-green',
  complete: 'badge-teal',
};
export default function PatientsPage() {
  const router = useRouter();
  const [patients, setPatients] = useState<Patient[]>([]);
  const [search, setSearch] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  useEffect(() => {
    fetch('/api/patients')
      .then((r) => {
        if (!r.ok) throw new Error('Failed to load patients');
        return r.json();
      })
      .then((data) => {
        const list: Patient[] = Array.isArray(data) ? data : (data.patients ?? []);
        setPatients(list);
      })
      .catch((e) => setError(e.message))
      .finally(() => setLoading(false));
  }, []);
  const filtered = patients.filter((p) => {
    const q = search.toLowerCase();
    return (p.name ?? '').toLowerCase().includes(q) || (p.patientId ?? p.id).toLowerCase().includes(q);
  });
  function formatDOB(val?: string) {
    if (!val) return '—';
```

```
    try {
      const d = new Date(val);
      return new Date(d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate())
        .toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' });
    } catch {
      return val;
    }
  }
  return (
    <>
      <div className="page-header">
        <div>
          <div className="page-title">Patients</div>
          <div className="page-subtitle">
            {loading ? 'Loading...' : `${filtered.length} patient${filtered.length !== 1 ? 's' : ''}`}
          </div>
        </div>
        <Link href="/patients/new" className="btn">
          + Register Patient
        </Link>
      </div>
      <div className="page-body">
        {error && <div className="alert alert-error">■■ {error}</div>}
        {/* Search */}
        <div className="card mb-6">
          <div className="search-wrapper">
            <span className="search-icon">■</span>
            <input
              type="text"
              className="form-input search-input"
              placeholder="Search by name or patient ID…"
              value={search}
              onChange={(e) => setSearch(e.target.value)}
            />
          </div>
        </div>
        {/* Table */}
        {loading ? (
          <div className="loading-center"><div className="spinner" /><span>Loading patients…</span></div>
        ) : filtered.length === 0 ? (
          <div className="empty-state">
            <div className="empty-state-icon">■</div>
            <div className="empty-state-title">{search ? 'No patients match your search' : 'No patients
yet'}</div>
            {!search && (
              <div style={{ marginTop: 16 }}>
                <Link href="/patients/new" className="btn">Register First Patient</Link>
              </div>
            )}
          </div>
        ) : (
          <div className="table-container">
            <table>
              <thead>
                <tr>
                  <th>Patient ID</th>
                  <th>Name</th>
                  <th>Date of Birth</th>
                  <th>Status</th>
                  <th>Location</th>
                  <th>Actions</th>
                </tr>
              </thead>
              <tbody>
```

```
                {filtered.map((p) => (
                  <tr key={p.id} onClick={() => router.push(`/patients/${p.id}`)}>
                    <td>
                      <code style={{ fontSize: 12, background: '#f1f5f9', padding: '2px 6px', borderRadius: 4 }}>
                        {p.patientId ?? p.id.slice(0, 8).toUpperCase()}
                      </code>
                    </td>
                    <td>
                      <div style={{ fontWeight: 600 }}>
                        {p.name}
                      </div>
                      {p.email && <div style={{ fontSize: 12, color: '#64748b' }}>{p.email}</div>}
                    </td>
                    <td>{formatDOB(p.dob)}</td>
                    <td>
                      <span className={`badge ${STATUS_BADGE[p.status?.toLowerCase()] ?? 'badge-gray'}`}>
                        {p.status ?? 'unknown'}
                      </span>
                    </td>
                    <td>{p.clinicLocation ?? '—'}</td>
                    <td onClick={(e) => e.stopPropagation()}>
                      <div className="flex gap-2">
                        <Link href={`/patients/${p.id}`} className="btn btn-sm btn-secondary">View</Link>
                        <Link href={`/testing?patientId=${p.id}`} className="btn btn-sm">Test</Link>
                      </div>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </div>
    </>
  );
}
```

page.tsx

app/patients/new/page.tsx

```
'use client';
import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
interface Doctor { id: string; name: string; }
interface Location { id: string; name: string; }
export default function NewPatientPage() {
  const router = useRouter();
  const [doctors, setDoctors] = useState<Doctor[]>([]);
  const [locations, setLocations] = useState<Location[]>([]);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState('');
  const [form, setForm] = useState({
    // Section 1 — Patient
    name: '',
    dob: '',
    email: '',
    phone: '',
    homePhone: '',
    street: '',
    city: '',
    state: '',
    zip: '',
    // Section 2 — Clinical
```

```
    physician: '',
    clinicLocation: '',
    diagnosis: '',
    // Section 3 — Insurance / Notes
    insuranceId: '',
    insuranceProvider: '',
    insuranceGroup: '',
    notes: '',
  });
  useEffect(() => {
    fetch('/api/doctors')
      .then(r => r.ok ? r.json() : [])
      .then((d: Doctor[] | { doctors?: Doctor[] }) => setDoctors(Array.isArray(d) ? d : (d.doctors ?? [])))
      .catch(() => {});
    fetch('/api/locations')
      .then(r => r.ok ? r.json() : [])
      .then((d: Location[]) => setLocations(Array.isArray(d) ? d : []))
      .catch(() => {});
  }, []);
  function setField(field: string, value: string) {
    setForm(prev => ({ ...prev, [field]: value }));
  }
  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setError('');
    if (!form.name.trim()) { setError('Name is required'); return; }
    if (!form.dob) { setError('Date of birth is required'); return; }
    setSaving(true);
    try {
      const res = await fetch('/api/patients', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          name: form.name.trim(),
          dob: form.dob,
          email: form.email || undefined,
          phone: form.phone || undefined,
          physician: form.physician || undefined,
          clinicLocation: form.clinicLocation || undefined,
          diagnosis: form.diagnosis || undefined,
          insuranceId: form.insuranceId || undefined,
          notes: [
            form.homePhone ? `Home: ${form.homePhone}` : '',
            form.street || form.city || form.state || form.zip
              ? `Address: ${[form.street, form.city, form.state, form.zip].filter(Boolean).join(', ')}`
              : '',
            form.insuranceProvider ? `Insurance: ${form.insuranceProvider}${form.insuranceGroup ? ` Group:
${form.insuranceGroup}` : ''}` : '',
            form.notes || '',
          ].filter(Boolean).join(' | ') || undefined,
        }),
      });
      if (!res.ok) {
        const errData = await res.json().catch(() => ({}));
        throw new Error(errData.error ?? `Request failed: ${res.status}`);
      }
      const created = await res.json();
      router.push(`/patients/${created.id}`);
    } catch (e: unknown) {
      setError(e instanceof Error ? e.message : 'Failed to register patient');
    } finally {
      setSaving(false);
    }
  }
```

```
  return (
    <>
      <div className="page-header">
        <div>
          <div className="page-title">Register New Patient</div>
          <div className="page-subtitle">Complete all required fields to register the patient</div>
        </div>
        <Link href="/patients" className="btn btn-secondary">← Back to Patients</Link>
      </div>
      <div className="page-body">
        {error && <div className="alert alert-error">■■ {error}</div>}
        <form onSubmit={handleSubmit}>
          {/* Section 1 — Patient Info */}
          <div className="card form-section">
            <div className="form-section-title">
              <span className="form-section-number">1</span>
              Patient Information
            </div>
            <div className="form-row form-row-2">
              <div className="form-group">
                <label className="form-label">Full Name <span className="required">*</span></label>
                <input
                  type="text"
                  className="form-input"
                  placeholder="e.g. Jane Smith"
                  value={form.name}
                  onChange={e => setField('name', e.target.value)}
                  required
                  style={{ fontSize: 16 }}
                />
              </div>
              <div className="form-group">
                <label className="form-label">Date of Birth <span className="required">*</span></label>
                <input
                  type="date"
                  className="form-input"
                  value={form.dob}
                  onChange={e => setField('dob', e.target.value)}
                  required
                  style={{ fontSize: 16 }}
                />
              </div>
            </div>
            <div className="form-row form-row-2">
              <div className="form-group">
                <label className="form-label">Cell Phone</label>
                <input type="tel" className="form-input" placeholder="(555) 555-0100"
                  value={form.phone} onChange={e => setField('phone', e.target.value)} style={{ fontSize: 16 }} />
              </div>
              <div className="form-group">
                <label className="form-label">Home Phone</label>
                <input type="tel" className="form-input" placeholder="(555) 555-0101"
                  value={form.homePhone} onChange={e => setField('homePhone', e.target.value)} style={{ fontSize:
16 }} />
              </div>
            </div>
            <div className="form-row form-row-2">
              <div className="form-group">
                <label className="form-label">Email</label>
                <input type="email" className="form-input" placeholder="patient@example.com"
                  value={form.email} onChange={e => setField('email', e.target.value)} style={{ fontSize: 16 }} />
              </div>
            </div>
            <div className="form-group">
```

```
              <label className="form-label">Street Address</label>
              <input type="text" className="form-input" placeholder="123 Main St"
                value={form.street} onChange={e => setField('street', e.target.value)} style={{ fontSize: 16 }} />
            </div>
            <div className="form-row" style={{ display: 'grid', gridTemplateColumns: '2fr 1fr 1fr', gap: 12 }}>
              <div className="form-group">
                <label className="form-label">City</label>
                <input type="text" className="form-input" placeholder="City"
                  value={form.city} onChange={e => setField('city', e.target.value)} style={{ fontSize: 16 }} />
              </div>
              <div className="form-group">
                <label className="form-label">State</label>
                <input type="text" className="form-input" placeholder="VA" maxLength={2}
                  value={form.state} onChange={e => setField('state', e.target.value.toUpperCase())} style={{
fontSize: 16 }} />
              </div>
              <div className="form-group">
                <label className="form-label">ZIP</label>
                <input type="text" className="form-input" placeholder="22026" maxLength={10}
                  value={form.zip} onChange={e => setField('zip', e.target.value)} style={{ fontSize: 16 }} />
              </div>
            </div>
          </div>
          {/* Section 2 — Clinical */}
          <div className="card form-section">
            <div className="form-section-title">
              <span className="form-section-number">2</span>
              Clinical Information
            </div>
            <div className="form-row form-row-2">
              <div className="form-group">
                <label className="form-label">Physician</label>
                <input
                  type="text"
                  className="form-input"
                  placeholder="Referring physician name"
                  value={form.physician}
                  onChange={e => setField('physician', e.target.value)}
                  list="doctors-list"
                  style={{ fontSize: 16 }}
                />
                <datalist id="doctors-list">
                  {doctors.map(d => <option key={d.id} value={d.name} />)}
                </datalist>
              </div>
              <div className="form-group">
                <label className="form-label">Clinic Location</label>
                <select
                  className="form-input"
                  value={form.clinicLocation}
                  onChange={e => setField('clinicLocation', e.target.value)}
                  style={{ fontSize: 16 }}
                >
                  <option value="">— Select Location —</option>
                  {locations.map(loc => (
                    <option key={loc.id} value={loc.name}>{loc.name}</option>
                  ))}
                </select>
              </div>
            </div>
            <div className="form-group">
              <label className="form-label">Diagnosis</label>
              <input
                type="text"
```

```
                className="form-input"
                placeholder="Primary diagnosis or reason for testing"
                value={form.diagnosis}
                onChange={e => setField('diagnosis', e.target.value)}
                style={{ fontSize: 16 }}
              />
            </div>
          </div>
          {/* Section 3 — Insurance & Notes */}
          <div className="card form-section">
            <div className="form-section-title">
              <span className="form-section-number">3</span>
              Insurance &amp; Notes
            </div>
            <div className="form-row form-row-2">
              <div className="form-group">
                <label className="form-label">Insurance Provider</label>
                <input type="text" className="form-input" placeholder="e.g. BlueCross BlueShield"
                  value={form.insuranceProvider} onChange={e => setField('insuranceProvider', e.target.value)}
style={{ fontSize: 16 }} />
              </div>
              <div className="form-group">
                <label className="form-label">Insurance ID / Member ID</label>
                <input type="text" className="form-input" placeholder="Member ID"
                  value={form.insuranceId} onChange={e => setField('insuranceId', e.target.value)} style={{
fontSize: 16 }} />
              </div>
            </div>
            <div className="form-row form-row-2">
              <div className="form-group">
                <label className="form-label">Group Number</label>
                <input type="text" className="form-input" placeholder="Group #"
                  value={form.insuranceGroup} onChange={e => setField('insuranceGroup', e.target.value)} style={{
fontSize: 16 }} />
              </div>
            </div>
            <div className="form-group">
              <label className="form-label">Notes</label>
              <textarea
                className="form-input"
                placeholder="Any relevant clinical notes or observations…"
                rows={4}
                value={form.notes}
                onChange={e => setField('notes', e.target.value)}
                style={{ resize: 'vertical', minHeight: 100, fontSize: 16 }}
              />
            </div>
          </div>
          {/* Actions */}
          <div className="flex gap-3" style={{ justifyContent: 'flex-end' }}>
            <Link href="/patients" className="btn btn-secondary">Cancel</Link>
            <button type="submit" className="btn" disabled={saving}>
              {saving
                ? <><div className="spinner" style={{ width: 16, height: 16, borderWidth: 2 }} /> Registering…</>
                : '✓ Register Patient'}
            </button>
          </div>
        </form>
      </div>
    </>
  );
}
```

page.tsx

app/patients/[id]/page.tsx

```
'use client';
import { useEffect, useState, useCallback, useRef } from 'react';
import { useParams } from 'next/navigation';
import Link from 'next/link';
interface Patient {
  id: string;
  patientId?: string;
  name: string;
  dob?: string;
  email?: string;
  phone?: string;
  homePhone?: string;
  street?: string;
  apt?: string;
  city?: string;
  state?: string;
  zip?: string;
  clinicLocation?: string;
  physician?: string;
  diagnosis?: string;
  status: string;
  doctorId?: string;
  notes?: string;
  insuranceId?: string;
  insuranceProvider?: string;
  insuranceGroup?: string;
  emergencyName?: string;
  emergencyPhone?: string;
  emergencyRelation?: string;
  createdAt?: string;
  updatedAt?: string;
  doctor?: { id: string; name: string; title?: string } | null;
  testResults?: TestResult[];
  videoActivity?: VideoWatch[];
  formActivity?: FormSigned[];
}
interface TestResult {
  id: string;
  allergen?: { name: string; type?: string };
  testType?: string;
  reaction?: number;
  wheal?: string;
  notes?: string;
  nurseName?: string;
  testedAt?: string;
}
interface VideoWatch {
  id: string;
  video?: { title: string };
  watchedAt?: string;
  acknowledged?: boolean;
}
interface LocationOption {
  id: string;
  name: string;
}
interface FormSigned {
  id: string;
  form?: { name: string };
  signedAt?: string;
  printedAt?: string;
  emailedAt?: string;
```

```
}
const STATUS_BADGE: Record<string, { cls: string; color: string; bg: string }> = {
  'build-up': { cls: 'badge-blue', color: '#1d4ed8', bg: '#dbeafe' },
  'maintenance':{ cls: 'badge-green', color: '#15803d', bg: '#dcfce7' },
  registered: { cls: 'badge-blue', color: '#1d4ed8', bg: '#dbeafe' },
  tested: { cls: 'badge-yellow', color: '#b45309', bg: '#fef9c3' },
  consented: { cls: 'badge-green', color: '#15803d', bg: '#dcfce7' },
  complete: { cls: 'badge-teal', color: '#0d9488', bg: '#e8f9f7' },
};
const REACTION_COLOR: Record<number, string> = {
  0: '#94a3b8', 1: '#ca8a04', 2: '#ea580c', 3: '#dc2626', 4: '#991b1b', 5: '#7f1d1d',
};
type Tab = 'overview' | 'tests' | 'videos' | 'forms' | 'consent' | 'encounters';
export default function PatientDetailPage() {
  const params = useParams();
  const id = params?.id as string;
  const [patient, setPatient] = useState<Patient | null>(null);
  const [tab, setTab] = useState<Tab>('overview');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [editing, setEditing] = useState(false);
  const [saving, setSaving] = useState(false);
  const [editForm, setEditForm] = useState<Partial<Patient>>({});
  const [locationOptions, setLocationOptions] = useState<LocationOption[]>([]);
  const [doctorOptions, setDoctorOptions] = useState<{ id: string; name: string; title?: string }[]>([]);
  const [nurses, setNurses] = useState<{ id: string; name: string; title?: string }[]>([]);
  const [sessionNurses, setSessionNurses] = useState<Record<string, string>>({});
  const [sessionNurseSaving, setSessionNurseSaving] = useState<Record<string, boolean>>({});
  const [editingRows, setEditingRows] = useState<Record<string, { reaction: number; wheal: string; flare: string;
notes: string }>>({});
  const [savingRows, setSavingRows] = useState<Record<string, boolean>>({});
  const locationsFetchedRef = useRef(false);
  const load = useCallback(() => {
    if (!id) return;
    setLoading(true);
    fetch(`/api/patients/${id}`)
      .then(r => { if (!r.ok) throw new Error('Patient not found'); return r.json(); })
      .then(data => { setPatient(data); setEditForm(data); })
      .catch(e => setError(e.message))
      .finally(() => setLoading(false));
  }, [id]);
  useEffect(() => { load(); }, [load]);
  useEffect(() => {
    if (editing && !locationsFetchedRef.current) {
      locationsFetchedRef.current = true;
      Promise.allSettled([
        fetch('/api/locations').then(r => r.ok ? r.json() : []),
        fetch('/api/doctors').then(r => r.ok ? r.json() : []),
      ]).then(([locRes, docRes]) => {
        if (locRes.status === 'fulfilled') {
          const d = locRes.value as LocationOption[] | { locations?: LocationOption[] };
          setLocationOptions(Array.isArray(d) ? d : (d.locations ?? []));
        }
        if (docRes.status === 'fulfilled') {
          const d = docRes.value as { id: string; name: string; title?: string }[] | { doctors?: { id: string;
name: string; title?: string }[] };
          setDoctorOptions(Array.isArray(d) ? d : (d.doctors ?? []));
        }
      });
    }
  }, [editing]);
  // Load nurses once
  useEffect(() => {
```

```
    fetch('/api/nurses')
      .then(r => r.ok ? r.json() : [])
      .then((d: { id: string; name: string; title?: string }[] | { nurses?: { id: string; name: string; title?:
string }[] }) => {
        setNurses(Array.isArray(d) ? d : (d.nurses ?? []));
      })
      .catch(() => {});
  }, []);
  async function saveSessionNurse(sessionKey: string, sessionResults: TestResult[]) {
    const nurseNameVal = sessionNurses[sessionKey] ?? (sessionResults[0]?.nurseName ?? '');
    setSessionNurseSaving(prev => ({ ...prev, [sessionKey]: true }));
    await Promise.allSettled(sessionResults.map(r =>
      fetch(`/api/test-results/${r.id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ nurseName: nurseNameVal || null }),
      })
    ));
    setSessionNurseSaving(prev => ({ ...prev, [sessionKey]: false }));
    load();
  }
  async function handleSave() {
    if (!patient) return;
    setSaving(true);
    try {
      const res = await fetch(`/api/patients/${patient.id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          name: editForm.name,
          phone: editForm.phone,
          homePhone: editForm.homePhone,
          email: editForm.email,
          street: editForm.street,
          city: editForm.city,
          state: editForm.state,
          zip: editForm.zip,
          status: editForm.status,
          physician: editForm.physician,
          clinicLocation: editForm.clinicLocation,
          diagnosis: editForm.diagnosis,
          notes: editForm.notes,
          insuranceId: editForm.insuranceId,
          insuranceProvider: editForm.insuranceProvider,
          insuranceGroup: editForm.insuranceGroup,
          emergencyName: editForm.emergencyName,
          emergencyPhone: editForm.emergencyPhone,
          emergencyRelation: editForm.emergencyRelation,
        }),
      });
      if (!res.ok) throw new Error('Save failed');
      setEditing(false);
      load();
    } catch (e) {
      alert((e as Error).message);
    } finally {
      setSaving(false);
    }
  }
  function fmt(val?: string) {
    if (!val) return '—';
    try {
      // Parse date and format in UTC to avoid timezone day-shift
      const d = new Date(val);
```

```
      return new Date(d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate())
        .toLocaleDateString('en-US', { month: 'long', day: 'numeric', year: 'numeric' });
    }
    catch { return val; }
  }
  function handlePrintResults() {
    if (!patient || tests.length === 0) return;
    const gradeLabel: Record<number, string> = { 0:'Negative',1:'Trace',2:'Positive',3:'Strong',4:'Very
Strong',5:'Extreme' };
    const gradeColor: Record<number, string> = { 0:'#64748b',1:'#ca8a04',2:'#ea580c',3:'#dc2626',4:'#991b1b',5:'#7
f1d1d' };
    // Group by date + testType
    const groups = new Map<string, TestResult[]>();
    tests.forEach(r => {
      const d = r.testedAt ? new Date(r.testedAt).toLocaleDateString('en-US',{month:'long',day:'numeric',year:'num
eric'}) : 'Unknown Date';
      const key = `${d}||${r.testType ?? 'scratch'}`;
      if (!groups.has(key)) groups.set(key, []);
      groups.get(key)!.push(r);
    });
    const sectionsHtml = Array.from(groups.entries()).map(([key, results]) => {
      const [dateLabel, testType] = key.split('||');
      const icon = testType === 'scratch' ? '■' : '■';
      const title = testType === 'scratch' ? 'Prick Test' : 'Intradermal Test';
      const rows = results.map((r, i) => {
        const notes = r.notes ?? '';
        const flareMatch = notes.match(/Flare:\s*([\d.]+\s*mm?)/i);
        const locMatch = notes.match(/Location:\s*([^;]+)/i);
        const flare = flareMatch ? flareMatch[1] : '—';
        const injectionSite = locMatch ? locMatch[1].trim() : 'Back';
        const grade = r.reaction ?? 0;
        return `<tr style="border-bottom:1px solid #e2e8f0; background:${grade >= 2 ? '#fff7ed' : i % 2 === 0 ? '
#fff' : '#f8fafc'}">
          <td style="padding:6px 10px;">${i+1}</td>
          <td style="padding:6px 10px; font-weight:600;">${r.allergen?.name ?? '—'}</td>
          <td style="padding:6px 10px; font-weight:800; font-size:16px; color:${gradeColor[grade]};">${grade}</td>
          <td style="padding:6px 10px; color:#64748b; font-size:12px;">${gradeLabel[grade]}</td>
          <td style="padding:6px 10px; color:#374151;">${r.wheal ?? '—'}</td>
          <td style="padding:6px 10px; color:#374151;">${flare}</td>
          <td style="padding:6px 10px; color:#64748b;">${injectionSite}</td>
          <td style="padding:6px 10px; color:#64748b; font-size:11px;">${dateLabel}</td>
        </tr>`;
      }).join('');
      return `<div style="margin-bottom:24px;">
        <h3 style="font-size:14px; font-weight:700; color:#0055A5; text-transform:uppercase;
letter-spacing:0.06em; margin:0 0 8px; padding-bottom:6px; border-bottom:2px solid #0055A5;">
          ${icon} ${title}  ·  <span style="font-weight:500; text-transform:none;">${dateLabel}</span>
        </h3>
        <table style="width:100%; border-collapse:collapse; font-size:13px; font-family:system-ui,sans-serif;">
          <thead><tr style="background:#0055A5; color:#fff;">
            <th style="padding:6px 10px; text-align:left;">#</th>
            <th style="padding:6px 10px; text-align:left;">Allergen</th>
            <th style="padding:6px 10px; text-align:left;">Grade</th>
            <th style="padding:6px 10px; text-align:left;">Result</th>
            <th style="padding:6px 10px; text-align:left;">Wheal</th>
            <th style="padding:6px 10px; text-align:left;">Flare</th>
            <th style="padding:6px 10px; text-align:left;">Site</th>
            <th style="padding:6px 10px; text-align:left;">Test Date</th>
          </tr></thead>
          <tbody>${rows}</tbody>
        </table>
      </div>`;
    }).join('');
    // Get nurse from dedicated nurseName field (falls back to notes for legacy data)
```

```
    const nurseFromField = tests[0]?.nurseName;
    const nurseFromNotes = tests[0]?.notes?.match(/Nurse:\s*([^;]+)/i)?.[1]?.trim();
    const nurseName = nurseFromField || nurseFromNotes || '—';
    const html = `<!DOCTYPE html><html><head><title>Test Results — ${patient.name}</title>
<style>body{font-family:system-ui,sans-serif;margin:0;padding:20px;color:#1a2233;}</style>
</head><body>
<div style="border-bottom:3px solid #0055A5;padding-bottom:12px;margin-bottom:16px;">
  <div style="font-size:20px;font-weight:800;color:#0055A5;margin-bottom:8px;">Integrated Allergy Testing — Test
Results</div>
  <div style="display:flex;gap:28px;font-size:13px;flex-wrap:wrap;">
    <span><strong>Patient:</strong> ${patient.name}</span>
    <span><strong>ID:</strong> ${patient.patientId ?? patient.id.slice(0,8).toUpperCase()}</span>
    <span><strong>DOB:</strong> ${fmt(patient.dob)}</span>
    <span><strong>Physician:</strong> ${patient.physician ?? '—'}</span>
    <span><strong>Tested By:</strong> ${nurseName}</span>
    <span><strong>Clinic Location:</strong> ${patient.clinicLocation ?? '—'}</span>
    <span><strong>Insurance ID:</strong> ${patient.insuranceId ?? '—'}</span>
    <span><strong>Print Date:</strong> ${new Date().toLocaleDateString('en-US',{month:'long',day:'numeric',year:'n
umeric'})}</span>
  </div>
</div>
<div style="background:#f0fdf4;border:1px solid #86efac;border-radius:6px;padding:8px
14px;margin-bottom:16px;font-size:12px;color:#166534;">
  <strong>Grade Legend:</strong> 0=Negative  |  1=Trace  |  2=Positive  |  3=Strong
 |  4=Very Strong  |  5=Extreme
</div>
${sectionsHtml}
<div style="margin-top:32px;border-top:1px solid #e2e8f0;padding-top:12px;font-size:11px;color:#94a3b8;display:fle
x;justify-content:space-between;">
  <span>Integrated Allergy Testing — HIPAA Compliant</span>
  <span>Printed: ${new Date().toLocaleString('en-US')}</span>
</div>
</body></html>`;
    const w = window.open('', '_blank', 'width=900,height=700');
    if (w) { w.document.write(html); w.document.close(); w.focus(); setTimeout(() => w.print(), 400); }
  }
  function InfoRow({ label, value }: { label: string; value?: string | null }) {
    return (
      <div style={{ display: 'flex', gap: 16, padding: '10px 0', borderBottom: '1px solid #f1f5f9' }}>
        <div style={{ width: 160, flexShrink: 0, fontSize: 13, fontWeight: 600, color: '#64748b' }}>{label}</div>
        <div style={{ fontSize: 14, color: value ? '#1a2233' : '#94a3b8' }}>{value || '—'}</div>
      </div>
    );
  }
  function Field({ label, field, type = 'text' }: { label: string; field: keyof Patient; type?: string }) {
    // For date fields, normalize ISO datetime to YYYY-MM-DD
    const rawVal = (editForm[field] as string) ?? '';
    // For date inputs: extract YYYY-MM-DD from ISO string, handling timezone
    const displayVal = type === 'date' && rawVal
      ? (() => {
          const s = rawVal.split('T')[0];
          // Validate it's a real date string
          if (/^\d{4}-\d{2}-\d{2}$/.test(s) && parseInt(s.substring(0,4)) > 1900) return s;
          // Fallback: parse and format as UTC
          try {
            const d = new Date(rawVal);
            return `${d.getUTCFullYear()}-${String(d.getUTCMonth()+1).padStart(2,'0')}-${String(d.getUTCDate()).pa
dStart(2,'0')}`;
          } catch { return s; }
        })()
      : rawVal;
    return (
      <div style={{ marginBottom: 14 }}>
```

```
        <label style={{ display: 'block', fontSize: 12, fontWeight: 600, color: '#374151', marginBottom: 4,
textTransform: 'uppercase', letterSpacing: '0.04em' }}>{label}</label>
        <input
          type={type}
          className="form-input"
          value={displayVal}
          onChange={e => setEditForm(f => ({ ...f, [field]: e.target.value }))}
        />
      </div>
    );
  }
  if (loading) return (
    <><div className="page-header"><div className="page-title">Patient Detail</div></div>
    <div className="page-body loading-center"><div className="spinner" /><span>Loading…</span></div></>
  );
  if (error || !patient) return (
    <><div className="page-header"><div className="page-title">Patient Not Found</div></div>
    <div className="page-body">
      <div className="alert alert-error">■■ {error || 'Patient not found'}</div>
      <Link href="/patients" className="btn">← Back to Patients</Link>
    </div></>
  );
  const statusStyle = STATUS_BADGE[patient.status?.toLowerCase()] ?? { color: '#374151', bg: '#f1f5f9' };
  const tests = patient.testResults ?? [];
  const videos = patient.videoActivity ?? [];
  const forms = patient.formActivity ?? [];
  return (
    <>
      <style>{`
        @media print {
          .sidebar, .sidebar-toggle, .sidebar-overlay, .page-header, .tabs-bar, .no-print { display: none !
important; }
          .main-content { margin-left: 0 !important; padding: 0 !important; }
          .print-header { display: block !important; margin-bottom: 16px; padding-bottom: 10px; border-bottom:
2px solid #0055A5; }
          body { background: #fff !important; }
        }
        .print-header { display: none; }
        .tabs-bar { /* referenced above */ }
      `}</style>
      {/* Print-only patient header */}
      <div className="print-header">
        <div style={{ fontSize: 18, fontWeight: 800, color: '#0055A5', marginBottom: 6 }}>Integrated Allergy
Testing — Patient Test Results</div>
        <div style={{ display: 'flex', gap: 24, fontSize: 13, flexWrap: 'wrap' }}>
          <span><strong>Patient:</strong> {patient.name}</span>
          <span><strong>ID:</strong> {patient.patientId ?? patient.id.slice(0,8).toUpperCase()}</span>
          <span><strong>DOB:</strong> {fmt(patient.dob)}</span>
          <span><strong>Physician:</strong> {patient.physician ?? '—'}</span>
          <span><strong>Location:</strong> {patient.clinicLocation ?? '—'}</span>
          <span><strong>Print Date:</strong> {new Date().toLocaleDateString('en-US', { month: 'long', day: '
numeric', year: 'numeric' })}</span>
        </div>
      </div>
      {/* Header */}
      <div className="page-header no-print">
        <div>
          <div className="page-title">{patient.name}</div>
          <div style={{ display: 'flex', gap: 8, alignItems: 'center', marginTop: 4 }}>
            <code style={{ fontSize: 12, background: '#f1f5f9', padding: '2px 8px', borderRadius: 4 }}>
              {patient.patientId ?? patient.id.slice(0, 8).toUpperCase()}
            </code>
            <span style={{ fontSize: 12, fontWeight: 700, padding: '2px 10px', borderRadius: 999, color:
statusStyle.color, background: statusStyle.bg }}>
```

```
              {patient.status}
            </span>
          </div>
        </div>
        <div style={{ display: 'flex', gap: 8 }}>
          <Link href="/patients" style={{ padding: '8px 16px', borderRadius: 8, border: '1px solid #e2e8f0',
background: '#fff', color: '#374151', fontSize: 14, fontWeight: 600, textDecoration: 'none' }}>← Patients</Link>
          <button onClick={() => { setEditForm(patient); setEditing(true); }}
            style={{ padding: '8px 16px', borderRadius: 8, border: '1px solid #0d9488', background: '#fff',
color: '#0d9488', fontSize: 14, fontWeight: 700, cursor: 'pointer' }}>
            ✏ Edit
          </button>
          <Link href={`/testing?patientId=${patient.id}`}
            style={{ padding: '8px 16px', borderRadius: 8, background: '#0d9488', color: '#fff', fontSize: 14,
fontWeight: 700, textDecoration: 'none' }}>
            ▶ Start Testing
          </Link>
          <Link href={`/calendar?action=new&patientId=${patient.id}&patientName=${encodeURIComponent(patient.name)
}`}
            style={{ padding: '8px 16px', borderRadius: 8, background: '#7c3aed', color: '#fff', fontSize: 14,
fontWeight: 700, textDecoration: 'none' }}>
            ■ Book Appt
          </Link>
        </div>
      </div>
      <div className="page-body">
        {/* Tabs */}
        <div className="no-print" style={{ display: 'flex', gap: 4, borderBottom: '2px solid #e2e8f0',
marginBottom: 24 }}>
          {(['overview', 'tests', 'videos', 'forms', 'consent', 'encounters'] as Tab[]).map(t => (
            <button key={t} onClick={() => setTab(t)} style={{
              padding: '10px 20px', border: 'none', background: 'transparent', fontSize: 14, fontWeight: 600,
              color: tab === t ? '#0d9488' : '#64748b', cursor: 'pointer',
              borderBottom: tab === t ? '2px solid #0d9488' : '2px solid transparent', marginBottom: -2,
            }}>
              {t === 'overview' && '■ Overview'}
              {t === 'tests' && `■ Test Results (${tests.length})`}
              {t === 'videos' && `■ Videos (${videos.length})`}
              {t === 'forms' && `■ Forms (${forms.length})`}
              {t === 'consent' && '■ Consent'}
              {t === 'encounters' && '■ Encounters'}
            </button>
          ))}
        </div>
        {/* Overview */}
        {tab === 'overview' && (
          <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 24 }}>
            <div className="card">
              <div className="card-title">Personal Information</div>
              <InfoRow label="Full Name" value={patient.name} />
              <InfoRow label="Date of Birth" value={fmt(patient.dob)} />
              <InfoRow label="Cell Phone" value={patient.phone} />
              <InfoRow label="Home Phone" value={patient.homePhone} />
              <InfoRow label="Email" value={patient.email} />
              {(patient.street || patient.city) && (
                <InfoRow label="Address" value={[patient.street, patient.apt, patient.city, patient.state,
patient.zip].filter(Boolean).join(', ')} />
              )}
              {(patient.emergencyName) && (
                <>
                  <InfoRow label="Emergency" value={`${patient.emergencyName}${patient.emergencyRelation ? `
(${patient.emergencyRelation})` : ''}`} />
                  <InfoRow label="Emerg. Phone" value={patient.emergencyPhone} />
                </>
```

```
              )}
              <InfoRow label="Insurance" value={[patient.insuranceProvider, patient.insuranceId,
patient.insuranceGroup ? `Group: ${patient.insuranceGroup}` : ''].filter(Boolean).join(' · ')} />
            </div>
            <div className="card">
              <div className="card-title">Clinical Information</div>
              <InfoRow label="Status" value={patient.status} />
              <InfoRow label="Physician" value={patient.physician || patient.doctor?.name} />
              <InfoRow label="Location" value={patient.clinicLocation} />
              <InfoRow label="Diagnosis" value={patient.diagnosis} />
              <InfoRow label="Registered" value={fmt(patient.createdAt)} />
              <InfoRow label="Last Updated" value={fmt(patient.updatedAt)} />
              {patient.notes && <InfoRow label="Notes" value={patient.notes} />}
            </div>
          </div>
        )}
        {/* Test Results */}
        {tab === 'tests' && (
          <div>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
16 }}>
              <div style={{ fontSize: 16, fontWeight: 700, color: '#1a2233' }}>Test Results ({tests.length})</div>
              <div style={{ display: 'flex', gap: 8 }}>
                {tests.length > 0 && (
                  <button onClick={handlePrintResults}
                    style={{ padding: '8px 16px', borderRadius: 8, border: '1px solid #e2e8f0', background: '
#fff', color: '#374151', fontSize: 14, fontWeight: 600, cursor: 'pointer' }}>
                    ■■ Print Results
                  </button>
                )}
                <Link href={`/testing?patientId=${patient.id}`} style={{ padding: '8px 16px', borderRadius: 8,
background: '#0d9488', color: '#fff', fontSize: 14, fontWeight: 700, textDecoration: 'none' }}>■ Start New
Test</Link>
              </div>
            </div>
            {tests.length === 0 ? (
              <div className="card">
                <div className="empty-state">
                  <div className="empty-state-icon">■</div>
                  <div className="empty-state-title">No test results yet</div>
                  <Link href={`/testing?patientId=${patient.id}`} className="btn" style={{ marginTop: 12 }}>Start
Testing</Link>
                </div>
              </div>
            ) : (() => {
              // Group by test date (YYYY-MM-DD) and testType
              const groups = new Map<string, TestResult[]>();
              tests.forEach(r => {
                const dateKey = r.testedAt ? new Date(r.testedAt).toLocaleDateString('en-US', { month: 'long',
day: 'numeric', year: 'numeric' }) : 'Unknown Date';
                const key = `${dateKey}||${r.testType ?? 'scratch'}`;
                if (!groups.has(key)) groups.set(key, []);
                groups.get(key)!.push(r);
              });
              return Array.from(groups.entries()).map(([key, results]) => {
                const [dateLabel, testType] = key.split('||');
                return (
                  <div key={key} className="card" style={{ marginBottom: 16 }}>
                    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: 12, flexWrap: 'wrap', gap: 8 }}>
                      <div>
                        <div style={{ fontWeight: 700, fontSize: 15, color: '#0d9488' }}>{dateLabel}</div>
                        <span style={{ fontSize: 12, fontWeight: 600, padding: '2px 10px', borderRadius: 999,
background: '#dbeafe', color: '#1d4ed8', textTransform: 'capitalize' }}>
                          {testType === 'scratch' ? '■ Prick Test' : '■ Intradermal'}
```

```
                        </span>
                      </div>
                      <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
                        <span style={{ fontSize: 12, fontWeight: 600, color: '#64748b' }}>Tested By:</span>
                        <select
                          value={sessionNurses[key] ?? (results[0]?.nurseName ?? '')}
                          onChange={e => setSessionNurses(prev => ({ ...prev, [key]: e.target.value }))}
                          style={{ fontSize: 13, padding: '4px 10px', borderRadius: 6, border: '1px solid
#e2e8f0', color: '#374151', cursor: 'pointer' }}
                        >
                          <option value="">— Assign Nurse —</option>
                          {nurses.map(n => (
                            <option key={n.id} value={n.name}>{n.title ? `${n.title} ${n.name}` : n.name}</option>
                          ))}
                        </select>
                        <button
                          onClick={() => saveSessionNurse(key, results)}
                          disabled={sessionNurseSaving[key]}
                          style={{ padding: '4px 12px', borderRadius: 6, border: 'none', background: '#0d9488',
color: '#fff', fontSize: 12, fontWeight: 700, cursor: 'pointer' }}
                        >
                          {sessionNurseSaving[key] ? '■' : '■ Save'}
                        </button>
                      </div>
                    </div>
                    <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 14 }}>
                      <thead>
                        <tr style={{ background: '#f8fafc' }}>
                          {['Allergen', 'Reaction', 'Wheal', 'Flare', 'Site', 'Notes', ''].map(h => (
                            <th key={h} style={{ padding: '8px 12px', textAlign: 'left', fontSize: 12,
fontWeight: 700, color: '#374151', textTransform: 'uppercase', borderBottom: '2px solid #e2e8f0' }}>{h}</th>
                          ))}
                        </tr>
                      </thead>
                      <tbody>
                        {results.map(r => {
                          const rowNotes = r.notes ?? '';
                          const flareMatch = rowNotes.match(/Flare:\s*([\d.]+\s*mm?)/i);
                          const locMatch = rowNotes.match(/Location:\s*([^;]+)/i);
                          const parsedFlare = flareMatch ? flareMatch[1].replace(/mm?/i,'').trim() : '';
                          const parsedSite = locMatch ? locMatch[1].trim() : 'Back';
                          const cleanNotes = rowNotes.replace(/Flare:\s*[\d.]+\s*mm?;?\s*/i,'').replace(/Location:
\s*[^;]+;?\s*/i,'').trim();
                          const editing = editingRows[r.id] ?? {
                            reaction: r.reaction ?? 0,
                            wheal: r.wheal ?? '',
                            flare: parsedFlare,
                            notes: cleanNotes,
                          };
                          async function saveRow() {
                            setSavingRows(prev => ({ ...prev, [r.id]: true }));
                            const newNotes = [
                              editing.flare ? `Flare: ${editing.flare}mm` : '',
                              parsedSite !== 'Back' ? `Location: ${parsedSite}` : '',
                              editing.notes || '',
                            ].filter(Boolean).join('; ');
                            await fetch(`/api/test-results/${r.id}`, {
                              method: 'PUT',
                              headers: { 'Content-Type': 'application/json' },
                              body: JSON.stringify({
                                reaction: editing.reaction,
                                wheal: editing.wheal || null,
                                notes: newNotes || null,
                              }),
```

```
                            });
                            setSavingRows(prev => ({ ...prev, [r.id]: false }));
                            setEditingRows(prev => { const n = {...prev}; delete n[r.id]; return n; });
                            load();
                          }
                          const changed = editing.reaction !== (r.reaction ?? 0) ||
                            editing.wheal !== (r.wheal ?? '') ||
                            editing.flare !== parsedFlare ||
                            editing.notes !== cleanNotes;
                          return (
                            <tr key={r.id} style={{ borderBottom: '1px solid #f1f5f9', background:
editing.reaction >= 2 ? '#fff7ed' : 'transparent' }}>
                              <td style={{ padding: '8px 12px', fontWeight: 600, fontSize: 13
}}>{r.allergen?.name ?? '—'}</td>
                              <td style={{ padding: '8px 8px' }}>
                                <select value={editing.reaction}
                                  onChange={e => setEditingRows(prev => ({ ...prev, [r.id]: { ...editing,
reaction: Number(e.target.value) } }))}
                                  style={{ fontWeight: 700, fontSize: 14, color:
REACTION_COLOR[editing.reaction], border: '1px solid #e2e8f0', borderRadius: 6, padding: '2px 6px', width: 52 }}>
                                  {[0,1,2,3,4,5].map(g => <option key={g} value={g}>{g}</option>)}
                                </select>
                              </td>
                              <td style={{ padding: '8px 6px' }}>
                                <input type="text" value={editing.wheal} placeholder="mm"
                                  onChange={e => setEditingRows(prev => ({ ...prev, [r.id]: { ...editing, wheal:
e.target.value } }))}
                                  style={{ width: 52, padding: '3px 6px', border: '1px solid #e2e8f0',
borderRadius: 6, fontSize: 13 }} />
                              </td>
                              <td style={{ padding: '8px 6px' }}>
                                <input type="text" value={editing.flare} placeholder="mm"
                                  onChange={e => setEditingRows(prev => ({ ...prev, [r.id]: { ...editing, flare:
e.target.value } }))}
                                  style={{ width: 52, padding: '3px 6px', border: '1px solid #e2e8f0',
borderRadius: 6, fontSize: 13 }} />
                              </td>
                              <td style={{ padding: '8px 12px', color: '#64748b', fontSize: 13
}}>{parsedSite}</td>
                              <td style={{ padding: '8px 6px' }}>
                                <input type="text" value={editing.notes} placeholder="Notes…"
                                  onChange={e => setEditingRows(prev => ({ ...prev, [r.id]: { ...editing, notes:
e.target.value } }))}
                                  style={{ width: 120, padding: '3px 6px', border: '1px solid #e2e8f0',
borderRadius: 6, fontSize: 12 }} />
                              </td>
                              <td style={{ padding: '8px 8px' }}>
                                {changed && (
                                  <button onClick={saveRow} disabled={savingRows[r.id]}
                                    style={{ padding: '3px 10px', borderRadius: 6, border: 'none', background: '
#0d9488', color: '#fff', fontSize: 12, fontWeight: 700, cursor: 'pointer', whiteSpace: 'nowrap' }}>
                                    {savingRows[r.id] ? '■' : '■ Save'}
                                  </button>
                                )}
                              </td>
                            </tr>
                          );
                        })}
                      </tbody>
                    </table>
                  </div>
                );
              });
            })()}
```

```
          </div>
        )}
        {/* Videos */}
        {tab === 'videos' && (
          <div className="card">
            <div className="card-title">Videos Watched</div>
            {videos.length === 0 ? (
              <div className="empty-state"><div className="empty-state-icon">■</div><div
className="empty-state-title">No videos watched yet</div></div>
            ) : (
              <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 14 }}>
                <thead><tr style={{ background: '#f8fafc' }}>{['Video','Watched','Acknowledged'].map(h=><th
key={h} style={{ padding:'8px 12px',textAlign:'left',fontSize:12,fontWeight:700,color:'#374151',textTransform:'upp
ercase',borderBottom:'2px solid #e2e8f0'}}>{h}</th>)}</tr></thead>
                <tbody>{videos.map(v=><tr key={v.id} style={{borderBottom:'1px solid #f1f5f9'}}><td
style={{padding:'10px 12px',fontWeight:600}}>{v.video?.title??'—'}</td><td style={{padding:'10px
12px',color:'#64748b'}}>{fmt(v.watchedAt)}</td><td style={{padding:'10px
12px'}}>{v.acknowledged?'■':'—'}</td></tr>)}</tbody>
              </table>
            )}
          </div>
        )}
        {/* Forms */}
        {tab === 'forms' && (
          <div className="card">
            <div className="card-title">Forms</div>
            {forms.length === 0 ? (
              <div className="empty-state"><div className="empty-state-icon">■</div><div
className="empty-state-title">No forms signed yet</div></div>
            ) : (
              <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 14 }}>
                <thead><tr style={{ background: '#f8fafc' }}>{['Form','Signed','Printed','Emailed'].map(h=><th
key={h} style={{padding:'8px 12px',textAlign:'left',fontSize:12,fontWeight:700,color:'#374151',textTransform:'uppe
rcase',borderBottom:'2px solid #e2e8f0'}}>{h}</th>)}</tr></thead>
                <tbody>{forms.map(f=><tr key={f.id} style={{borderBottom:'1px solid #f1f5f9'}}><td
style={{padding:'10px 12px',fontWeight:600}}>{f.form?.name??'—'}</td><td style={{padding:'10px
12px',color:'#64748b'}}>{fmt(f.signedAt)}</td><td style={{padding:'10px
12px',color:'#64748b'}}>{fmt(f.printedAt)}</td><td style={{padding:'10px
12px',color:'#64748b'}}>{fmt(f.emailedAt)}</td></tr>)}</tbody>
              </table>
            )}
          </div>
        )}
        {/* Consent */}
        {tab === 'consent' && (
          <div>
            <div style={{ fontSize: 16, fontWeight: 700, color: '#1a2233', marginBottom: 16 }}>Consent Forms</div>
            <ConsentStatus patientId={patient.id} />
          </div>
        )}
        {tab === 'encounters' && (
          <EncountersTab patientId={patient.id} patientName={patient.name} />
        )}
      </div>
      {/* Edit Modal */}
      {editing && (
        <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.5)', display: 'flex', alignItems: '
center', justifyContent: 'center', zIndex: 1000 }}>
          <div style={{ background: '#fff', borderRadius: 16, width: 580, maxHeight: '90vh', overflowY: 'auto',
boxShadow: '0 20px 60px rgba(0,0,0,0.2)' }}>
            <div style={{ padding: '20px 24px', borderBottom: '1px solid #e2e8f0', display: 'flex',
justifyContent: 'space-between', alignItems: 'center' }}>
              <h2 style={{ margin: 0, fontSize: 18, fontWeight: 700, color: '#1a2233' }}>✏■ Edit Patient</h2>
```

```
              <button onClick={() => setEditing(false)} style={{ background: 'none', border: 'none', fontSize:
20, cursor: 'pointer', color: '#64748b' }}>✕</button>
            </div>
            <div style={{ padding: 24 }}>
              <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 12 }}>
                <div style={{ gridColumn: '1/-1' }}><Field label="Full Name *" field="name" /></div>
                <Field label="Date of Birth" field="dob" type="date" />
                <Field label="Cell Phone" field="phone" />
                <Field label="Home Phone" field="homePhone" />
                <Field label="Email" field="email" type="email" />
               <div style={{ gridColumn: '1/-1', fontWeight: 700, fontSize: 12, color: '#374151', textTransform: '
uppercase', letterSpacing: '0.05em', paddingTop: 8, borderTop: '1px solid #e2e8f0' }}>Address</div>
                <Field label="Street" field="street" />
                <Field label="Apt / Suite" field="apt" />
                <Field label="City" field="city" />
                <Field label="State" field="state" />
                <Field label="ZIP" field="zip" />
               <div style={{ gridColumn: '1/-1', fontWeight: 700, fontSize: 12, color: '#374151', textTransform: '
uppercase', letterSpacing: '0.05em', paddingTop: 8, borderTop: '1px solid #e2e8f0' }}>Insurance</div>
                <Field label="Insurance Provider" field="insuranceProvider" />
                <Field label="Member ID" field="insuranceId" />
                <Field label="Group #" field="insuranceGroup" />
               <div style={{ gridColumn: '1/-1', fontWeight: 700, fontSize: 12, color: '#374151', textTransform: '
uppercase', letterSpacing: '0.05em', paddingTop: 8, borderTop: '1px solid #e2e8f0' }}>Emergency Contact</div>
                <Field label="Name" field="emergencyName" />
                <Field label="Phone" field="emergencyPhone" />
                <Field label="Relationship" field="emergencyRelation" />
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 600, color: '#374151',
marginBottom: 4, textTransform: 'uppercase', letterSpacing: '0.04em' }}>Status</label>
                  <select className="form-input" value={editForm.status ?? ''} onChange={e => setEditForm(f => ({ .
..f, status: e.target.value }))}>
                    {['build-up', 'maintenance', 'registered', 'tested', 'consented', 'complete'].map(s => (
                      <option key={s} value={s}>{s}</option>
                    ))}
                  </select>
                </div>
                <div style={{ marginBottom: 14 }}>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 600, color: '#374151',
marginBottom: 4, textTransform: 'uppercase', letterSpacing: '0.04em' }}>Physician</label>
                  <select className="form-input" value={editForm.physician ?? ''} onChange={e => setEditForm(f =>
({ ...f, physician: e.target.value }))}>
                    <option value="">— Select Physician —</option>
                    {doctorOptions.map(d => (
                      <option key={d.id} value={`${d.title ? d.title + ' ' : ''}${d.name}`}>
                        {d.title ? `${d.title} ${d.name}` : d.name}
                      </option>
                    ))}
                  </select>
                </div>
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 600, color: '#374151',
marginBottom: 4, textTransform: 'uppercase', letterSpacing: '0.04em' }}>Clinic Location</label>
                  <select className="form-input" value={editForm.clinicLocation ?? ''} onChange={e =>
setEditForm(f => ({ ...f, clinicLocation: e.target.value }))}>
                    <option value="">— Select Location —</option>
                    {locationOptions.map(loc => (
                      <option key={loc.id} value={loc.name}>{loc.name}</option>
                    ))}
                  </select>
                </div>
                <div style={{ gridColumn: '1/-1' }}><Field label="Diagnosis" field="diagnosis" /></div>
                <div style={{ gridColumn: '1/-1' }}>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 600, color: '#374151',
marginBottom: 4, textTransform: 'uppercase', letterSpacing: '0.04em' }}>Notes</label>
```

```
                  <textarea className="form-input" rows={3} value={editForm.notes ?? ''} onChange={e =>
setEditForm(f => ({ ...f, notes: e.target.value }))} style={{ resize: 'vertical' }} />
                </div>
              </div>
            </div>
            <div style={{ padding: '16px 24px', borderTop: '1px solid #e2e8f0', display: 'flex', gap: 10,
justifyContent: 'flex-end' }}>
              <button onClick={() => setEditing(false)} style={{ padding: '8px 20px', borderRadius: 8, border: '
1px solid #e2e8f0', background: '#fff', color: '#374151', cursor: 'pointer', fontWeight: 600 }}>Cancel</button>
              <button onClick={handleSave} disabled={saving} style={{ padding: '8px 20px', borderRadius: 8,
border: 'none', background: '#0d9488', color: '#fff', cursor: 'pointer', fontWeight: 700 }}>
                {saving ? '■ Saving…' : '■ Save Changes'}
              </button>
            </div>
          </div>
        </div>
      )}
    </>
  );
}
function ConsentStatus({ patientId }: { patientId: string }) {
  const [forms, setForms] = useState<{ formId: string; name: string; signed: boolean; signedAt?: string }[]>([])
  const [loading, setLoading] = useState(true)
  const [nurses, setNurses] = useState<{ id: string; name: string; title?: string }[]>([])
  const [ackMap, setAckMap] = useState<Record<string, string>>({})
  const [ackSaving, setAckSaving] = useState<Record<string, boolean>>({})
  useEffect(() => {
    Promise.allSettled([
      fetch(`/api/consent/check?patientId=${patientId}`).then(r => r.json()),
      fetch('/api/nurses').then(r => r.json()),
    ]).then(([formsRes, nursesRes]) => {
      if (formsRes.status === 'fulfilled') setForms(formsRes.value.forms ?? [])
      if (nursesRes.status === 'fulfilled') {
        const d = nursesRes.value
        setNurses(Array.isArray(d) ? d : (d.nurses ?? []))
      }
      setLoading(false)
    }).catch(() => setLoading(false))
  }, [patientId])
  async function ackForm(formId: string, nurseName: string) {
    if (!nurseName) return
    setAckSaving(prev => ({ ...prev, [formId]: true }))
    setAckMap(prev => ({ ...prev, [formId]: nurseName }))
    // Store ack in video activity notes for now — future: dedicated ack table
    // For now just show visual confirmation
    setTimeout(() => setAckSaving(prev => ({ ...prev, [formId]: false })), 500)
  }
  if (loading) return <div className="loading-center"><div className="spinner" /></div>
  if (forms.length === 0) return (
    <div className="card">
      <div className="empty-state">
        <div className="empty-state-icon">■</div>
        <div className="empty-state-title">No consent forms configured</div>
      </div>
    </div>
  )
  return (
    <div style={{ display: 'flex', flexDirection: 'column', gap: 12 }}>
      {forms.map(f => (
        <div key={f.formId} className="card" style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center' }}>
          <div>
            <div style={{ fontWeight: 700, fontSize: 15, marginBottom: 4 }}>{f.name}</div>
            {f.signed ? (
```

```
              <div style={{ fontSize: 13, color: '#15803d' }}>
                ■ Signed on {new Date(f.signedAt!).toLocaleDateString('en-US', { month: 'long', day: 'numeric',
year: 'numeric' })}
              </div>
            ) : (
              <div style={{ fontSize: 13, color: '#b45309' }}>■■ Not yet signed</div>
            )}
          </div>
          <div style={{ display: 'flex', gap: 8, alignItems: 'center' }}>
            {f.signed && ackMap[f.formId] ? (
              <span style={{ fontSize: 13, fontWeight: 700, color: '#15803d', background: '#dcfce7', padding: '
4px 12px', borderRadius: 999 }}>
                ■ Verified by {ackMap[f.formId]}
              </span>
            ) : f.signed && nurses.length > 0 ? (
              <select
                defaultValue=""
                disabled={ackSaving[f.formId]}
                onChange={e => e.target.value && ackForm(f.formId, e.target.value)}
               style={{ fontSize: 12, padding: '4px 10px', borderRadius: 6, border: '1px solid #e2e8f0', cursor: '
pointer', color: '#374151' }}
              >
                <option value="">✓ Nurse Verify</option>
                {nurses.map(n => <option key={n.id} value={n.name}>{n.title ? `${n.title} ${n.name}` :
n.name}</option>)}
              </select>
            ) : null}
            {f.signed && (
              <a
                href={`/api/consent/pdf?patientId=${patientId}&formId=${f.formId}`}
                target="_blank"
                rel="noopener noreferrer"
                style={{ padding: '6px 14px', borderRadius: 8, border: '1px solid #e2e8f0', background: '#fff',
color: '#374151', fontSize: 13, fontWeight: 600, textDecoration: 'none' }}
              >
                ■ Download PDF
              </a>
            )}
            {!f.signed && (
              <span style={{ padding: '6px 14px', borderRadius: 8, background: '#fef9c3', color: '#b45309',
fontSize: 13, fontWeight: 600 }}>
                Pending Patient Signature
              </span>
            )}
          </div>
        </div>
      ))}
    </div>
  )
}
function EncountersTab({ patientId, patientName }: { patientId: string; patientName: string }) {
  const [encounters, setEncounters] = useState<{ id: string; encounterDate: string; doctorName?: string;
nurseName?: string; chiefComplaint: string; assessment?: string; plan?: string; status: string }[]>([]);
  const [loading, setLoading] = useState(true);
  const [showNew, setShowNew] = useState(false);
  const [saving, setSaving] = useState(false);
  const [form, setForm] = useState({ chiefComplaint: '', subjectiveNotes: '', objectiveNotes: '', assessment: '',
plan: '', doctorName: '', nurseName: '', followUpDays: '', status: 'open' });
  function load() {
    setLoading(true);
    fetch(`/api/encounters?patientId=${patientId}`)
      .then(r => r.json())
      .then(d => { setEncounters(d.encounters ?? []); setLoading(false); })
      .catch(() => setLoading(false));
```

```
  }
  useEffect(() => { load(); }, [patientId]);
  async function saveEncounter() {
    setSaving(true);
    await fetch('/api/encounters', { method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ patientId, ...form, followUpDays: form.followUpDays ? Number(form.followUpDays) :
null }) });
    setSaving(false); setShowNew(false);
    setForm({ chiefComplaint: '', subjectiveNotes: '', objectiveNotes: '', assessment: '', plan: '', doctorName: '
', nurseName: '', followUpDays: '', status: 'open' });
    load();
  }
  const STATUS = { open: { bg: '#fef9c3', color: '#b45309' }, complete: { bg: '#dcfce7', color: '#15803d' },
cancelled: { bg: '#f3f4f6', color: '#64748b' } } as Record<string, { bg: string; color: string }>;
  const fmt = (d: string) => { try { return new Date(d).toLocaleDateString('en-US', { month: 'short', day: '
numeric', year: 'numeric' }); } catch { return d; }};
  return (
    <div>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: 16 }}>
        <div style={{ fontSize: 16, fontWeight: 700, color: '#1a2233' }}>Encounter Records
({encounters.length})</div>
        <button onClick={() => setShowNew(true)} style={{ padding: '8px 16px', borderRadius: 8, background: '
#0d9488', color: '#fff', border: 'none', fontSize: 14, fontWeight: 700, cursor: 'pointer' }}>+ New
Encounter</button>
      </div>
      {loading ? <div className="loading-center"><div className="spinner" /></div> :
       encounters.length === 0 ? (
        <div className="card" style={{ textAlign: 'center', padding: 32 }}>
          <div style={{ fontSize: 32, marginBottom: 8 }}>■</div>
          <div style={{ fontWeight: 700, marginBottom: 4 }}>No encounters yet</div>
          <button onClick={() => setShowNew(true)} className="btn" style={{ marginTop: 12 }}>Create First
Encounter</button>
        </div>
      ) : (
        <div style={{ display: 'flex', flexDirection: 'column', gap: 12 }}>
          {encounters.map(e => {
            const s = STATUS[e.status] ?? STATUS.open;
            return (
              <div key={e.id} className="card" style={{ padding: 16 }}>
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start',
marginBottom: 8 }}>
                  <div style={{ fontWeight: 700, fontSize: 15 }}>{e.chiefComplaint}</div>
                  <span style={{ fontSize: 12, fontWeight: 700, padding: '2px 10px', borderRadius: 999, ...s
}}>{e.status}</span>
                </div>
                <div style={{ fontSize: 13, color: '#64748b', marginBottom: 8 }}>
                  ■ {fmt(e.encounterDate)} {e.doctorName && `· ■■■■ ${e.doctorName}`} {e.nurseName && `· ■■■■
${e.nurseName}`}
                </div>
                {e.assessment && <div style={{ fontSize: 13, marginBottom: 4 }}><strong>Assessment:</strong>
{e.assessment}</div>}
                {e.plan && <div style={{ fontSize: 13 }}><strong>Plan:</strong> {e.plan}</div>}
              </div>
            );
          })}
        </div>
      )}
      {showNew && (
        <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.5)', display: 'flex', alignItems: '
center', justifyContent: 'center', zIndex: 1100 }}>
          <div style={{ background: '#fff', borderRadius: 16, width: 620, maxHeight: '90vh', overflowY: 'auto',
boxShadow: '0 20px 60px rgba(0,0,0,0.2)' }}>
            <div style={{ padding: '16px 24px', borderBottom: '1px solid #e2e8f0', display: 'flex',
justifyContent: 'space-between', alignItems: 'center' }}>
```

```
              <h2 style={{ margin: 0, fontSize: 17, fontWeight: 700 }}>■ New Encounter — {patientName}</h2>
              <button onClick={() => setShowNew(false)} style={{ background: 'none', border: 'none', fontSize:
20, cursor: 'pointer' }}>✕</button>
            </div>
            <div style={{ padding: 24, display: 'flex', flexDirection: 'column', gap: 14 }}>
              {[
                { label: 'Chief Complaint *', key: 'chiefComplaint', required: true },
                { label: 'Physician', key: 'doctorName' },
                { label: 'Nurse', key: 'nurseName' },
              ].map(f => (
                <div key={f.key}>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>{f.label}</label>
                  <input className="form-input" value={(form as Record<string,string>)[f.key]} onChange={e =>
setForm(p => ({ ...p, [f.key]: e.target.value }))} />
                </div>
              ))}
              {[
                { label: 'Subjective (Patient reports)', key: 'subjectiveNotes' },
                { label: 'Objective (Clinical observations)', key: 'objectiveNotes' },
                { label: 'Assessment / Diagnosis', key: 'assessment' },
                { label: 'Plan / Treatment', key: 'plan' },
              ].map(f => (
                <div key={f.key}>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>{f.label}</label>
                  <textarea className="form-input" rows={3} value={(form as Record<string,string>)[f.key]}
onChange={e => setForm(p => ({ ...p, [f.key]: e.target.value }))} style={{ resize: 'vertical' }} />
                </div>
              ))}
              <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 12 }}>
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Follow-up (days)</label>
                  <input className="form-input" type="number" value={form.followUpDays} onChange={e => setForm(p
=> ({ ...p, followUpDays: e.target.value }))} placeholder="e.g. 30" />
                </div>
                <div>
                  <label style={{ display: 'block', fontSize: 12, fontWeight: 700, color: '#374151',
marginBottom: 4, textTransform: 'uppercase' }}>Status</label>
                  <select className="form-input" value={form.status} onChange={e => setForm(p => ({ ...p, status:
e.target.value }))}>
                    <option value="open">Open</option>
                    <option value="complete">Complete</option>
                    <option value="cancelled">Cancelled</option>
                  </select>
                </div>
              </div>
            </div>
            <div style={{ padding: '14px 24px', borderTop: '1px solid #e2e8f0', display: 'flex', gap: 10,
justifyContent: 'flex-end' }}>
              <button onClick={() => setShowNew(false)} style={{ padding: '8px 18px', borderRadius: 8, border: '
1px solid #e2e8f0', background: '#fff', cursor: 'pointer', fontWeight: 600 }}>Cancel</button>
              <button onClick={saveEncounter} disabled={saving || !form.chiefComplaint.trim()} style={{ padding: '
8px 18px', borderRadius: 8, border: 'none', background: '#0d9488', color: '#fff', cursor: 'pointer', fontWeight:
700 }}>
                {saving ? '■ Saving…' : '■ Save Encounter'}
              </button>
            </div>
          </div>
        </div>
      )}
    </div>
  );
}
```

page.tsx

app/calendar/page.tsx

```
'use client';
import { useEffect, useState, useCallback, Suspense } from 'react';
import { useSearchParams } from 'next/navigation';
import Link from 'next/link';
interface Appointment {
  id: string;
  title: string;
  patientId?: string | null;
  patientName?: string | null;
  startTime: string;
  endTime: string;
  type: string;
  notes?: string | null;
  status: string;
}
interface Patient {
  id: string;
  patientId: string;
  name: string;
}
interface AppointmentReason {
  id: string;
  name: string;
  color: string;
  duration: number;
  active: boolean;
}
function getReasonIcon(name: string): string {
  const n = name.toLowerCase();
  if (n.includes('allergy shot') || n === 'shot') return '■';
  if (n.includes('allergy test') || n.includes('testing')) return '■';
  if (n.includes('new patient') || n.includes('intake')) return '■';
  if (n.includes('follow') || n.includes('follow-up')) return '■';
  if (n.includes('consultation')) return '■■■■';
  if (n.includes('test results') || n.includes('results review')) return '■';
  if (n.includes('immunotherapy')) return '■';
  return '■';
}
function hexToRgba(hex: string, alpha: number) {
  const r = parseInt(hex.slice(1, 3), 16);
  const g = parseInt(hex.slice(3, 5), 16);
  const b = parseInt(hex.slice(5, 7), 16);
  return `rgba(${r},${g},${b},${alpha})`;
}
const HOURS = Array.from({ length: 12 }, (_, i) => i + 7); // 7am–6pm
function formatHour(h: number) {
  if (h === 12) return '12pm';
  if (h > 12) return `${h - 12}pm`;
  return `${h}am`;
}
function isoDate(d: Date) {
  return d.toISOString().slice(0, 10);
}
function startOfWeek(d: Date) {
  const copy = new Date(d);
  const day = copy.getDay();
  const diff = day === 0 ? -6 : 1 - day;
  copy.setDate(copy.getDate() + diff);
  copy.setHours(0, 0, 0, 0);
  return copy;
```

```
}
function addDays(d: Date, n: number) {
  const copy = new Date(d);
  copy.setDate(copy.getDate() + n);
  return copy;
}
function sameDay(a: Date, b: Date) {
  return a.getFullYear() === b.getFullYear() &&
    a.getMonth() === b.getMonth() &&
    a.getDate() === b.getDate();
}
function apptHour(appt: Appointment) {
  return new Date(appt.startTime).getHours();
}
function apptDay(appt: Appointment) {
  return new Date(appt.startTime);
}
function formatTime(iso: string) {
  const d = new Date(iso);
  const h = d.getHours();
  const m = d.getMinutes();
  const ampm = h >= 12 ? 'pm' : 'am';
  const displayH = h % 12 || 12;
  return m > 0 ? `${displayH}:${String(m).padStart(2, '0')}${ampm}` : `${displayH}${ampm}`;
}
const EMPTY_FORM = {
  title: '',
  patientId: '',
  patientName: '',
  date: '',
  startHour: '9',
  startMin: '00',
  endHour: '10',
  endMin: '00',
  reasonId: '',
  reasonName: '',
  notes: '',
};
function CalendarInner() {
  const searchParams = useSearchParams();
  const [view, setView] = useState<'week' | 'month'>('week');
  const [weekStart, setWeekStart] = useState<Date>(() => startOfWeek(new Date()));
  const [appointments, setAppointments] = useState<Appointment[]>([]);
  const [patients, setPatients] = useState<Patient[]>([]);
  const [reasons, setReasons] = useState<AppointmentReason[]>([]);
  const [loading, setLoading] = useState(false);
  // Modal state
  const [showModal, setShowModal] = useState(false);
  const [modalMode, setModalMode] = useState<'add' | 'view'>('add');
  const [selectedAppt, setSelectedAppt] = useState<Appointment | null>(null);
  const [form, setForm] = useState({ ...EMPTY_FORM });
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState('');
  const [editMode, setEditMode] = useState(false);
  const [titleManuallyEdited, setTitleManuallyEdited] = useState(false);
  const weekDays = Array.from({ length: 7 }, (_, i) => addDays(weekStart, i));
  const today = new Date();
  const loadAppointments = useCallback(async () => {
    setLoading(true);
    try {
      const res = await fetch(`/api/iat-appointments?date=${isoDate(weekStart)}`);
      if (res.ok) {
        const data = await res.json();
```

```
      setAppointments(Array.isArray(data) ? data : []);
    }
  } catch {}
  setLoading(false);
}, [weekStart]);
useEffect(() => {
  loadAppointments();
}, [loadAppointments]);
useEffect(() => {
  fetch('/api/patients')
    .then(r => r.ok ? r.json() : [])
    .then(d => setPatients(Array.isArray(d) ? d : d.patients ?? []))
    .catch(() => {});
}, []);
useEffect(() => {
  fetch('/api/appointment-reasons')
    .then(r => r.ok ? r.json() : { reasons: [] })
    .then(d => setReasons(Array.isArray(d) ? d : d.reasons ?? []))
    .catch(() => {});
}, []);
// Handle URL params: auto-open modal if ?action=new
useEffect(() => {
  const action = searchParams.get('action');
  const patientId = searchParams.get('patientId');
  const patientName = searchParams.get('patientName');
  if (action === 'new') {
    const d = isoDate(today);
    const baseForm = { ...EMPTY_FORM, date: d, startHour: '9', endHour: '10' };
    if (patientId) {
      baseForm.patientId = patientId;
      baseForm.patientName = patientName ?? '';
    }
    setForm(baseForm);
    setSelectedAppt(null);
    setModalMode('add');
    setEditMode(false);
    setTitleManuallyEdited(false);
    setError('');
    setShowModal(true);
  }
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [searchParams]);
// Auto-fill title when patient and reason both selected
useEffect(() => {
  if (!titleManuallyEdited && form.patientName && form.reasonName) {
    setForm(f => ({ ...f, title: `${form.patientName} - ${form.reasonName}` }));
  }
}, [form.patientName, form.reasonName, titleManuallyEdited]);
function openAddModal(date?: Date, hour?: number) {
  const d = date ? isoDate(date) : isoDate(today);
  const h = hour ?? 9;
  setForm({ ...EMPTY_FORM, date: d, startHour: String(h), endHour: String(h + 1) });
  setSelectedAppt(null);
  setModalMode('add');
  setEditMode(false);
  setTitleManuallyEdited(false);
  setError('');
  setShowModal(true);
}
function openViewModal(appt: Appointment) {
  setSelectedAppt(appt);
  setModalMode('view');
  setEditMode(false);
  setError('');
```

```
  setShowModal(true);
}
function openEditModal(appt: Appointment) {
  const start = new Date(appt.startTime);
  const end = new Date(appt.endTime);
  const reason = reasons.find(r => r.name === appt.type || r.id === appt.type);
  setForm({
    title: appt.title,
    patientId: appt.patientId ?? '',
    patientName: appt.patientName ?? '',
    date: isoDate(start),
    startHour: String(start.getHours()),
    startMin: String(start.getMinutes()).padStart(2, '0'),
    endHour: String(end.getHours()),
    endMin: String(end.getMinutes()).padStart(2, '0'),
    reasonId: reason?.id ?? appt.type,
    reasonName: reason?.name ?? appt.type,
    notes: appt.notes ?? '',
  });
  setSelectedAppt(appt);
  setModalMode('add');
  setEditMode(true);
  setTitleManuallyEdited(true);
  setError('');
  setShowModal(true);
}
async function handleSave() {
  if (!form.title.trim()) { setError('Title is required'); return; }
  if (!form.date) { setError('Date is required'); return; }
  if (!form.patientId) { setError('Patient selection is required'); return; }
  setSaving(true);
  setError('');
  const startDt = new Date(`${form.date}T${String(form.startHour).padStart(2, '0')}:${form.startMin}:00`);
  const endDt = new Date(`${form.date}T${String(form.endHour).padStart(2, '0')}:${form.endMin}:00`);
  const selectedReason = reasons.find(r => r.id === form.reasonId);
  const body = {
    title: form.title.trim(),
    patientId: form.patientId || undefined,
    patientName: form.patientName || undefined,
    startTime: startDt.toISOString(),
    endTime: endDt.toISOString(),
    type: selectedReason?.name ?? form.reasonName ?? form.reasonId,
    notes: form.notes || undefined,
  };
  try {
    let res;
    if (editMode && selectedAppt) {
      res = await fetch(`/api/iat-appointments/${selectedAppt.id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(body),
      });
    } else {
      res = await fetch('/api/iat-appointments', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(body),
      });
    }
    if (!res.ok) {
      const d = await res.json();
      setError(d.error ?? 'Save failed');
    } else {
      setShowModal(false);
```

```
        loadAppointments();
      }
    } catch {
      setError('Network error');
    }
    setSaving(false);
  }
  async function handleDelete(id: string) {
    if (!confirm('Delete this appointment?')) return;
    await fetch(`/api/iat-appointments/${id}`, { method: 'DELETE' });
    setShowModal(false);
    loadAppointments();
  }
  async function handleStatusChange(id: string, status: string) {
    await fetch(`/api/iat-appointments/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status }),
    });
    loadAppointments();
    setShowModal(false);
  }
  function getReasonForAppt(appt: Appointment) {
    return reasons.find(r => r.name === appt.type || r.id === appt.type);
  }
  function getApptColors(appt: Appointment) {
    const reason = getReasonForAppt(appt);
    if (reason) {
      return {
        bg: hexToRgba(reason.color, 0.12),
        border: reason.color,
        text: reason.color,
      };
    }
    return { bg: '#e8f9f7', border: '#0d9488', text: '#0d9488' };
  }
  // Week view rendering
  function getApptsForSlot(day: Date, hour: number) {
    return appointments.filter(a => {
      const d = apptDay(a);
      return sameDay(d, day) && apptHour(a) === hour;
    });
  }
  // Month view: compute month grid
  const monthStart = new Date(weekStart.getFullYear(), weekStart.getMonth(), 1);
  const monthFirstDay = monthStart.getDay();
  const daysInMonth = new Date(weekStart.getFullYear(), weekStart.getMonth() + 1, 0).getDate();
  const DAYS_SHORT = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'];
  const MONTHS = ['January','February','March','April','May','June','July','August','September','October','Novembe
r','December'];
  function prevWeek() { setWeekStart(w => addDays(w, -7)); }
  function nextWeek() { setWeekStart(w => addDays(w, 7)); }
  function prevMonth() { setWeekStart(w => new Date(w.getFullYear(), w.getMonth() - 1, 1)); }
  function nextMonth() { setWeekStart(w => new Date(w.getFullYear(), w.getMonth() + 1, 1)); }
  function goToday() { setWeekStart(startOfWeek(new Date())); }
  const monthLabel = `${MONTHS[weekStart.getMonth()]} ${weekStart.getFullYear()}`;
  const weekLabel = `${weekDays[0].toLocaleDateString('en-US', { month: 'short', day: 'numeric' })} –
${weekDays[6].toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' })}`;
  return (
    <>
      <div className="page-header">
        <div>
          <div className="page-title">■ Calendar</div>
```

```
          <div className="page-subtitle">Appointment scheduling — multiple patients per slot supported</div>
        </div>
        <button
          onClick={() => openAddModal()}
          className="btn btn-sm"
        >
          + Add Appointment
        </button>
      </div>
      <div className="page-body">
        {/* Toolbar */}
        <div style={{ display: 'flex', alignItems: 'center', gap: 12, marginBottom: 20, flexWrap: 'wrap' }}>
          {/* View toggle */}
          <div style={{ display: 'flex', border: '1.5px solid #e2e8f0', borderRadius: 8, overflow: 'hidden' }}>
            {(['week', 'month'] as const).map(v => (
              <button key={v} onClick={() => setView(v)}
                style={{ padding: '6px 18px', fontSize: 13, fontWeight: 600, border: 'none', cursor: 'pointer',
background: view === v ? '#0d9488' : '#fff', color: view === v ? '#fff' : '#374151' }}>
                {v.charAt(0).toUpperCase() + v.slice(1)}
              </button>
            ))}
          </div>
          {/* Navigation */}
          <button onClick={view === 'week' ? prevWeek : prevMonth}
            style={{ padding: '6px 14px', borderRadius: 8, border: '1.5px solid #e2e8f0', background: '#fff',
cursor: 'pointer', fontWeight: 700, fontSize: 16 }}>
            ‹
          </button>
          <div style={{ fontWeight: 700, fontSize: 15, minWidth: 200, textAlign: 'center', color: '#111827' }}>
            {view === 'week' ? weekLabel : monthLabel}
          </div>
          <button onClick={view === 'week' ? nextWeek : nextMonth}
            style={{ padding: '6px 14px', borderRadius: 8, border: '1.5px solid #e2e8f0', background: '#fff',
cursor: 'pointer', fontWeight: 700, fontSize: 16 }}>
            ›
          </button>
          <button onClick={goToday}
            style={{ padding: '6px 14px', borderRadius: 8, border: '1.5px solid #0d9488', background: '#e8f9f7',
color: '#0d9488', cursor: 'pointer', fontWeight: 600, fontSize: 13 }}>
            Today
          </button>
          {loading && <span style={{ fontSize: 12, color: '#9ca3af' }}>Loading…</span>}
        </div>
        {/* WEEK VIEW */}
        {view === 'week' && (
          <div className="card" style={{ padding: 0, overflow: 'hidden' }}>
            <div style={{ display: 'grid', gridTemplateColumns: '60px repeat(7, 1fr)', borderBottom: '2px solid
#e2e8f0' }}>
              <div style={{ padding: '10px 0', background: '#f8fafc' }} />
              {weekDays.map((day, i) => {
                const isToday = sameDay(day, today);
                return (
                  <div key={i} style={{ padding: '10px 6px', textAlign: 'center', background: isToday ? '#e8f9f7' :
 '#f8fafc', borderLeft: '1px solid #e2e8f0' }}>
                    <div style={{ fontSize: 11, fontWeight: 600, color: '#6b7280', textTransform: 'uppercase'
}}>{DAYS_SHORT[i]}</div>
                    <div style={{ fontSize: 20, fontWeight: 700, color: isToday ? '#0d9488' : '#111827',
marginTop: 2 }}>{day.getDate()}</div>
                  </div>
                );
              })}
            </div>
            {/* Time slots */}
            <div style={{ maxHeight: '65vh', overflowY: 'auto' }}>
```

```
              {HOURS.map(hour => (
                <div key={hour} style={{ display: 'grid', gridTemplateColumns: '60px repeat(7, 1fr)',
borderBottom: '1px solid #f1f5f9', minHeight: 70 }}>
                  <div style={{ padding: '6px 8px', fontSize: 11, color: '#9ca3af', fontWeight: 600, textAlign: '
right', paddingTop: 8 }}>
                    {formatHour(hour)}
                  </div>
                  {weekDays.map((day, di) => {
                    const isToday = sameDay(day, today);
                    const slotAppts = getApptsForSlot(day, hour);
                    return (
                      <div
                        key={di}
                        onClick={() => openAddModal(day, hour)}
                        style={{ borderLeft: '1px solid #f1f5f9', padding: '4px', cursor: 'pointer', background:
isToday ? '#fafffe' : 'white', transition: 'background 0.1s' }}
                        onMouseEnter={e => (e.currentTarget.style.background = '#f0fdf4')}
                        onMouseLeave={e => (e.currentTarget.style.background = isToday ? '#fafffe' : 'white')}
                      >
                        {slotAppts.map(appt => {
                          const colors = getApptColors(appt);
                          const reason = getReasonForAppt(appt);
                          const icon = reason ? getReasonIcon(reason.name) : '■';
                          return (
                            <div
                              key={appt.id}
                              onClick={e => { e.stopPropagation(); openViewModal(appt); }}
                              style={{ background: colors.bg, border: `1.5px solid ${colors.border}`,
borderRadius: 6, padding: '3px 6px', marginBottom: 3, cursor: 'pointer', fontSize: 11 }}
                            >
                              <div style={{ fontWeight: 700, color: colors.text, overflow: 'hidden',
textOverflow: 'ellipsis', whiteSpace: 'nowrap' }}>{icon} {appt.title}</div>
                              {appt.patientName && <div style={{ color: '#374151', fontSize: 10, overflow: '
hidden', textOverflow: 'ellipsis', whiteSpace: 'nowrap' }}>{appt.patientName}</div>}
                              <div style={{ color: '#6b7280', fontSize: 10
}}>{formatTime(appt.startTime)}–{formatTime(appt.endTime)}</div>
                            </div>
                          );
                        })}
                      </div>
                    );
                  })}
                </div>
              ))}
            </div>
          </div>
        )}
        {/* MONTH VIEW */}
        {view === 'month' && (
          <div className="card" style={{ padding: 0, overflow: 'hidden' }}>
            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(7, 1fr)', borderBottom: '2px solid
#e2e8f0' }}>
              {DAYS_SHORT.map(d => (
                <div key={d} style={{ padding: '10px 0', textAlign: 'center', background: '#f8fafc', fontSize:
12, fontWeight: 700, color: '#6b7280', textTransform: 'uppercase' }}>{d}</div>
              ))}
            </div>
            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(7, 1fr)' }}>
              {/* Leading empty cells: monthFirstDay offset (Monday-first) */}
              {Array.from({ length: (monthFirstDay === 0 ? 6 : monthFirstDay - 1) }).map((_, i) => (
                <div key={`empty-${i}`} style={{ minHeight: 100, background: '#fafafa', borderRight: '1px solid
#f1f5f9', borderBottom: '1px solid #f1f5f9' }} />
              ))}
              {Array.from({ length: daysInMonth }, (_, i) => {
                const day = new Date(weekStart.getFullYear(), weekStart.getMonth(), i + 1);
```

```
                const isToday = sameDay(day, today);
                const dayAppts = appointments.filter(a => sameDay(apptDay(a), day));
                return (
                  <div
                    key={i}
                    onClick={() => openAddModal(day)}
                    style={{ minHeight: 100, borderRight: '1px solid #f1f5f9', borderBottom: '1px solid #f1f5f9',
padding: 6, cursor: 'pointer', background: isToday ? '#fafffe' : 'white' }}
                    onMouseEnter={e => (e.currentTarget.style.background = '#f0fdf4')}
                    onMouseLeave={e => (e.currentTarget.style.background = isToday ? '#fafffe' : 'white')}
                  >
                    <div style={{ fontWeight: 700, fontSize: 13, color: isToday ? '#0d9488' : '#374151',
marginBottom: 4,
                      ...(isToday ? { background: '#0d9488', color: '#fff', width: 24, height: 24, borderRadius: '
50%', display: 'flex', alignItems: 'center', justifyContent: 'center', fontSize: 12 } : {}) }}>
                      {i + 1}
                    </div>
                    {dayAppts.slice(0, 3).map(appt => {
                      const colors = getApptColors(appt);
                      const reason = getReasonForAppt(appt);
                      const icon = reason ? getReasonIcon(reason.name) : '■';
                      return (
                        <div key={appt.id} onClick={e => { e.stopPropagation(); openViewModal(appt); }}
                          style={{ background: colors.bg, border: `1px solid ${colors.border}`, borderRadius: 4,
padding: '2px 5px', marginBottom: 2, fontSize: 10, overflow: 'hidden', textOverflow: 'ellipsis', whiteSpace: '
nowrap', fontWeight: 600, color: colors.text, cursor: 'pointer' }}>
                          {icon} {formatTime(appt.startTime)} {appt.title}
                        </div>
                      );
                    })}
                    {dayAppts.length > 3 && (
                      <div style={{ fontSize: 10, color: '#9ca3af', fontWeight: 600 }}>+{dayAppts.length - 3}
more</div>
                    )}
                  </div>
                );
              })}
            </div>
          </div>
        )}
        {/* Legend */}
        {reasons.length > 0 && (
          <div style={{ display: 'flex', gap: 16, marginTop: 16, flexWrap: 'wrap' }}>
            {reasons.map(r => (
              <div key={r.id} style={{ display: 'flex', alignItems: 'center', gap: 6, fontSize: 12, color: '
#374151' }}>
                <div style={{ width: 14, height: 14, borderRadius: 3, background: hexToRgba(r.color, 0.15),
border: `1.5px solid ${r.color}` }} />
                {getReasonIcon(r.name)} {r.name}
              </div>
            ))}
          </div>
        )}
      </div>
      {/* ■■ MODAL ■■ */}
      {showModal && (
        <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.4)', zIndex: 1000, display: 'flex',
alignItems: 'center', justifyContent: 'center', padding: 16 }}>
          <div style={{ background: '#fff', borderRadius: 16, width: '100%', maxWidth: 520, maxHeight: '90vh',
overflowY: 'auto', boxShadow: '0 20px 60px rgba(0,0,0,0.2)' }}>
            {/* Header */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '20px
24px', borderBottom: '1px solid #f1f5f9' }}>
              <div style={{ fontWeight: 700, fontSize: 18, color: '#111827' }}>
```

```
                {modalMode === 'view' && !editMode ? '■ Appointment Details' : editMode ? '✏■ Edit Appointment' :
 '+ Add Appointment'}
              </div>
              <button onClick={() => setShowModal(false)}
                style={{ background: 'none', border: 'none', fontSize: 22, cursor: 'pointer', color: '#6b7280',
lineHeight: 1 }}>×</button>
            </div>
            <div style={{ padding: '20px 24px' }}>
              {/* VIEW mode */}
              {modalMode === 'view' && !editMode && selectedAppt && (
                <div>
                  {(() => {
                    const colors = getApptColors(selectedAppt);
                    const reason = getReasonForAppt(selectedAppt);
                    const icon = reason ? getReasonIcon(reason.name) : '■';
                    return (
                      <>
                        <div style={{ marginBottom: 16 }}>
                          <span style={{ background: colors.bg, color: colors.text, border: `1px solid
${colors.border}`, borderRadius: 999, padding: '2px 12px', fontSize: 12, fontWeight: 700 }}>
                            {icon} {reason?.name ?? selectedAppt.type}
                          </span>
                          <span style={{ marginLeft: 10, fontSize: 12, padding: '2px 10px', borderRadius: 999,
background: '#f1f5f9', color: '#374151', fontWeight: 600 }}>
                            {selectedAppt.status}
                          </span>
                        </div>
                        <h3 style={{ margin: '0 0 12px', fontSize: 20, color: '#111827'
}}>{selectedAppt.title}</h3>
                        {selectedAppt.patientName && (
                          <div style={{ marginBottom: 8, fontSize: 14, color: '#374151' }}>■
{selectedAppt.patientName}</div>
                        )}
                        <div style={{ marginBottom: 8, fontSize: 14, color: '#374151' }}>
                          ■ {formatTime(selectedAppt.startTime)} – {formatTime(selectedAppt.endTime)}
                        </div>
                        <div style={{ marginBottom: 12, fontSize: 14, color: '#374151' }}>
                          ■ {new Date(selectedAppt.startTime).toLocaleDateString('en-US', { weekday: 'long',
month: 'long', day: 'numeric', year: 'numeric' })}
                        </div>
                        {selectedAppt.notes && (
                          <div style={{ background: '#f8fafc', borderRadius: 8, padding: '10px 14px', fontSize:
14, color: '#374151', marginBottom: 12 }}>
                            ■ {selectedAppt.notes}
                          </div>
                        )}
                        {/* Status actions */}
                        {selectedAppt.status === 'scheduled' && (
                          <button onClick={() => handleStatusChange(selectedAppt.id, 'in-progress')}
                            style={{ padding: '8px 16px', borderRadius: 8, border: 'none', background: '#0d9488',
color: '#fff', fontWeight: 700, cursor: 'pointer', fontSize: 13, marginBottom: 8, marginRight: 8 }}>
                            ■ Start
                          </button>
                        )}
                        {selectedAppt.status === 'in-progress' && (
                          <button onClick={() => handleStatusChange(selectedAppt.id, 'complete')}
                            style={{ padding: '8px 16px', borderRadius: 8, border: 'none', background: '#059669',
color: '#fff', fontWeight: 700, cursor: 'pointer', fontSize: 13, marginBottom: 8, marginRight: 8 }}>
                            ■ Complete
                          </button>
                        )}
                      </>
                    );
                  })()}
                  <div style={{ display: 'flex', gap: 10, marginTop: 8 }}>
```

```
                    <button onClick={() => openEditModal(selectedAppt)}
                      style={{ flex: 1, padding: '10px 0', borderRadius: 10, border: '1.5px solid #0d9488',
background: '#fff', color: '#0d9488', fontWeight: 700, cursor: 'pointer', fontSize: 14 }}>
                      ✏ Edit
                    </button>
                    <button onClick={() => handleDelete(selectedAppt.id)}
                      style={{ flex: 1, padding: '10px 0', borderRadius: 10, border: '1.5px solid #dc2626',
background: '#fff', color: '#dc2626', fontWeight: 700, cursor: 'pointer', fontSize: 14 }}>
                      🗑 Delete
                    </button>
                  </div>
                </div>
              )}
              {/* ADD/EDIT form */}
              {(modalMode === 'add' || editMode) && (
                <div>
                  {error && (
                    <div style={{ background: '#fef2f2', border: '1px solid #fca5a5', borderRadius: 8, padding: '
8px 14px', fontSize: 13, color: '#dc2626', marginBottom: 14 }}>
                      {error}
                    </div>
                  )}
                  {/* Patient — required */}
                  <div style={{ marginBottom: 14 }}>
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>
                      Patient <span style={{ color: '#dc2626' }}>*</span>
                    </label>
                    <select value={form.patientId}
                      onChange={e => {
                        const p = patients.find(x => x.id === e.target.value);
                        setForm(f => ({ ...f, patientId: e.target.value, patientName: p?.name ?? '' }));
                      }}
                      style={{ width: '100%', padding: '8px 12px', borderRadius: 8, border: `1.5px solid
${!form.patientId && error ? '#fca5a5' : '#e2e8f0'}`, fontSize: 14, boxSizing: 'border-box' }}>
                      <option value="">— Select patient —</option>
                      {patients.map(p => <option key={p.id} value={p.id}>{p.name} ({p.patientId})</option>)}
                    </select>
                  </div>
                  {/* Reason */}
                  <div style={{ marginBottom: 14 }}>
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>Reason</label>
                    <select value={form.reasonId}
                      onChange={e => {
                        const r = reasons.find(x => x.id === e.target.value);
                        setForm(f => ({ ...f, reasonId: e.target.value, reasonName: r?.name ?? '' }));
                      }}
                      style={{ width: '100%', padding: '8px 12px', borderRadius: 8, border: '1.5px solid
#e2e8f0', fontSize: 14, boxSizing: 'border-box' }}>
                      <option value="">— Select reason —</option>
                      {reasons.map(r => (
                        <option key={r.id} value={r.id}>
                          {getReasonIcon(r.name)} {r.name} ({r.duration}min)
                        </option>
                      ))}
                    </select>
                  </div>
                  {/* Title — auto-filled but editable */}
                  <div style={{ marginBottom: 14 }}>
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>
                      Title <span style={{ color: '#dc2626' }}>*</span>
                      {!titleManuallyEdited && form.patientName && form.reasonName && (
```

```
                        <span style={{ fontWeight: 400, color: '#9ca3af', marginLeft: 6, fontSize: 12
}}>(auto-filled)</span>
                      )}
                    </label>
                    <input value={form.title}
                      onChange={e => { setTitleManuallyEdited(true); setForm(f => ({ ...f, title: e.target.value
})); }}
                      placeholder="e.g. John Smith - Allergy Testing"
                      style={{ width: '100%', padding: '8px 12px', borderRadius: 8, border: '1.5px solid
#e2e8f0', fontSize: 14, boxSizing: 'border-box' }} />
                    {titleManuallyEdited && (
                      <button type="button" onClick={() => { setTitleManuallyEdited(false); }}
                        style={{ fontSize: 11, color: '#0d9488', background: 'none', border: 'none', cursor: '
pointer', padding: '2px 0', marginTop: 2 }}>
                        ■ Reset to auto-fill
                      </button>
                    )}
                  </div>
                  {/* Date */}
                  <div style={{ marginBottom: 14 }}>
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>Date <span style={{ color: '#dc2626' }}>*</span></label>
                    <input type="date" value={form.date} onChange={e => setForm(f => ({ ...f, date:
e.target.value }))}
                      style={{ width: '100%', padding: '8px 12px', borderRadius: 8, border: '1.5px solid
#e2e8f0', fontSize: 14, boxSizing: 'border-box' }} />
                  </div>
                  {/* Time */}
                  <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 12, marginBottom: 14 }}>
                    <div>
                      <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>Start Time <span style={{ color: '#dc2626' }}>*</span></label>
                      <div style={{ display: 'flex', gap: 4 }}>
                        <select value={form.startHour} onChange={e => setForm(f => ({ ...f, startHour:
e.target.value }))}
                          style={{ flex: 1, padding: '8px 6px', borderRadius: 8, border: '1.5px solid #e2e8f0',
fontSize: 13 }}>
                          {HOURS.map(h => <option key={h} value={h}>{formatHour(h)}</option>)}
                        </select>
                        <select value={form.startMin} onChange={e => setForm(f => ({ ...f, startMin:
e.target.value }))}
                          style={{ width: 56, padding: '8px 4px', borderRadius: 8, border: '1.5px solid #e2e8f0',
fontSize: 13 }}>
                          {['00','15','30','45'].map(m => <option key={m} value={m}>:{m}</option>)}
                        </select>
                      </div>
                    </div>
                    <div>
                      <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>End Time <span style={{ color: '#dc2626' }}>*</span></label>
                      <div style={{ display: 'flex', gap: 4 }}>
                        <select value={form.endHour} onChange={e => setForm(f => ({ ...f, endHour: e.target.value
}))}
                          style={{ flex: 1, padding: '8px 6px', borderRadius: 8, border: '1.5px solid #e2e8f0',
fontSize: 13 }}>
                          {HOURS.map(h => <option key={h} value={h}>{formatHour(h)}</option>)}
                        </select>
                        <select value={form.endMin} onChange={e => setForm(f => ({ ...f, endMin: e.target.value
}))}
                          style={{ width: 56, padding: '8px 4px', borderRadius: 8, border: '1.5px solid #e2e8f0',
fontSize: 13 }}>
                          {['00','15','30','45'].map(m => <option key={m} value={m}>:{m}</option>)}
                        </select>
                      </div>
```

```
                    </div>
                  </div>
                  {/* Notes */}
                  <div style={{ marginBottom: 20 }}>
                    <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 4 }}>Notes</label>
                    <textarea value={form.notes} onChange={e => setForm(f => ({ ...f, notes: e.target.value }))}
                      rows={3} placeholder="Optional notes…"
                      style={{ width: '100%', padding: '8px 12px', borderRadius: 8, border: '1.5px solid
#e2e8f0', fontSize: 14, resize: 'vertical', boxSizing: 'border-box' }} />
                  </div>
                  {/* Actions */}
                  <div style={{ display: 'flex', gap: 10 }}>
                    <button onClick={() => setShowModal(false)}
                      style={{ flex: 1, padding: '10px 0', borderRadius: 10, border: '1.5px solid #e2e8f0',
background: '#fff', color: '#374151', fontWeight: 700, cursor: 'pointer', fontSize: 14 }}>
                      Cancel
                    </button>
                    <button onClick={handleSave} disabled={saving}
                      style={{ flex: 2, padding: '10px 0', borderRadius: 10, border: 'none', background: '
#0d9488', color: '#fff', fontWeight: 700, cursor: 'pointer', fontSize: 14, opacity: saving ? 0.7 : 1 }}>
                      {saving ? 'Saving…' : editMode ? 'Save Changes' : 'Book Appointment'}
                    </button>
                  </div>
                </div>
              )}
            </div>
          </div>
        </div>
      )}
    </>
  );
}
export default function CalendarPage() {
  return (
    <Suspense fallback={<div className="page-body">Loading calendar…</div>}>
      <CalendarInner />
    </Suspense>
  );
}
```

page.tsx

app/encounters/page.tsx

```
'use client';
import { useEffect, useState } from 'react';
import Link from 'next/link';
import { useRouter } from 'next/navigation';
interface Encounter {
  id: string; patientId: string; encounterDate: string; doctorName?: string;
  nurseName?: string; chiefComplaint: string; status: string; assessment?: string;
}
const STATUS_STYLE: Record<string, { bg: string; color: string }> = {
  open: { bg: '#fef9c3', color: '#b45309' },
  complete: { bg: '#dcfce7', color: '#15803d' },
  cancelled: { bg: '#f3f4f6', color: '#64748b' },
};
export default function EncountersPage() {
  const [encounters, setEncounters] = useState<Encounter[]>([]);
  const [loading, setLoading] = useState(true);
  const router = useRouter();
  useEffect(() => {
    fetch('/api/encounters?limit=100')
```

```
      .then(r => r.json())
      .then(d => { setEncounters(d.encounters ?? []); setLoading(false); })
      .catch(() => setLoading(false));
  }, []);
  function fmt(d: string) {
    try { return new Date(d).toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' }); }
    catch { return d; }
  }
  return (
    <>
      <div className="page-header">
        <div>
          <div className="page-title">■ Encounter Records</div>
          <div className="page-subtitle">{loading ? 'Loading…' : `${encounters.length} encounters`}</div>
        </div>
      </div>
      <div className="page-body">
        {loading ? <div className="loading-center"><div className="spinner" /></div> :
        encounters.length === 0 ? (
          <div className="card" style={{ textAlign: 'center', padding: 48 }}>
            <div style={{ fontSize: 40, marginBottom: 12 }}>■</div>
            <div style={{ fontSize: 18, fontWeight: 700, color: '#374151', marginBottom: 8 }}>No encounters
yet</div>
            <div style={{ color: '#64748b', marginBottom: 20 }}>Create encounters from the patient detail
page</div>
            <Link href="/patients" className="btn">Go to Patients</Link>
          </div>
        ) : (
          <div className="card">
            <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 14 }}>
              <thead>
                <tr style={{ background: '#f8fafc' }}>
                  {['Date', 'Patient ID', 'Physician', 'Chief Complaint', 'Status', 'Actions'].map(h => (
                    <th key={h} style={{ padding: '10px 14px', textAlign: 'left', fontSize: 12, fontWeight: 700,
color: '#374151', textTransform: 'uppercase', borderBottom: '2px solid #e2e8f0' }}>{h}</th>
                  ))}
                </tr>
              </thead>
              <tbody>
                {encounters.map(e => {
                  const s = STATUS_STYLE[e.status] ?? STATUS_STYLE.open;
                  return (
                    <tr key={e.id} style={{ borderBottom: '1px solid #f1f5f9', cursor: 'pointer' }}
                      onClick={() => router.push(`/patients/${e.patientId}#encounters`)}
                      onMouseEnter={ev => (ev.currentTarget.style.background = '#f8fafc')}
                      onMouseLeave={ev => (ev.currentTarget.style.background = '')}>
                      <td style={{ padding: '10px 14px', color: '#64748b' }}>{fmt(e.encounterDate)}</td>
                      <td style={{ padding: '10px 14px', fontFamily: 'monospace', fontSize: 12, color: '#64748b'
}}>{e.patientId.slice(0,12)}…</td>
                      <td style={{ padding: '10px 14px' }}>{e.doctorName ?? '—'}</td>
                      <td style={{ padding: '10px 14px', fontWeight: 600 }}>{e.chiefComplaint}</td>
                      <td style={{ padding: '10px 14px' }}>
                        <span style={{ fontSize: 12, fontWeight: 700, padding: '2px 10px', borderRadius: 999, .
..s }}>{e.status}</span>
                      </td>
                      <td style={{ padding: '10px 14px' }}>
                        <Link href={`/patients/${e.patientId}`} onClick={ev => ev.stopPropagation()}
                          style={{ fontSize: 12, color: '#0d9488', fontWeight: 600, textDecoration: 'none'
}}>View Patient</Link>
                      </td>
                    </tr>
                  );
                })}
              </tbody>
```

```
            </table>
          </div>
        )}
      </div>
    </>
  );
}
```

layout.tsx

app/kiosk/layout.tsx

```
/**
 * @file app/kiosk/layout.tsx
 * @description Shared layout for all kiosk routes. Fullscreen kiosk mode with
 * branded header and HIPAA footer. Tap logo 5× to exit fullscreen (staff only).
 */
'use client';
import { useState, useEffect, useRef, useCallback } from 'react';
export default function KioskLayout({ children }: { children: React.ReactNode }) {
  const [isFullscreen, setIsFullscreen] = useState(false);
  const [hasBeenFullscreen, setHasBeenFullscreen] = useState(false);
  const [logoTaps, setLogoTaps] = useState(0);
  const tapTimer = useRef<ReturnType<typeof setTimeout> | null>(null);
  // Clear tap reset timer on unmount to prevent state-after-unmount
  useEffect(() => {
    return () => {
      if (tapTimer.current) clearTimeout(tapTimer.current);
    };
  }, []);
  // Track fullscreen state changes
  useEffect(() => {
    function onFsChange() {
      const fs = !!document.fullscreenElement;
      setIsFullscreen(fs);
      if (fs) setHasBeenFullscreen(true); // track that we entered fullscreen at least once
    }
    document.addEventListener('fullscreenchange', onFsChange);
    return () => document.removeEventListener('fullscreenchange', onFsChange);
  }, []);
  // Block keyboard shortcuts that could let patients escape kiosk
  useEffect(() => {
    function blockKeys(e: KeyboardEvent) {
      // Block F11 (fullscreen toggle), F12 (DevTools), F5 (refresh)
      if (e.key === 'F11' || e.key === 'F12' || e.key === 'F5') {
        e.preventDefault(); e.stopPropagation(); return false;
      }
      // Block Ctrl+W (close tab), Ctrl+T (new tab), Ctrl+N (new window)
      // Block Ctrl+R (refresh), Ctrl+L (address bar), Ctrl+U (view source)
      if (e.ctrlKey && ['w','t','n','r','l','u','j'].includes(e.key.toLowerCase())) {
        e.preventDefault(); e.stopPropagation(); return false;
      }
      // Block Alt+F4 (close window), Alt+Tab (switch app)
      if (e.altKey && (e.key === 'F4' || e.key === 'Tab')) {
        e.preventDefault(); e.stopPropagation(); return false;
      }
    }
    // Block right-click context menu
    function blockContext(e: MouseEvent) {
      e.preventDefault(); return false;
    }
    // Warn before closing window
    function blockClose(e: BeforeUnloadEvent) {
      e.preventDefault();
```

```
    e.returnValue = '';
    return '';
  }
  document.addEventListener('keydown', blockKeys, true);
  document.addEventListener('contextmenu', blockContext, true);
  window.addEventListener('beforeunload', blockClose);
  return () => {
    document.removeEventListener('keydown', blockKeys, true);
    document.removeEventListener('contextmenu', blockContext, true);
    window.removeEventListener('beforeunload', blockClose);
  };
}, []);
// Enter fullscreen on first interaction
const enterFullscreen = useCallback(async () => {
  try {
    await document.documentElement.requestFullscreen();
  } catch { /* browser may deny */ }
}, []);
// Tap logo 5× quickly to exit fullscreen (staff escape hatch)
function handleLogoTap() {
  setLogoTaps(prev => {
    const next = prev + 1;
    if (tapTimer.current) clearTimeout(tapTimer.current);
    if (next >= 5) {
      document.exitFullscreen?.().catch(() => {});
      return 0;
    }
    tapTimer.current = setTimeout(() => setLogoTaps(0), 3000);
    return next;
  });
}
return (
  <div style={{ minHeight: '100vh', background: '#F8FAFC', display: 'flex', flexDirection: 'column' }}>
    {/* Fullscreen guard overlay — only shown AFTER patient was already fullscreen */}
    {!isFullscreen && hasBeenFullscreen && (
      <div style={{
        position: 'fixed', inset: 0, zIndex: 9999,
        background: 'rgba(0,0,0,0.92)',
        display: 'flex', flexDirection: 'column',
        alignItems: 'center', justifyContent: 'center',
        gap: 20,
      }}>
        <div style={{ fontSize: 64 }}>■</div>
        <div style={{ fontSize: 28, fontWeight: 800, color: '#fff', textAlign: 'center' }}>
          Please ask a staff member for assistance
        </div>
        <div style={{ fontSize: 16, color: 'rgba(255,255,255,0.6)', textAlign: 'center', maxWidth: 400 }}>
          This kiosk is in protected mode
        </div>
        <button
          onClick={enterFullscreen}
          style={{
            marginTop: 12, padding: '16px 40px',
            background: '#0d9488', color: '#fff',
            border: 'none', borderRadius: 12,
            fontSize: 18, fontWeight: 700, cursor: 'pointer',
          }}
        >
          ■ Return to Kiosk
        </button>
      </div>
    )}
    {/* Header */}
```

```
      <div style={{ background: '#0055A5', padding: '12px 24px', display: 'flex', alignItems: 'center',
justifyContent: 'space-between' }}>
        <div style={{ display: 'flex', alignItems: 'center', gap: 12 }}>
          {/* eslint-disable-next-line @next/next/no-img-element */}
          <img
            src="/integrated-allergy-logo.jpg"
            alt="Integrated Allergy Testing"
            style={{ height: 40, width: 'auto', cursor: 'pointer', userSelect: 'none' }}
            onClick={handleLogoTap}
            title="Tap 5× to exit fullscreen"
          />
          <span style={{ color: 'rgba(255,255,255,0.8)', fontSize: 14 }}>Patient Check-In</span>
        </div>
        {/* Fullscreen button — shown when not fullscreen */}
        {!isFullscreen && (
          <button
            onClick={enterFullscreen}
            style={{
              background: 'rgba(255,255,255,0.15)',
              border: '1px solid rgba(255,255,255,0.3)',
              color: '#fff',
              borderRadius: 8,
              padding: '6px 14px',
              fontSize: 13,
              fontWeight: 600,
              cursor: 'pointer',
              display: 'flex',
              alignItems: 'center',
              gap: 6,
            }}
          >
            ⛶ Fullscreen
          </button>
        )}
        {logoTaps > 0 && logoTaps < 5 && (
          <span style={{ color: 'rgba(255,255,255,0.6)', fontSize: 11 }}>
            {5 - logoTaps} more tap{5 - logoTaps !== 1 ? 's' : ''} to exit
          </span>
        )}
      </div>
      {/* Content */}
      <div style={{ flex: 1, display: 'flex', alignItems: 'center', justifyContent: 'center', padding: 24 }}>
        {children}
      </div>
      {/* Footer */}
      <div style={{ padding: '12px 24px', textAlign: 'center', fontSize: 12, color: '#94a3b8' }}>
        ⛶ HIPAA Compliant · Integrated Allergy Testing
      </div>
    </div>
  );
}
```

page.tsx

app/kiosk/page.tsx

```
/**
 * @file app/kiosk/page.tsx
 * @description Kiosk home page — patient check-in entry point.
 * Prompts for date of birth, calls /api/kiosk/lookup, then routes to
 * verify (existing patient) or register (new patient).
 */
'use client';
import { useState } from 'react';
import { useRouter } from 'next/navigation';
```

```
export default function KioskHomePage() {
  const router = useRouter();
  const [dob, setDob] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  async function handleContinue() {
    if (!dob) return;
    setLoading(true);
    setError('');
    try {
      const res = await fetch('/api/kiosk/lookup', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ dob }),
      });
      const data = await res.json();
      sessionStorage.setItem('kiosk_dob', dob);
      sessionStorage.setItem('kiosk_lookup', JSON.stringify(data));
      if (data.found) {
        router.push(`/kiosk/verify?dob=${encodeURIComponent(dob)}`);
      } else {
        router.push(`/kiosk/register?dob=${encodeURIComponent(dob)}`);
      }
    } catch {
      setError('Something went wrong. Please try again or ask a staff member for help.');
      setLoading(false);
    }
  }
  return (
    <div style={{
      background: '#fff',
      borderRadius: 20,
      boxShadow: '0 8px 40px rgba(0,0,0,0.10)',
      padding: '48px 56px',
      maxWidth: 560,
      width: '100%',
      textAlign: 'center',
    }}>
      <div style={{ fontSize: 56, marginBottom: 16 }}>■</div>
      <h1 style={{
        fontSize: 36,
        fontWeight: 800,
        color: '#0055A5',
        marginBottom: 12,
        lineHeight: 1.2,
      }}>
        Welcome — Patient Check-In
      </h1>
      <p style={{ fontSize: 20, color: '#475569', marginBottom: 40 }}>
        Please enter your date of birth to get started.
      </p>
      <div style={{ marginBottom: 32 }}>
        <label style={{ display: 'block', fontSize: 16, fontWeight: 600, color: '#334155', marginBottom: 10,
textAlign: 'left' }}>
          Date of Birth
        </label>
        <input
          type="date"
          value={dob}
          onChange={e => setDob(e.target.value)}
          max={new Date().toISOString().split('T')[0]}
          style={{
            width: '100%',
            fontSize: 24,
```

```
            padding: '16px 20px',
            border: '2px solid #cbd5e1',
            borderRadius: 12,
            outline: 'none',
            color: '#1e293b',
            background: '#f8fafc',
            cursor: 'pointer',
            boxSizing: 'border-box',
            transition: 'border-color 0.2s',
          }}
          onFocus={e => (e.target.style.borderColor = '#0d9488')}
          onBlur={e => (e.target.style.borderColor = '#cbd5e1')}
        />
      </div>
      {error && (
        <div style={{
          background: '#fef2f2',
          border: '1px solid #fca5a5',
          borderRadius: 10,
          padding: '14px 18px',
          color: '#dc2626',
          fontSize: 16,
          marginBottom: 24,
        }}>
          {error}
        </div>
      )}
      <button
        onClick={handleContinue}
        disabled={!dob || loading}
        style={{
          width: '100%',
          padding: '22px 0',
          fontSize: 22,
          fontWeight: 700,
          background: !dob || loading ? '#94a3b8' : '#0d9488',
          color: '#fff',
          border: 'none',
          borderRadius: 14,
          cursor: !dob || loading ? 'not-allowed' : 'pointer',
          transition: 'background 0.2s, transform 0.1s',
          minHeight: 70,
          letterSpacing: 0.3,
        }}
        onMouseDown={e => { if (dob && !loading) (e.currentTarget.style.transform = 'scale(0.98)'); }}
        onMouseUp={e => { (e.currentTarget.style.transform = 'scale(1)'); }}
      >
        {loading ? 'Looking up…' : 'Continue →'}
      </button>
    </div>
  );
}
```

page.tsx

app/kiosk/verify/page.tsx

```
/**
 * @file app/kiosk/verify/page.tsx
 * @description Kiosk patient identity verification page.
 * Prompts the patient to confirm their first name, then calls /api/kiosk/verify.
 * On success, checks videos/consent completion and routes to the appropriate next step.
 */
'use client';
import { useState, useEffect, Suspense } from 'react';
```

```
import { useRouter, useSearchParams } from 'next/navigation';
interface Patient {
  id: string;
  firstName: string;
}
interface LookupResult {
  found: boolean;
  patients?: Patient[];
}
function VerifyContent() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const [patients, setPatients] = useState<Patient[]>([]);
  const [selectedPatient, setSelectedPatient] = useState<Patient | null>(null);
  const [firstName, setFirstName] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  useEffect(() => {
    const raw = sessionStorage.getItem('kiosk_lookup');
    if (raw) {
      const data: LookupResult = JSON.parse(raw);
      if (data.found && data.patients && data.patients.length > 0) {
        setPatients(data.patients);
        if (data.patients.length === 1) {
          setSelectedPatient(data.patients[0]);
        }
      }
    }
  }, []);
  async function handleConfirm() {
    if (!selectedPatient || !firstName.trim()) return;
    setLoading(true);
    setError('');
    try {
      const res = await fetch('/api/kiosk/verify', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ patientId: selectedPatient.id, firstName: firstName.trim() }),
      });
      const data = await res.json();
      if (data.verified) {
        // Use full patient data from API response (has full name, not just firstName)
        const patientData = data.patient || { ...selectedPatient, name: `${firstName.trim()}
${selectedPatient.id}` };
        sessionStorage.setItem('kiosk_patient', JSON.stringify(patientData));
        // Check videos watched + consent signed — skip steps already completed
        const patId = patientData.id || selectedPatient.id;
        try {
          const [videosRes, watchedRes, consentRes] = await Promise.all([
            fetch('/api/videos').then(r => r.json()),
            fetch(`/api/kiosk/videos-watched?patientId=${patId}`).then(r => r.json()),
            fetch(`/api/consent/check?patientId=${patId}`).then(r => r.json()),
          ]);
          const totalVideos = (Array.isArray(videosRes) ? videosRes : videosRes.videos ?? []).filter((v: {
active?: boolean }) => v.active !== false).length;
          const watchedCount = (watchedRes.watchedIds ?? []).length;
          const allVideosDone = totalVideos === 0 || watchedCount >= totalVideos;
          const allConsentDone = consentRes.allSigned === true;
          let nextStep: string;
          if (allVideosDone && allConsentDone) {
            // Everything done — go straight to done/waiting room
            nextStep = 'done';
          } else if (allVideosDone) {
```

```
            // Videos done, consent needed
            nextStep = 'consent';
          } else {
            // Need to watch videos first
            nextStep = 'videos';
          }
          // Store next step so update-info knows where to continue
          sessionStorage.setItem('kiosk_next_step', nextStep);
          // Fetch full patient record to check for missing info
          // (verify API only returns id/firstName for privacy — need separate fetch)
          try {
            const fullPatient = await fetch(`/api/patients/${patientData.id}`).then(r => r.ok ? r.json() : null);
            const missingInfo = fullPatient && (!fullPatient.phone || !fullPatient.email || !
fullPatient.insuranceId);
            if (missingInfo) {
              router.push('/kiosk/update-info');
            } else {
              router.push(`/kiosk/${nextStep}`);
            }
          } catch {
            router.push(`/kiosk/${nextStep}`);
          }
        } catch {
          router.push('/kiosk/videos');
        }
      } else {
        // Name doesn't match — automatically take them to new patient registration
        const dob = sessionStorage.getItem('kiosk_dob') || searchParams?.get('dob') || '';
       router.push(`/kiosk/register?dob=${encodeURIComponent(dob)}&firstName=${encodeURIComponent(firstName.trim()
)}`);
      }
    } catch {
      setError('Something went wrong. Please try again or ask a staff member.');
    } finally {
      setLoading(false);
    }
  }
  const initialsList = patients.map(p => p.firstName.charAt(0).toUpperCase() + '...');
  return (
    <div style={{
      background: '#fff',
      borderRadius: 20,
      boxShadow: '0 8px 40px rgba(0,0,0,0.10)',
      padding: '48px 56px',
      maxWidth: 580,
      width: '100%',
      textAlign: 'center',
    }}>
      <div style={{ fontSize: 48, marginBottom: 16 }}>■</div>
      <h1 style={{ fontSize: 32, fontWeight: 800, color: '#0055A5', marginBottom: 12 }}>
        Please confirm your first name
      </h1>
      <p style={{ fontSize: 18, color: '#475569', marginBottom: 36 }}>
        We found a record matching your date of birth.
      </p>
      {/* Multiple patients — show initials selector */}
      {patients.length > 1 && !selectedPatient && (
        <div style={{ marginBottom: 36 }}>
          <p style={{ fontSize: 16, color: '#64748b', marginBottom: 16, fontWeight: 600 }}>
            Multiple records found. Select the first initial that matches you:
          </p>
          <div style={{ display: 'flex', gap: 16, justifyContent: 'center', flexWrap: 'wrap' }}>
            {patients.map((p, i) => (
              <button
```

```
                key={p.id}
                onClick={() => setSelectedPatient(p)}
                style={{
                  width: 80,
                  height: 80,
                  fontSize: 28,
                  fontWeight: 700,
                  background: '#f0f9ff',
                  color: '#0055A5',
                  border: '2px solid #bfdbfe',
                  borderRadius: 14,
                  cursor: 'pointer',
                  transition: 'all 0.15s',
                }}
                onMouseEnter={e => {
                  e.currentTarget.style.background = '#0055A5';
                  e.currentTarget.style.color = '#fff';
                }}
                onMouseLeave={e => {
                  e.currentTarget.style.background = '#f0f9ff';
                  e.currentTarget.style.color = '#0055A5';
                }}
              >
                {initialsList[i]}
              </button>
            ))}
          </div>
        </div>
      )}
      {/* Name input — show once patient selected or only one patient */}
      {(selectedPatient || patients.length === 1) && (
        <div style={{ marginBottom: 32 }}>
          <label style={{ display: 'block', fontSize: 16, fontWeight: 600, color: '#334155', marginBottom: 10,
textAlign: 'left' }}>
            Your First Name
          </label>
          <input
            type="text"
            value={firstName}
            onChange={e => setFirstName(e.target.value)}
            placeholder="Enter your first name"
            autoComplete="off"
            style={{
              width: '100%',
              fontSize: 24,
              padding: '16px 20px',
              border: '2px solid #cbd5e1',
              borderRadius: 12,
              outline: 'none',
              color: '#1e293b',
              background: '#f8fafc',
              boxSizing: 'border-box',
            }}
            onFocus={e => (e.target.style.borderColor = '#0d9488')}
            onBlur={e => (e.target.style.borderColor = '#cbd5e1')}
            onKeyDown={e => { if (e.key === 'Enter') handleConfirm(); }}
          />
        </div>
      )}
      {error && (
        <div style={{
          background: '#fef2f2',
          border: '1px solid #fca5a5',
          borderRadius: 10,
```

```
          padding: '14px 18px',
          color: '#dc2626',
          fontSize: 16,
          marginBottom: 24,
          textAlign: 'left',
        }}>
          ■■ {error}
        </div>
      )}
      {(selectedPatient || patients.length === 1) && (
        <button
          onClick={handleConfirm}
          disabled={!firstName.trim() || loading}
          style={{
            width: '100%',
            padding: '22px 0',
            fontSize: 22,
            fontWeight: 700,
            background: !firstName.trim() || loading ? '#94a3b8' : '#0d9488',
            color: '#fff',
            border: 'none',
            borderRadius: 14,
            cursor: !firstName.trim() || loading ? 'not-allowed' : 'pointer',
            minHeight: 70,
            marginBottom: 20,
          }}
        >
          {loading ? 'Verifying…' : '✓ Confirm'}
        </button>
      )}
      {/* If name doesn't match, offer new patient registration */}
      {error && (
        <div style={{ marginBottom: 16, textAlign: 'center' }}>
          <p style={{ fontSize: 15, color: '#64748b', marginBottom: 12 }}>
            Not in our system yet?
          </p>
          <button
            onClick={() => {
              // Keep dob, pass to register
              const dob = sessionStorage.getItem('kiosk_dob') || searchParams?.get('dob') || '';
             router.push(`/kiosk/register?dob=${encodeURIComponent(dob)}&firstName=${encodeURIComponent(firstName.
trim())}`);
            }}
            style={{
              width: '100%',
              padding: '18px 0',
              fontSize: 18,
              fontWeight: 700,
              background: '#0055A5',
              color: '#fff',
              border: 'none',
              borderRadius: 14,
              cursor: 'pointer',
              minHeight: 60,
              marginBottom: 12,
            }}
          >
            ■ Register as New Patient →
          </button>
        </div>
      )}
      <button
        onClick={() => router.push('/kiosk')}
        style={{
```

```
          background: 'transparent',
          border: 'none',
          color: '#64748b',
          fontSize: 16,
          cursor: 'pointer',
          textDecoration: 'underline',
          padding: '8px 0',
          minHeight: 44,
        }}
      >
        ← Start over
      </button>
    </div>
  );
}
export default function VerifyPage() {
  return (
    <Suspense fallback={<div style={{ fontSize: 24, color: '#64748b' }}>Loading…</div>}>
      <VerifyContent />
    </Suspense>
  );
}
```

page.tsx

app/kiosk/register/page.tsx

```
/**
 * @file app/kiosk/register/page.tsx
 * @description Kiosk new-patient registration — collects full contact info.
 */
'use client';
import { useState, Suspense } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
function formatDob(dob: string): string {
  if (!dob) return '';
  const [year, month, day] = dob.split('-');
  const months = ['January','February','March','April','May','June','July','August','September','October','Novembe
r','December'];
  return `${months[parseInt(month, 10) - 1]} ${parseInt(day, 10)}, ${year}`;
}
const inputStyle: React.CSSProperties = {
  width: '100%', padding: '14px 16px', fontSize: 18,
  border: '2px solid #e2e8f0', borderRadius: 10, boxSizing: 'border-box',
  outline: 'none', color: '#1f2937',
};
const labelStyle: React.CSSProperties = {
  display: 'block', fontSize: 14, fontWeight: 700,
  color: '#374151', marginBottom: 6, textTransform: 'uppercase', letterSpacing: '0.04em',
};
function RegisterContent() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const dob = searchParams.get('dob') || '';
  const [step, setStep] = useState(1);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  // Step 1: Name
  const [firstName, setFirstName] = useState(searchParams?.get('firstName') || '');
  const [lastName, setLastName] = useState('');
  // Step 2: Contact
  const [cellPhone, setCellPhone] = useState('');
  const [email, setEmail] = useState('');
  // Step 3: Address
```

```
  const [street, setStreet] = useState('');
  const [apt, setApt] = useState('');
  const [city, setCity] = useState('');
  const [state, setState] = useState('');
  const [zip, setZip] = useState('');
  // Step 4: Insurance
  const [insuranceProvider, setInsuranceProvider] = useState('');
  const [insuranceId, setInsuranceId] = useState('');
  async function handleRegister() {
    setLoading(true); setError('');
    try {
      const res = await fetch('/api/kiosk/register', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          firstName: firstName.trim(), lastName: lastName.trim(), dob,
          phone: cellPhone, email, street, apt, city, state, zip,
          insuranceProvider, insuranceId,
        }),
      });
      const data = await res.json();
      if (res.ok && data.patient) {
        sessionStorage.setItem('kiosk_patient', JSON.stringify(data.patient));
        setStep(5);
        setTimeout(() => router.push('/kiosk/videos'), 2000);
      } else {
        setError(data.error || 'Registration failed. Please see a staff member.');
      }
    } catch {
      setError('Something went wrong. Please see a staff member.');
    } finally { setLoading(false); }
  }
  const primaryBtn: React.CSSProperties = {
    width: '100%', padding: '18px', fontSize: 20, fontWeight: 800,
    background: '#0d9488', color: '#fff', border: 'none', borderRadius: 14,
    cursor: 'pointer', marginTop: 8, minHeight: 64,
  };
  const secondaryBtn: React.CSSProperties = {
    background: 'transparent', border: 'none', color: '#64748b',
    fontSize: 16, cursor: 'pointer', textDecoration: 'underline', padding: '8px 0',
  };
  const TOTAL_STEPS = 4;
  const progressPct = (step / TOTAL_STEPS) * 100;
  return (
    <div style={{ width: '100%', maxWidth: 560 }}>
      <div style={{ background: '#fff', borderRadius: 20, boxShadow: '0 8px 40px rgba(0,0,0,0.10)', overflow: '
hidden' }}>
        {/* Header */}
        <div style={{ background: 'linear-gradient(135deg, #0055A5, #0d9488)', padding: '24px 32px' }}>
          <div style={{ fontSize: 22, fontWeight: 800, color: '#fff', marginBottom: 4 }}>New Patient
Registration</div>
          {step < 5 && (
            <>
              <div style={{ fontSize: 14, color: 'rgba(255,255,255,0.75)', marginBottom: 12 }}>
                Step {step} of {TOTAL_STEPS}
              </div>
              <div style={{ height: 6, background: 'rgba(255,255,255,0.2)', borderRadius: 999 }}>
                <div style={{ height: '100%', width: `${progressPct}%`, background: '#fff', borderRadius: 999,
transition: 'width 0.4s' }} />
              </div>
            </>
          )}
        </div>
        <div style={{ padding: '28px 32px' }}>
```

```
          {error && <div style={{ background: '#fef2f2', border: '1px solid #fecaca', color: '#b91c1c',
borderRadius: 8, padding: '12px 16px', fontSize: 14, marginBottom: 16 }}>■■ {error}</div>}
          {/* Step 1: Name */}
          {step === 1 && (
            <div>
              <div style={{ fontSize: 18, fontWeight: 700, color: '#0055A5', marginBottom: 20 }}>What is your
name?</div>
              <div style={{ marginBottom: 16 }}>
                <label style={labelStyle}>First Name *</label>
                <input style={inputStyle} value={firstName} onChange={e => setFirstName(e.target.value)}
                  placeholder="Jane" autoFocus
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
              <div style={{ marginBottom: 24 }}>
                <label style={labelStyle}>Last Name *</label>
                <input style={inputStyle} value={lastName} onChange={e => setLastName(e.target.value)}
                  placeholder="Smith"
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
              <div style={{ background: '#f0f9ff', borderRadius: 8, padding: '10px 14px', fontSize: 14, color: '
#0369a1', marginBottom: 20 }}>
                ■ Date of Birth: <strong>{formatDob(dob)}</strong>
              </div>
              <button style={{ ...primaryBtn, opacity: (!firstName.trim() || !lastName.trim()) ? 0.5 : 1 }}
                disabled={!firstName.trim() || !lastName.trim()}
                onClick={() => setStep(2)}>Continue →</button>
            </div>
          )}
          {/* Step 2: Contact */}
          {step === 2 && (
            <div>
              <div style={{ fontSize: 18, fontWeight: 700, color: '#0055A5', marginBottom: 20 }}>Contact
Information</div>
              <div style={{ marginBottom: 16 }}>
                <label style={labelStyle}>Cell Phone</label>
                <input style={inputStyle} type="tel" value={cellPhone} onChange={e =>
setCellPhone(e.target.value)}
                  placeholder="(555) 555-0100"
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
              <div style={{ marginBottom: 24 }}>
                <label style={labelStyle}>Email Address</label>
                <input style={inputStyle} type="email" value={email} onChange={e => setEmail(e.target.value)}
                  placeholder="jane@example.com"
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
              <button style={primaryBtn} onClick={() => setStep(3)}>Continue →</button>
              <div style={{ textAlign: 'center', marginTop: 12 }}>
                <button style={secondaryBtn} onClick={() => setStep(1)}>← Back</button>
              </div>
            </div>
          )}
          {/* Step 3: Address */}
          {step === 3 && (
            <div>
              <div style={{ fontSize: 18, fontWeight: 700, color: '#0055A5', marginBottom: 20 }}>Home
Address</div>
              <div style={{ marginBottom: 12 }}>
                <label style={labelStyle}>Street Address</label>
                <input style={inputStyle} value={street} onChange={e => setStreet(e.target.value)}
```

```
                placeholder="123 Main Street"
                onFocus={e => e.target.style.borderColor = '#0d9488'}
                onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
            </div>
            <div style={{ marginBottom: 12 }}>
              <label style={labelStyle}>Apt / Suite</label>
              <input style={inputStyle} value={apt} onChange={e => setApt(e.target.value)}
                placeholder="Apt 2B (optional)"
                onFocus={e => e.target.style.borderColor = '#0d9488'}
                onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
            </div>
            <div style={{ display: 'grid', gridTemplateColumns: '2fr 1fr 1fr', gap: 10, marginBottom: 24 }}>
              <div>
                <label style={labelStyle}>City</label>
                <input style={inputStyle} value={city} onChange={e => setCity(e.target.value)}
placeholder="City"
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
              <div>
                <label style={labelStyle}>State</label>
                <input style={inputStyle} value={state} onChange={e => setState(e.target.value.toUpperCase())}
placeholder="VA" maxLength={2}
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
              <div>
                <label style={labelStyle}>ZIP</label>
                <input style={inputStyle} value={zip} onChange={e => setZip(e.target.value)}
placeholder="22026" maxLength={10}
                  onFocus={e => e.target.style.borderColor = '#0d9488'}
                  onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
              </div>
            </div>
            <button style={primaryBtn} onClick={() => setStep(4)}>Continue →</button>
            <div style={{ textAlign: 'center', marginTop: 12 }}>
              <button style={secondaryBtn} onClick={() => setStep(2)}>← Back</button>
            </div>
          </div>
        )}
        {/* Step 4: Insurance + Confirm */}
        {step === 4 && (
          <div>
            <div style={{ fontSize: 18, fontWeight: 700, color: '#0055A5', marginBottom: 20 }}>Insurance
Information</div>
            <div style={{ marginBottom: 12 }}>
              <label style={labelStyle}>Insurance Provider</label>
              <input style={inputStyle} value={insuranceProvider} onChange={e =>
setInsuranceProvider(e.target.value)}
                placeholder="e.g. BlueCross BlueShield"
                onFocus={e => e.target.style.borderColor = '#0d9488'}
                onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
            </div>
            <div style={{ marginBottom: 20 }}>
              <label style={labelStyle}>Member / Insurance ID</label>
              <input style={inputStyle} value={insuranceId} onChange={e => setInsuranceId(e.target.value)}
                placeholder="Member ID number"
                onFocus={e => e.target.style.borderColor = '#0d9488'}
                onBlur={e => e.target.style.borderColor = '#e2e8f0'} />
            </div>
            {/* Summary */}
            <div style={{ background: '#f8fafc', borderRadius: 10, padding: 16, marginBottom: 20, fontSize: 14
}}>
```

```
                <div style={{ fontWeight: 700, marginBottom: 8, color: '#374151' }}>Please confirm your
information:</div>
                <div><strong>Name:</strong> {firstName} {lastName}</div>
                <div><strong>DOB:</strong> {formatDob(dob)}</div>
                {cellPhone && <div><strong>Phone:</strong> {cellPhone}</div>}
                {email && <div><strong>Email:</strong> {email}</div>}
                {city && <div><strong>Address:</strong> {[street, apt, city, state, zip].filter(Boolean).join(', '
)}</div>}
              </div>
              <button style={{ ...primaryBtn, background: loading ? '#94a3b8' : '#0d9488' }}
                disabled={loading} onClick={handleRegister}>
                {loading ? '■ Registering…' : '■ Register & Continue'}
              </button>
              <div style={{ textAlign: 'center', marginTop: 12 }}>
                <button style={secondaryBtn} onClick={() => setStep(3)}>← Back</button>
              </div>
            </div>
          )}
          {/* Step 5: Done */}
          {step === 5 && (
            <div style={{ textAlign: 'center', padding: '20px 0' }}>
              <div style={{ fontSize: 64, marginBottom: 16 }}>■</div>
              <div style={{ fontSize: 24, fontWeight: 800, color: '#0d9488', marginBottom: 8 }}>Welcome,
{firstName}!</div>
              <div style={{ fontSize: 16, color: '#64748b' }}>You&apos;ve been registered. Proceeding to next
step…</div>
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
export default function RegisterPage() {
  return (
   <Suspense fallback={<div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', minHeight: '
50vh', fontSize: 18, color: '#64748b' }}>Loading…</div>}>
      <RegisterContent />
    </Suspense>
  );
}
```

page.tsx

app/kiosk/videos/page.tsx

```
/**
 * @file app/kiosk/videos/page.tsx
 * @description Kiosk required-video viewing page.
 * Loads active videos from /api/videos, tracks completion via /api/kiosk/video-watched,
 * and auto-advances to /kiosk/consent once all videos are marked watched.
 */
'use client';
import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
interface Video {
  id: string;
  title: string;
  description?: string;
  url: string;
  isActive: boolean;
}
interface Patient {
  id: string;
  name?: string;
```

```
  firstName?: string;
  lastName?: string;
}
export default function KioskVideosPage() {
  const router = useRouter();
  const [videos, setVideos] = useState<Video[]>([]);
  const [patient, setPatient] = useState<Patient | null>(null);
  const [watched, setWatched] = useState<Set<string>>(new Set());
  const [marking, setMarking] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  useEffect(() => {
    const raw = sessionStorage.getItem('kiosk_patient') || '';
    if (raw) {
      try {
        const parsed = raw.startsWith('{') ? JSON.parse(raw) : { id: raw, name: raw };
        setPatient(parsed);
      } catch { /* ignore parse error */ }
    }
    // Get patient ID
    const currentPatientId = raw.startsWith('{') ? (() => { try { return JSON.parse(raw).id } catch { return '' }
})() : raw;
    // Only restore sessionStorage watched state if same patient
    const savedPatientId = sessionStorage.getItem('kiosk_watched_for_patient');
    if (savedPatientId && savedPatientId === currentPatientId) {
      const savedWatched = sessionStorage.getItem('kiosk_watched_ids');
      if (savedWatched) {
        try { setWatched(new Set(JSON.parse(savedWatched))); } catch {}
      }
    } else {
      sessionStorage.removeItem('kiosk_watched_ids');
      sessionStorage.removeItem('kiosk_videos_watched');
    }
    const dbPatientId = currentPatientId
    fetch('/api/videos?active=true')
      .then(r => r.json())
      .then(async data => {
        const list: Video[] = Array.isArray(data) ? data : (data.videos ?? []);
        const activeVideos = list.filter((v: Video) => v.isActive !== false);
        setVideos(activeVideos);
        // Check DB for already-watched videos this session
        if (dbPatientId && activeVideos.length > 0) {
          try {
            const watchRes = await fetch(`/api/kiosk/videos-watched?patientId=${dbPatientId}`);
            if (watchRes.ok) {
              const watchData = await watchRes.json();
              const watchedIds: string[] = watchData.watchedIds ?? [];
              if (watchedIds.length > 0) {
                const watchedSet = new Set(watchedIds);
                setWatched(watchedSet);
                sessionStorage.setItem('kiosk_watched_ids', JSON.stringify(watchedIds));
                sessionStorage.setItem('kiosk_watched_for_patient', dbPatientId);
                sessionStorage.setItem('kiosk_videos_watched', String(watchedIds.length));
                // (routing handled at verify step — no redirect needed here)
              }
            }
          } catch { /* non-blocking */ }
        }
        setLoading(false);
      })
      .catch(() => {
        setError('Could not load videos. Please see a staff member.');
        setLoading(false);
      });
```

```
  }, []);
  async function markWatched(videoId: string) {
    setMarking(videoId);
    try {
      await fetch('/api/kiosk/video-watched', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ patientId: patient?.id, videoId, completed: true }),
      });
    } catch {
      // Non-blocking — still mark locally
    } finally {
      setWatched(prev => {
        const next = new Set([...prev, videoId]);
        // Persist watched IDs tied to this patient
        if (patient?.id) sessionStorage.setItem('kiosk_watched_for_patient', patient.id);
        sessionStorage.setItem('kiosk_watched_ids', JSON.stringify([...next]));
        sessionStorage.setItem('kiosk_videos_watched', String(next.size));
        return next;
      });
      setMarking(null);
    }
  }
  const allWatched = videos.length > 0 && videos.every(v => watched.has(v.id));
  // Auto-proceed only if videos were watched in this session (patient is set)
  useEffect(() => {
    if (allWatched && !loading && patient?.id) {
      const timer = setTimeout(() => router.push('/kiosk/consent'), 1500);
      return () => clearTimeout(timer);
    }
  }, [allWatched, loading, patient, router]);
  const watchedCount = watched.size;
  function getEmbedUrl(url: string): string | null {
    // YouTube
    const ytMatch = url.match(/(?:youtu\.be\/|youtube\.com\/(?:watch\?v=|embed\/|shorts\/))([A-Za-z0-9_-]{11})/);
    if (ytMatch) return `https://www.youtube.com/embed/${ytMatch[1]}?rel=0`;
    // Vimeo
    const vimeoMatch = url.match(/vimeo\.com\/(\d+)/);
    if (vimeoMatch) return `https://player.vimeo.com/video/${vimeoMatch[1]}`;
    return null;
  }
  if (loading) {
    return (
      <div style={{ textAlign: 'center', padding: 48 }}>
        <div style={{ fontSize: 24, color: '#64748b' }}>Loading videos…</div>
      </div>
    );
  }
  if (error) {
    return (
      <div style={{ background: '#fff', borderRadius: 20, boxShadow: '0 8px 40px rgba(0,0,0,0.10)', padding: '
48px 56px', maxWidth: 560, width: '100%', textAlign: 'center' }}>
        <div style={{ fontSize: 48, marginBottom: 16 }}>■■</div>
        <p style={{ fontSize: 20, color: '#dc2626' }}>{error}</p>
      </div>
    );
  }
  if (videos.length === 0) {
    return (
      <div style={{ background: '#fff', borderRadius: 20, boxShadow: '0 8px 40px rgba(0,0,0,0.10)', padding: '
48px 56px', maxWidth: 560, width: '100%', textAlign: 'center' }}>
        <div style={{ fontSize: 56, marginBottom: 16 }}>■</div>
        <h1 style={{ fontSize: 30, fontWeight: 800, color: '#0d9488', marginBottom: 16 }}>No Videos Required</h1>
```

```
        <p style={{ fontSize: 18, color: '#475569', marginBottom: 32 }}>You&apos;re all set! Proceed to
check-in.</p>
        <button
          onClick={() => router.push('/kiosk/consent')}
          style={{ padding: '22px 48px', fontSize: 22, fontWeight: 700, background: '#0d9488', color: '#fff',
border: 'none', borderRadius: 14, cursor: 'pointer', minHeight: 70 }}
        >
          Continue to Check-In →
        </button>
      </div>
    );
  }
  return (
    <div style={{ maxWidth: 720, width: '100%' }}>
      {/* Header */}
      <div style={{ textAlign: 'center', marginBottom: 36 }}>
        {patient?.name && (
          <div style={{ fontSize: 16, fontWeight: 600, color: '#0d9488', marginBottom: 8, background: '#e8f9f7',
display: 'inline-block', padding: '4px 16px', borderRadius: 999 }}>
            ■ {patient.name || [patient.firstName, patient.lastName].filter(Boolean).join(' ')}
          </div>
        )}
        <h1 style={{ fontSize: 30, fontWeight: 800, color: '#0055A5', marginBottom: 10 }}>
          Before Your Appointment
        </h1>
        <p style={{ fontSize: 18, color: '#475569', marginBottom: 16 }}>
          Please watch the following video{videos.length > 1 ? 's' : ''} before checking in.
        </p>
        {/* Progress */}
        <div style={{
          display: 'inline-flex',
          alignItems: 'center',
          gap: 10,
          background: watchedCount === videos.length ? '#d1fae5' : '#f0f9ff',
          border: `2px solid ${watchedCount === videos.length ? '#6ee7b7' : '#bfdbfe'}`,
          borderRadius: 50,
          padding: '10px 24px',
          fontSize: 18,
          fontWeight: 700,
          color: watchedCount === videos.length ? '#065f46' : '#0055A5',
        }}>
          {watchedCount === videos.length ? '■' : '■■'}
          {watchedCount} of {videos.length} watched
        </div>
      </div>
      {/* Video cards */}
      <div style={{ display: 'flex', flexDirection: 'column', gap: 20, marginBottom: 32 }}>
        {videos.map((video, idx) => {
          const isWatched = watched.has(video.id);
          const embedUrl = getEmbedUrl(video.url);
          return (
            <div key={video.id} style={{
              background: '#fff',
              borderRadius: 16,
              boxShadow: '0 4px 20px rgba(0,0,0,0.08)',
              border: isWatched ? '2px solid #6ee7b7' : '2px solid #e2e8f0',
              overflow: 'hidden',
              transition: 'border-color 0.3s',
            }}>
              {/* Card header */}
              <div style={{
                background: isWatched ? '#d1fae5' : '#f8fafc',
                padding: '20px 24px',
                display: 'flex',
```

```
                alignItems: 'center',
                gap: 16,
                borderBottom: '1px solid #e2e8f0',
              }}>
                <div style={{
                  width: 40,
                  height: 40,
                  borderRadius: '50%',
                  background: isWatched ? '#059669' : '#0055A5',
                  color: '#fff',
                  display: 'flex',
                  alignItems: 'center',
                  justifyContent: 'center',
                  fontSize: 18,
                  fontWeight: 700,
                  flexShrink: 0,
                }}>
                  {isWatched ? '✓' : idx + 1}
                </div>
                <div style={{ flex: 1 }}>
                  <div style={{ fontSize: 20, fontWeight: 700, color: '#1e293b' }}>{video.title}</div>
                  {video.description && (
                    <div style={{ fontSize: 14, color: '#64748b', marginTop: 4 }}>{video.description}</div>
                  )}
                </div>
                {isWatched && (
                  <div style={{ fontSize: 14, fontWeight: 700, color: '#059669' }}>WATCHED</div>
                )}
              </div>
              {/* Video player */}
              <div style={{ padding: '20px 24px' }}>
                {embedUrl ? (
                  <div style={{ marginBottom: 16 }}>
                    <div style={{ position: 'relative', paddingBottom: '56.25%', height: 0, borderRadius: 10,
overflow: 'hidden', background: '#000' }}>
                      <iframe
                        src={`${embedUrl}&autoplay=0&rel=0&modestbranding=1`}
                        style={{ position: 'absolute', top: 0, left: 0, width: '100%', height: '100%', border: '
none' }}
                        allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"
                        allowFullScreen
                        referrerPolicy="strict-origin-when-cross-origin"
                        title={video.title}
                      />
                    </div>
                    {/* Fallback link if embed fails */}
                    <a href={video.url} target="_blank" rel="noopener noreferrer"
                      style={{ display: 'block', textAlign: 'center', marginTop: 8, fontSize: 13, color: '
#64748b', textDecoration: 'underline' }}>
                      Video not loading? Click here to open in new tab ■
                    </a>
                  </div>
                ) : (
                  <a
                    href={video.url}
                    target="_blank"
                    rel="noopener noreferrer"
                    style={{
                      display: 'inline-flex',
                      alignItems: 'center',
                      gap: 8,
                      padding: '16px 28px',
                      fontSize: 18,
                      fontWeight: 600,
```

```
                    background: '#0055A5',
                    color: '#fff',
                    borderRadius: 12,
                    textDecoration: 'none',
                    marginBottom: 16,
                    minHeight: 60,
                  }}
                >
                  ■ Watch Video
                </a>
              )}
              {!isWatched && (
                <button
                  onClick={() => markWatched(video.id)}
                  disabled={marking === video.id}
                  style={{
                    width: '100%',
                    padding: '18px 0',
                    fontSize: 18,
                    fontWeight: 700,
                    background: marking === video.id ? '#94a3b8' : '#0d9488',
                    color: '#fff',
                    border: 'none',
                    borderRadius: 12,
                    cursor: marking === video.id ? 'not-allowed' : 'pointer',
                    minHeight: 60,
                    transition: 'background 0.2s',
                  }}
                >
                  {marking === video.id ? 'Saving…' : '■ I watched this video'}
                </button>
              )}
            </div>
          </div>
        );
      })}
    </div>
    {/* Continue button */}
    {allWatched && (
      <div style={{ textAlign: 'center' }}>
        <button
          onClick={() => router.push('/kiosk/consent')}
          style={{
            padding: '22px 56px',
            fontSize: 24,
            fontWeight: 700,
            background: '#0d9488',
            color: '#fff',
            border: 'none',
            borderRadius: 16,
            cursor: 'pointer',
            minHeight: 70,
            boxShadow: '0 4px 20px rgba(13,148,136,0.3)',
            animation: 'pulse 2s infinite',
          }}
        >
          Continue to Check-In Complete →
        </button>
        <style>{`@keyframes pulse { 0%,100% { box-shadow: 0 4px 20px rgba(13,148,136,0.3); } 50% { box-shadow:
0 4px 32px rgba(13,148,136,0.55); } }`}</style>
      </div>
    )}
  </div>
);
```

```
}
```

page.tsx

app/kiosk/consent/page.tsx

```
/**
 * @file app/kiosk/consent/page.tsx
 * @description Kiosk consent signing page. Presents unsigned consent forms to the patient
 * one at a time with a signature pad. Saves signatures via /api/consent/sign.
 * Redirects to /kiosk/done when all forms are signed.
 */
'use client'

import { useEffect, useRef, useState, useCallback } from 'react'
import { useRouter } from 'next/navigation'

interface ConsentForm {
  id: string
  formId?: string
  title?: string
  name?: string
  template?: string
  signed?: boolean
}
interface CheckResponse {
  allSigned: boolean
  forms: ConsentForm[]
  signedIds: string[]
}
export default function ConsentPage() {
  const router = useRouter()
  const canvasRef = useRef<HTMLCanvasElement>(null)
  // Track pending timeouts for cleanup on unmount
  const timeoutsRef = useRef<ReturnType<typeof setTimeout>[]>([])

  const [patientId, setPatientId] = useState<string | null>(null)
  const [patientName, setPatientName] = useState<string>('')
  const [forms, setForms] = useState<ConsentForm[]>([])
  const [currentIndex, setCurrentIndex] = useState(0)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)
  const [signing, setSigning] = useState(false)
  const [hasSig, setHasSig] = useState(false)
  const [agreed, setAgreed] = useState(false)
  const [submitting, setSubmitting] = useState(false)
  const [justSigned, setJustSigned] = useState(false)

  // Clear all pending timeouts on unmount to prevent memory leaks / state-after-unmount
  useEffect(() => {
    return () => {
      timeoutsRef.current.forEach(t => clearTimeout(t))
    }
  }, [])

  const drawWatermark = useCallback(() => {
    const canvas = canvasRef.current
    if (!canvas) return
    const ctx = canvas.getContext('2d')!
    ctx.clearRect(0, 0, canvas.width, canvas.height)
    ctx.font = '18px sans-serif'
    ctx.fillStyle = 'rgba(0,0,0,0.15)'
    ctx.textAlign = 'center'
    ctx.textBaseline = 'middle'
    ctx.fillText('Sign here', canvas.width / 2, canvas.height / 2)
  }, [])

  useEffect(() => {
    // Patient may be stored as JSON object or plain ID string
    const raw = sessionStorage.getItem('kiosk_patient') || sessionStorage.getItem('kiosk_patient_id') || ''
```

```
    let id = raw
    if (raw.startsWith('{')) {
      try {
        const parsed = JSON.parse(raw)
        id = parsed.id || ''
        setPatientName(parsed.name || parsed.firstName || '')
      } catch { id = '' }
    }
    if (!id) {
      router.replace('/kiosk')
      return
    }
    setPatientId(id)
    ;(async () => {
      try {
        const [checkRes, formsRes] = await Promise.all([
          fetch(`/api/consent/check?patientId=${encodeURIComponent(id)}`),
          fetch('/api/forms'),
        ])
        const data: CheckResponse = await checkRes.json()
        const formsData = await formsRes.json()
        const allForms: { id: string; name: string; template: string }[] = Array.isArray(formsData) ? formsData :
(formsData.forms ?? [])
        if (data.allSigned) { router.replace('/kiosk/done'); return }
        const unsigned = data.forms.filter((f: ConsentForm & { signed?: boolean }) => !f.signed)
        if (unsigned.length === 0) { router.replace('/kiosk/done'); return }
        const enriched = unsigned.map((u: ConsentForm & { formId?: string; name?: string; signed?: boolean }) => {
          const match = allForms.find(f => f.id === (u.formId || u.id))
          return {
            id: u.formId || u.id,
            title: u.name || u.title || 'Consent Form',
            name: u.name || u.title || 'Consent Form',
            template: match?.template || '<p>Please sign below to consent.</p>',
          }
        })
        setForms(enriched)
        setLoading(false)
      } catch {
        setError('Unable to load consent forms. Please ask staff for assistance.')
        setLoading(false)
      }
    })()
  }, [router])
  // Draw watermark when form changes
  useEffect(() => {
    if (!loading && forms.length > 0) {
      drawWatermark()
    }
  }, [loading, currentIndex, forms.length, drawWatermark])
  function startSign(e: React.MouseEvent | React.TouchEvent) {
    e.preventDefault()
    setSigning(true)
    const canvas = canvasRef.current!
    const ctx = canvas.getContext('2d')!
    const rect = canvas.getBoundingClientRect()
    const scaleX = canvas.width / rect.width
    const scaleY = canvas.height / rect.height
    const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX
    const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY
    const x = (clientX - rect.left) * scaleX
    const y = (clientY - rect.top) * scaleY
    ctx.beginPath()
    ctx.moveTo(x, y)
  }
```

```
function drawSign(e: React.MouseEvent | React.TouchEvent) {
  e.preventDefault()
  if (!signing) return
  const canvas = canvasRef.current!
  const ctx = canvas.getContext('2d')!
  ctx.lineWidth = 3
  ctx.lineCap = 'round'
  ctx.strokeStyle = '#0055A5'
  const rect = canvas.getBoundingClientRect()
  const scaleX = canvas.width / rect.width
  const scaleY = canvas.height / rect.height
  const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX
  const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY
  const x = (clientX - rect.left) * scaleX
  const y = (clientY - rect.top) * scaleY
  ctx.lineTo(x, y)
  ctx.stroke()
  setHasSig(true)
}
function stopSign() {
  setSigning(false)
}
function clearSig() {
  const canvas = canvasRef.current!
  const ctx = canvas.getContext('2d')!
  ctx.clearRect(0, 0, canvas.width, canvas.height)
  setHasSig(false)
  drawWatermark()
}
function getSignatureBase64(): string {
  return canvasRef.current!.toDataURL('image/png')
}
async function handleSubmit() {
  if (!patientId || !hasSig || !agreed) return
  const form = forms[currentIndex]
  setSubmitting(true)
  try {
    const signature = getSignatureBase64()
    const res = await fetch('/api/consent/sign', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ patientId, formId: form.id, signature }),
    })
    if (!res.ok) throw new Error('Sign failed')
    setJustSigned(true)
    // Use tracked timeouts so they can be cleaned up on unmount
    const t1 = setTimeout(() => {
      setJustSigned(false)
      setAgreed(false)
      setHasSig(false)
      setSigning(false)
      if (currentIndex + 1 >= forms.length) {
        router.replace('/kiosk/done')
      } else {
        setCurrentIndex(i => i + 1)
        const t2 = setTimeout(drawWatermark, 50)
        timeoutsRef.current.push(t2)
      }
    }, 1200)
    timeoutsRef.current.push(t1)
  } catch {
    setError('Failed to save signature. Please try again.')
  } finally {
```

```
      setSubmitting(false)
    }
  }
  if (loading) {
    return (
      <div style={styles.page}>
        <div style={styles.card}>
          <p style={{ fontSize: 22, color: '#555', textAlign: 'center' }}>Loading consent forms…</p>
        </div>
      </div>
    )
  }
  if (error) {
    return (
      <div style={styles.page}>
        <div style={styles.card}>
          <p style={{ fontSize: 20, color: '#c00', textAlign: 'center' }}>{error}</p>
        </div>
      </div>
    )
  }
  if (forms.length === 0) {
    return null
  }
  const form = forms[currentIndex]
  const totalForms = forms.length
  const canSign = hasSig && agreed && !submitting && !justSigned
  return (
    <div style={styles.page}>
      <div style={styles.card}>
        {/* Progress */}
        <p style={styles.progress}>Form {currentIndex + 1} of {totalForms}</p>
        {/* Title */}
        {patientName && (
          <div style={{ fontSize: 16, fontWeight: 600, color: '#0d9488', marginBottom: 12, background: '#e8f9f7',
display: 'inline-block', padding: '4px 16px', borderRadius: 999 }}>
            ■ {patientName}
          </div>
        )}
        <h1 style={styles.title}>{form.title}</h1>
        {/* Consent text */}
        <div
          style={styles.consentBox}
          dangerouslySetInnerHTML={{ __html: form.template ?? '' }}
        />
        {/* Signature pad */}
        <div style={styles.sigSection}>
          <p style={styles.sigLabel}>Please sign below:</p>
          <div style={styles.canvasWrapper}>
            <canvas
              ref={canvasRef}
              width={400}
              height={150}
              style={styles.canvas}
              onMouseDown={startSign}
              onMouseMove={drawSign}
              onMouseUp={stopSign}
              onMouseLeave={stopSign}
              onTouchStart={startSign}
              onTouchMove={drawSign}
              onTouchEnd={stopSign}
            />
          </div>
```

```
          <button onClick={clearSig} style={styles.clearBtn}>Clear</button>
        </div>
        {/* Agree checkbox */}
        <label style={styles.checkLabel}>
          <input
            type="checkbox"
            checked={agreed}
            onChange={e => setAgreed(e.target.checked)}
            style={styles.checkbox}
          />
          <span>I have read and agree to the above</span>
        </label>
        {/* Sign button or success */}
        {justSigned ? (
          <div style={styles.successBanner}>■ Signed!</div>
        ) : (
          <button
            onClick={handleSubmit}
            disabled={!canSign}
            style={{ ...styles.signBtn, ...(canSign ? {} : styles.signBtnDisabled) }}
          >
            {submitting ? 'Saving…' : 'Sign & Continue'}
          </button>
        )}
      </div>
    </div>
  )
}
const styles: Record<string, React.CSSProperties> = {
  page: {
    minHeight: '100vh',
    backgroundColor: '#f0f4f8',
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
    padding: '24px 16px',
  },
  card: {
    backgroundColor: '#ffffff',
    borderRadius: 16,
    padding: '40px 36px',
    maxWidth: 700,
    width: '100%',
    boxShadow: '0 4px 24px rgba(0,0,0,0.10)',
  },
  progress: {
    fontSize: 16,
    color: '#888',
    marginBottom: 8,
    fontWeight: 500,
    textAlign: 'center',
  },
  title: {
    fontSize: 30,
    color: '#0055A5',
    fontWeight: 700,
    marginBottom: 20,
    textAlign: 'center',
  },
  consentBox: {
    maxHeight: 300,
    overflowY: 'auto',
    border: '1px solid #d0d7e0',
    borderRadius: 8,
```

```
  padding: '16px 18px',
  fontSize: 16,
  lineHeight: 1.7,
  color: '#333',
  backgroundColor: '#fafbfc',
  marginBottom: 24,
},
sigSection: {
  marginBottom: 20,
},
sigLabel: {
  fontSize: 17,
  fontWeight: 600,
  color: '#333',
  marginBottom: 8,
},
canvasWrapper: {
  display: 'inline-block',
  border: '2px solid #0055A5',
  borderRadius: 8,
  backgroundColor: '#ffffff',
  overflow: 'hidden',
  touchAction: 'none',
  width: '100%',
  maxWidth: 400,
},
canvas: {
  display: 'block',
  width: '100%',
  height: 150,
  cursor: 'crosshair',
  touchAction: 'none',
},
clearBtn: {
  marginTop: 8,
  padding: '8px 20px',
  fontSize: 15,
  backgroundColor: '#f0f0f0',
  border: '1px solid #ccc',
  borderRadius: 8,
  cursor: 'pointer',
  color: '#444',
},
checkLabel: {
  display: 'flex',
  alignItems: 'center',
  gap: 12,
  fontSize: 18,
  color: '#333',
  cursor: 'pointer',
  marginBottom: 24,
  userSelect: 'none',
},
checkbox: {
  width: 26,
  height: 26,
  cursor: 'pointer',
  accentColor: '#0055A5',
  flexShrink: 0,
},
signBtn: {
  width: '100%',
  padding: '18px 0',
  fontSize: 22,
  fontWeight: 700,
```

```
    backgroundColor: '#0055A5',
    color: '#ffffff',
    border: 'none',
    borderRadius: 12,
    cursor: 'pointer',
    transition: 'background 0.2s',
  },
  signBtnDisabled: {
    backgroundColor: '#b0bec5',
    cursor: 'not-allowed',
  },
  successBanner: {
    textAlign: 'center',
    fontSize: 30,
    fontWeight: 700,
    color: '#2e7d32',
    padding: '18px 0',
    backgroundColor: '#e8f5e9',
    borderRadius: 12,
  },
}
```

page.tsx

app/kiosk/done/page.tsx

```
/**
 * @file app/kiosk/done/page.tsx
 * @description Kiosk check-in completion page. Adds patient to waiting room,
 * displays a thank-you message with a live clock, then auto-resets
 * to /kiosk after 10 seconds so the kiosk is ready for the next patient.
 * Session keys cleared on reset: kiosk_patient, kiosk_dob, kiosk_lookup,
 * kiosk_watched_ids, kiosk_videos_watched, kiosk_watched_for_patient, kiosk_next_step.
 */
'use client'
import { useEffect, useState } from 'react'
import { useRouter } from 'next/navigation'
/** All sessionStorage keys used by the kiosk flow. */
const KIOSK_KEYS = [
  'kiosk_patient',
  'kiosk_dob',
  'kiosk_lookup',
  'kiosk_watched_ids',
  'kiosk_videos_watched',
  'kiosk_watched_for_patient',
  'kiosk_next_step',
] as const
/** Parse patient name and ID from sessionStorage. */
function getPatientFromStorage(): { name: string; id: string } {
  const raw = sessionStorage.getItem('kiosk_patient') || ''
  if (raw.startsWith('{')) {
    try {
      const parsed = JSON.parse(raw)
      return {
        name: parsed.name || [parsed.firstName, parsed.lastName].filter(Boolean).join(' ') || '',
        id: parsed.id || '',
      }
    } catch { /* ignore */ }
  }
  return { name: raw, id: '' }
}
export default function DonePage() {
  const router = useRouter()
  const [patientName, setPatientName] = useState<string>('')
  const [currentTime, setCurrentTime] = useState<string>('')
```

```
const [countdown, setCountdown] = useState(10)
useEffect(() => {
  const { name, id: patientId } = getPatientFromStorage()
  // Defer state update to next tick to avoid cascading-render lint warning
  Promise.resolve().then(() => setPatientName(name))
  // Auto-add to waiting room with video count
  if (patientId && name) {
    const watchedRaw = sessionStorage.getItem('kiosk_videos_watched') || '0';
    const videosWatched = parseInt(watchedRaw, 10) || 0;
    fetch('/api/waiting-room', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ patientId, patientName: name, videosWatched }),
    }).catch(() => {})
  }
}, [])
useEffect(() => {
  function tick() {
    const now = new Date()
    setCurrentTime(
      now.toLocaleTimeString('en-US', {
        hour: 'numeric',
        minute: '2-digit',
        second: '2-digit',
        hour12: true,
      })
    )
  }
  tick()
  const timer = setInterval(tick, 1000)
  return () => clearInterval(timer)
}, [])
// 10-second countdown interval — pure tick, no side-effects in state updater.
useEffect(() => {
  const timer = setInterval(() => {
    setCountdown(prev => (prev > 1 ? prev - 1 : 0))
  }, 1000)
  return () => clearInterval(timer)
}, []) // intentionally empty — runs once for the lifetime of this page
// Navigate when countdown hits 0 (separated from the interval so the state
// updater stays pure — no side-effects like clearInterval inside setState).
useEffect(() => {
  if (countdown !== 0) return
  KIOSK_KEYS.forEach(k => sessionStorage.removeItem(k))
  router.push('/kiosk')
}, [countdown, router])
return (
  <div style={styles.page}>
    <div style={styles.card}>
      {/* Animated checkmark */}
      <div style={styles.checkWrap}>
        <span style={styles.checkEmoji}>■</span>
      </div>
      {/* Thank you */}
      <h1 style={styles.thankYou}>
        Thank you{patientName ? `, ${patientName}` : ''}!
      </h1>
      {/* Subtitle */}
      <p style={styles.subtitle}>Please have a seat.</p>
      {/* Staff message */}
      <p style={styles.message}>
        A member of our staff will call you when we are ready for you.
      </p>
```

```
        {/* Live clock */}
        <div style={styles.clock}>{currentTime}</div>
        {/* Countdown */}
        <div style={{ marginTop: 24, display: 'flex', flexDirection: 'column', alignItems: 'center', gap: 12 }}>
          <div style={{ fontSize: 14, color: '#64748b' }}>
            Returning to start in <strong style={{ color: countdown <= 5 ? '#dc2626' : '#0d9488', fontSize: 18
}}>{countdown}</strong> seconds…
          </div>
          {/* Progress bar */}
          <div style={{ width: 200, height: 6, background: '#e2e8f0', borderRadius: 999, overflow: 'hidden' }}>
            <div style={{ height: '100%', background: countdown <= 5 ? '#dc2626' : '#0d9488', width:
`${(countdown / 10) * 100}%`, transition: 'width 1s linear, background 0.3s' }} />
          </div>
          <button
            onClick={() => {
              KIOSK_KEYS.forEach(k => sessionStorage.removeItem(k))
              router.push('/kiosk')
            }}
            style={{ background: 'transparent', border: 'none', color: '#94a3b8', fontSize: 14, cursor: '
pointer', textDecoration: 'underline', padding: '4px 0' }}
          >
            Start Over Now
          </button>
        </div>
      </div>
      <style>{`
        @keyframes pulse {
          0%, 100% { transform: scale(1); }
          50% { transform: scale(1.12); }
        }
        .check-pulse {
          animation: pulse 2.2s ease-in-out infinite;
          display: inline-block;
        }
      `}</style>
    </div>
  )
}
const styles: Record<string, React.CSSProperties> = {
  page: {
    minHeight: '100vh',
    backgroundColor: '#e8f9f7',
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
    padding: '32px 16px',
    fontFamily: 'system-ui, sans-serif',
  },
  card: {
    backgroundColor: '#ffffff',
    borderRadius: 20,
    padding: '56px 48px 40px',
    maxWidth: 600,
    width: '100%',
    boxShadow: '0 6px 32px rgba(0,0,0,0.09)',
    display: 'flex',
    flexDirection: 'column',
    alignItems: 'center',
    textAlign: 'center',
  },
  checkWrap: {
    marginBottom: 24,
  },
  checkEmoji: {
```

```
    fontSize: 100,
    lineHeight: 1,
    display: 'inline-block',
    animation: 'pulse 2.2s ease-in-out infinite',
  },
  thankYou: {
    fontSize: 36,
    fontWeight: 700,
    color: '#1a4d3e',
    marginBottom: 12,
  },
  subtitle: {
    fontSize: 26,
    fontWeight: 600,
    color: '#2e7d6b',
    marginBottom: 16,
  },
  message: {
    fontSize: 20,
    color: '#555',
    lineHeight: 1.6,
    marginBottom: 36,
    maxWidth: 440,
  },
  clock: {
    fontSize: 48,
    fontWeight: 300,
    color: '#0055A5',
    letterSpacing: '0.04em',
    marginBottom: 48,
    fontVariantNumeric: 'tabular-nums',
  },
  startOver: {
    fontSize: 14,
    color: '#aaa',
    textDecoration: 'underline',
    cursor: 'pointer',
  },
}
```

page.tsx

app/login/page.tsx

```
'use client';
import { useState, useEffect } from 'react';
import { useSearchParams } from 'next/navigation';
import { Suspense } from 'react';
import { signIn } from 'next-auth/react';
type Step = 'credentials' | 'mfa-verify' | 'mfa-setup' | 'mfa-setup-verify';
function LoginPageInner() {
  const [step, setStep] = useState<Step>('credentials');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  // SSO error from URL
  const searchParams = useSearchParams();
  const ssoError = searchParams?.get('error');
  // MFA state
  const [tempToken, setTempToken] = useState('');
  const [mfaCode, setMfaCode] = useState('');
  const [setupSecret, setSetupSecret] = useState('');
  const [setupQr, setSetupQr] = useState('');
```

```
const [setupCode, setSetupCode] = useState('');
// Load QR code when entering setup step
useEffect(() => {
  if (step === 'mfa-setup' && tempToken) {
    fetch('/api/auth/mfa-setup', {
      headers: { 'x-temp-token': tempToken },
    })
      .then(r => r.json())
      .then(data => {
        if (data.secret) {
          setSetupSecret(data.secret);
          setSetupQr(data.qrCode);
        } else {
          setError(data.error || 'Failed to load QR code');
        }
      })
      .catch(() => setError('Network error loading QR code'));
  }
}, [step, tempToken]);
async function handleCredentials(e: React.FormEvent) {
  e.preventDefault();
  setError('');
  setLoading(true);
  try {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    });
    const data = await res.json();
    if (!res.ok) {
      if (res.status === 429) {
        setError('Too many failed attempts. Please wait 15 minutes and try again.');
      } else {
        setError(data.error || 'Invalid email or password.');
      }
      return;
    }
    if (data.requiresMfa) {
      setTempToken(data.tempToken);
      setStep('mfa-verify');
    } else if (data.requiresMfaSetup) {
      setTempToken(data.tempToken);
      setStep('mfa-setup');
    } else {
      // Shouldn't happen with full MFA enforcement, but handle gracefully
      window.location.replace('/');
    }
  } catch {
    setError('Network error. Please try again.');
  } finally {
    setLoading(false);
  }
}
async function handleMfaVerify(e: React.FormEvent) {
  e.preventDefault();
  setError('');
  setLoading(true);
  try {
    const res = await fetch('/api/auth/mfa-verify', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ tempToken, code: mfaCode }),
    });
```

```
      const data = await res.json();
      if (!res.ok) {
        setError(data.error || 'Invalid code.');
        return;
      }
      window.location.replace('/');
    } catch {
      setError('Network error. Please try again.');
    } finally {
      setLoading(false);
    }
  }
  async function handleMfaSetupVerify(e: React.FormEvent) {
    e.preventDefault();
    setError('');
    setLoading(true);
    try {
      const res = await fetch('/api/auth/mfa-setup', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ tempToken, secret: setupSecret, code: setupCode }),
      });
      const data = await res.json();
      if (!res.ok) {
        setError(data.error || 'Invalid code.');
        return;
      }
      window.location.replace('/');
    } catch {
      setError('Network error. Please try again.');
    } finally {
      setLoading(false);
    }
  }
  const headerBlock = (
    <div style={{ background: 'linear-gradient(135deg, #0d9488 0%, #0055A5 100%)', padding: '32px 32px 28px',
textAlign: 'center' }}>
      <div style={{ display: 'flex', justifyContent: 'center', marginBottom: 12 }}>
        {/* eslint-disable-next-line @next/next/no-img-element */}
        <img
          src="/integrated-allergy-logo.jpg"
          alt="Integrated Allergy"
          style={{ height: 56, width: 'auto', borderRadius: 8, background: '#fff', padding: '4px 8px' }}
          onError={e => {
            // Fallback if image fails to load
            const el = e.currentTarget as HTMLImageElement;
            el.style.display = 'none';
            const fallback = el.nextElementSibling as HTMLElement;
            if (fallback) fallback.style.display = 'flex';
          }}
        />
        <div style={{
          display: 'none', alignItems: 'center', justifyContent: 'center',
          height: 56, padding: '0 16px', background: '#fff', borderRadius: 8,
          fontSize: 18, fontWeight: 800, color: '#0d9488', letterSpacing: '-0.5px',
        }}>
          ■ Integrated Allergy
        </div>
      </div>
      <h1 style={{ color: '#fff', fontSize: 20, fontWeight: 700, margin: 0, letterSpacing: '0.01em' }}>
        Integrated Allergy Testing
      </h1>
      <p style={{ color: 'rgba(255,255,255,0.75)', fontSize: 13, marginTop: 4 }}>
        Clinical Testing Portal
```

```
      </p>
    </div>
  );
  const errorBlock = error && (
   <div style={{ background: '#fef2f2', border: '1px solid #fecaca', color: '#b91c1c', borderRadius: 8, padding: '
10px 14px', fontSize: 13, marginBottom: 16 }}>
      {error}
    </div>
  );
  const footerBlock = (
    <div style={{ padding: '0 32px 20px', textAlign: 'center' }}>
      <p style={{ fontSize: 11, color: '#9ca3af', margin: 0 }}>
        ■ HIPAA Compliant Clinical Portal · © {new Date().getFullYear()} Integrated Allergy Testing
      </p>
    </div>
  );
  return (
    <div style={{ minHeight: '100vh', background: 'linear-gradient(135deg, #e8f9f7 0%, #e8f0f8 100%)', display: '
flex', alignItems: 'center', justifyContent: 'center', padding: '16px' }}>
      <div style={{ width: '100%', maxWidth: 440 }}>
        <div style={{ background: '#fff', borderRadius: 20, boxShadow: '0 20px 60px rgba(0,0,0,0.12)', overflow: '
hidden' }}>
          {headerBlock}
          {/* Step 1: Credentials */}
          {step === 'credentials' && (
            <div style={{ padding: '28px 32px' }}>
              <h2 style={{ fontSize: 17, fontWeight: 600, color: '#1f2937', marginBottom: 20, textAlign: 'center'
}}>
                Sign in to your account
              </h2>
              {/* SSO domain error */}
              {ssoError === 'unauthorized_domain' && (
                <div style={{ background: '#fef2f2', border: '1px solid #fecaca', color: '#b91c1c', borderRadius:
8, padding: '10px 14px', fontSize: 13, marginBottom: 16 }}>
                  ■■ Your Microsoft account domain is not authorized. Contact your administrator.
                </div>
              )}
              {ssoError === 'account_disabled' && (
                <div style={{ background: '#fef2f2', border: '1px solid #fecaca', color: '#b91c1c', borderRadius:
8, padding: '10px 14px', fontSize: 13, marginBottom: 16 }}>
                  ■■ Your account has been disabled. Contact your administrator.
                </div>
              )}
              {ssoError === 'sso_failed' && (
                <div style={{ background: '#fef2f2', border: '1px solid #fecaca', color: '#b91c1c', borderRadius:
8, padding: '10px 14px', fontSize: 13, marginBottom: 16 }}>
                  ■■ Microsoft sign-in failed. Please try again or use email and password.
                </div>
              )}
              {/* Microsoft SSO button */}
              <button
                onClick={() => signIn('azure-ad', { callbackUrl: '/api/auth/azure-callback' })}
                style={{
                  display: 'flex', alignItems: 'center', justifyContent: 'center', gap: 10,
                  width: '100%', padding: '12px',
                  background: '#0055A5', color: '#fff',
                  border: 'none', borderRadius: 10,
                  fontSize: 15, fontWeight: 700,
                  cursor: 'pointer', marginBottom: 16,
                }}
              >
                {/* eslint-disable-next-line @next/next/no-img-element */}
                <img src="https://upload.wikimedia.org/wikipedia/commons/4/44/Microsoft_logo.svg" alt="" style={{
width: 20, height: 20 }} />
```

```
                Sign in with Microsoft 365
              </button>
              {/* Divider */}
              <div style={{ display: 'flex', alignItems: 'center', gap: 12, marginBottom: 16 }}>
                <div style={{ flex: 1, height: 1, background: '#e2e8f0' }} />
                <span style={{ fontSize: 13, color: '#94a3b8', whiteSpace: 'nowrap' }}>or sign in with
email</span>
                <div style={{ flex: 1, height: 1, background: '#e2e8f0' }} />
              </div>
              <form onSubmit={handleCredentials}>
                <div style={{ marginBottom: 16 }}>
                  <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 6 }}>
                    Email address
                  </label>
                  <input
                    type="email"
                    value={email}
                    onChange={e => setEmail(e.target.value)}
                    required
                    autoFocus
                    style={{ width: '100%', padding: '10px 14px', border: '1px solid #d1d5db', borderRadius: 10,
fontSize: 14, color: '#1f2937', outline: 'none', boxSizing: 'border-box' }}
                    onFocus={e => e.target.style.borderColor = '#0d9488'}
                    onBlur={e => e.target.style.borderColor = '#d1d5db'}
                    placeholder="you@clinic.com"
                    disabled={loading}
                  />
                </div>
                <div style={{ marginBottom: 20 }}>
                  <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 6 }}>
                    Password
                  </label>
                  <div style={{ position: 'relative' }}>
                    <input
                      type={showPassword ? 'text' : 'password'}
                      value={password}
                      onChange={e => setPassword(e.target.value)}
                      required
                      style={{ width: '100%', padding: '10px 44px 10px 14px', border: '1px solid #d1d5db',
borderRadius: 10, fontSize: 14, color: '#1f2937', outline: 'none', boxSizing: 'border-box' }}
                      onFocus={e => e.target.style.borderColor = '#0d9488'}
                      onBlur={e => e.target.style.borderColor = '#d1d5db'}
                      placeholder="••••••••"
                      disabled={loading}
                    />
                    <button
                      type="button"
                      onClick={() => setShowPassword(v => !v)}
                      style={{ position: 'absolute', right: 12, top: '50%', transform: 'translateY(-50%)',
background: 'none', border: 'none', cursor: 'pointer', color: '#6b7280', fontSize: 15, padding: 2 }}
                      title={showPassword ? 'Hide' : 'Show'}
                    >
                      {showPassword ? '■' : '■■'}
                    </button>
                  </div>
                </div>
                {errorBlock}
                <button
                  type="submit"
                  disabled={loading || !email || !password}
                 style={{ width: '100%', padding: '12px', background: loading || !email || !password ? '#9ca3af' :
 '#0d9488', color: '#fff', border: 'none', borderRadius: 10, fontSize: 15, fontWeight: 700, cursor: loading || !
```

```
email || !password ? 'not-allowed' : 'pointer' }}
                >
                  {loading ? '■ Signing in...' : 'Sign In'}
                </button>
              </form>
            </div>
          )}
          {/* Step 2: MFA Verify */}
          {step === 'mfa-verify' && (
            <div style={{ padding: '28px 32px' }}>
              <h2 style={{ fontSize: 17, fontWeight: 600, color: '#1f2937', marginBottom: 8, textAlign: 'center'
}}>
                ■ Two-Factor Authentication
              </h2>
              <p style={{ fontSize: 13, color: '#6b7280', textAlign: 'center', marginBottom: 20 }}>
                Enter the 6-digit code from your authenticator app.
              </p>
              <form onSubmit={handleMfaVerify}>
                <div style={{ marginBottom: 20 }}>
                  <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 6 }}>
                    Verification Code
                  </label>
                  <input
                    type="text"
                    inputMode="numeric"
                    pattern="[0-9]{6}"
                    maxLength={6}
                    value={mfaCode}
                    onChange={e => setMfaCode(e.target.value.replace(/\D/g, ''))}
                    required
                    autoFocus
                    placeholder="000000"
                    disabled={loading}
                    style={{ width: '100%', padding: '10px 14px', border: '1px solid #d1d5db', borderRadius: 10,
fontSize: 24, textAlign: 'center', letterSpacing: '0.3em', color: '#1f2937', outline: 'none', boxSizing: '
border-box' }}
                    onFocus={e => e.target.style.borderColor = '#0d9488'}
                    onBlur={e => e.target.style.borderColor = '#d1d5db'}
                  />
                </div>
                {errorBlock}
                <button
                  type="submit"
                  disabled={loading || mfaCode.length !== 6}
                  style={{ width: '100%', padding: '12px', background: loading || mfaCode.length !== 6 ? '
#9ca3af' : '#0d9488', color: '#fff', border: 'none', borderRadius: 10, fontSize: 15, fontWeight: 700, cursor:
loading || mfaCode.length !== 6 ? 'not-allowed' : 'pointer' }}
                >
                  {loading ? '■ Verifying...' : 'Verify'}
                </button>
                <button
                  type="button"
                  onClick={() => { setStep('credentials'); setMfaCode(''); setError(''); }}
                  style={{ width: '100%', marginTop: 10, padding: '10px', background: 'none', border: '1px solid
#d1d5db', borderRadius: 10, fontSize: 14, color: '#6b7280', cursor: 'pointer' }}
                >
                  ← Back
                </button>
              </form>
            </div>
          )}
          {/* Step 3: MFA Setup — show QR code */}
          {step === 'mfa-setup' && (
```

```
            <div style={{ padding: '28px 32px' }}>
              <h2 style={{ fontSize: 17, fontWeight: 600, color: '#1f2937', marginBottom: 8, textAlign: 'center'
}}>
                ■ Set Up Two-Factor Authentication
              </h2>
              <p style={{ fontSize: 13, color: '#6b7280', textAlign: 'center', marginBottom: 20 }}>
                Scan this QR code with your authenticator app (Google Authenticator, Authy, etc.), then click
Continue.
              </p>
              {!setupQr && !error && (
                <p style={{ textAlign: 'center', color: '#9ca3af', fontSize: 13 }}>■ Loading QR code…</p>
              )}
              {setupQr && (
                <div style={{ textAlign: 'center', marginBottom: 20 }}>
                  {/* eslint-disable-next-line @next/next/no-img-element */}
                  <img src={setupQr} alt="QR Code" style={{ width: 200, height: 200, margin: '0 auto' }} />
                  <p style={{ fontSize: 11, color: '#9ca3af', marginTop: 8 }}>
                    Can&apos;t scan? Enter manually: <code style={{ fontSize: 10 }}>{setupSecret}</code>
                  </p>
                </div>
              )}
              {errorBlock}
              {setupQr && (
                <button
                  onClick={() => setStep('mfa-setup-verify')}
                  style={{ width: '100%', padding: '12px', background: '#0d9488', color: '#fff', border: 'none',
borderRadius: 10, fontSize: 15, fontWeight: 700, cursor: 'pointer' }}
                >
                  Continue →
                </button>
              )}
              <button
                type="button"
                onClick={() => { setStep('credentials'); setError(''); }}
                style={{ width: '100%', marginTop: 10, padding: '10px', background: 'none', border: '1px solid
#d1d5db', borderRadius: 10, fontSize: 14, color: '#6b7280', cursor: 'pointer' }}
              >
                ← Back
              </button>
            </div>
          )}
          {/* Step 4: MFA Setup — verify the code */}
          {step === 'mfa-setup-verify' && (
            <div style={{ padding: '28px 32px' }}>
              <h2 style={{ fontSize: 17, fontWeight: 600, color: '#1f2937', marginBottom: 8, textAlign: 'center'
}}>
                ■ Confirm Your Authenticator
              </h2>
              <p style={{ fontSize: 13, color: '#6b7280', textAlign: 'center', marginBottom: 20 }}>
                Enter the 6-digit code from your authenticator app to complete setup.
              </p>
              <form onSubmit={handleMfaSetupVerify}>
                <div style={{ marginBottom: 20 }}>
                  <label style={{ display: 'block', fontSize: 13, fontWeight: 600, color: '#374151',
marginBottom: 6 }}>
                    Verification Code
                  </label>
                  <input
                    type="text"
                    inputMode="numeric"
                    pattern="[0-9]{6}"
                    maxLength={6}
                    value={setupCode}
                    onChange={e => setSetupCode(e.target.value.replace(/\D/g, ''))}
                    required
```

```
                    autoFocus
                    placeholder="000000"
                    disabled={loading}
                    style={{ width: '100%', padding: '10px 14px', border: '1px solid #d1d5db', borderRadius: 10,
fontSize: 24, textAlign: 'center', letterSpacing: '0.3em', color: '#1f2937', outline: 'none', boxSizing: '
border-box' }}
                    onFocus={e => e.target.style.borderColor = '#0d9488'}
                    onBlur={e => e.target.style.borderColor = '#d1d5db'}
                  />
                </div>
                {errorBlock}
                <button
                  type="submit"
                  disabled={loading || setupCode.length !== 6}
                  style={{ width: '100%', padding: '12px', background: loading || setupCode.length !== 6 ? '
#9ca3af' : '#0d9488', color: '#fff', border: 'none', borderRadius: 10, fontSize: 15, fontWeight: 700, cursor:
loading || setupCode.length !== 6 ? 'not-allowed' : 'pointer' }}
                >
                  {loading ? '■ Activating MFA...' : 'Enable MFA & Sign In'}
                </button>
                <button
                  type="button"
                  onClick={() => { setStep('mfa-setup'); setSetupCode(''); setError(''); }}
                  style={{ width: '100%', marginTop: 10, padding: '10px', background: 'none', border: '1px solid
#d1d5db', borderRadius: 10, fontSize: 14, color: '#6b7280', cursor: 'pointer' }}
                >
                  ← Back to QR Code
                </button>
              </form>
            </div>
          )}
          {footerBlock}
        </div>
      </div>
    </div>
  );
}
export default function LoginPage() {
  return (
    <Suspense fallback={<div style={{ minHeight: '100vh', display: 'flex', alignItems: 'center', justifyContent: '
center' }}>Loading...</div>}>
      <LoginPageInner />
    </Suspense>
  );
}
```

page.tsx

app/settings/page.tsx

```
'use client';
import Link from 'next/link';
import { useState, useEffect } from 'react';
type AuditLogEntry = {
  id: string;
  action: string;
  entity: string | null;
  entityId: string | null;
  patientId: string | null;
  details: string | null;
  createdAt: string;
  patient?: { id: string; patientId: string; name: string } | null;
};
function getActionColor(action: string): string {
  const upper = action.toUpperCase();
```

```
  if (upper.includes('CREATED') || upper.includes('REGISTER') || upper.includes('SIGN')) {
    return '#16a34a'; // green
  }
  if (upper.includes('VIEW') || upper.includes('READ') || upper.includes('ACCESS') || upper.includes('LOGIN') ||
upper.includes('LOOKUP')) {
    return '#2563eb'; // blue
  }
  if (upper.includes('UPDATED') || upper.includes('MODIFIED') || upper.includes('CHANGED')) {
    return '#d97706'; // orange
  }
  if (upper.includes('DELETED') || upper.includes('REMOVED') || upper.includes('LOGOUT')) {
    return '#dc2626'; // red
  }
  return '#64748b'; // gray default
}
function formatTimestamp(ts: string): string {
  try {
    return new Date(ts).toLocaleString('en-US', {
      month: 'short',
      day: '2-digit',
      year: 'numeric',
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit',
    });
  } catch {
    return ts;
  }
}
function AuditLogSection() {
  const [logs, setLogs] = useState<AuditLogEntry[]>([]);
  const [loading, setLoading] = useState(true);
  const [filter, setFilter] = useState('');
  const [page, setPage] = useState(0);
  const PAGE_SIZE = 50;
  useEffect(() => {
    fetch('/api/audit?limit=200')
      .then(r => r.json())
      .then(d => {
        setLogs(Array.isArray(d) ? d : (d.logs ?? d.entries ?? []));
        setLoading(false);
      })
      .catch(() => setLoading(false));
  }, []);
  const filtered = logs.filter(l =>
    !filter ||
    l.action.toLowerCase().includes(filter.toLowerCase()) ||
    (l.entity || '').toLowerCase().includes(filter.toLowerCase()) ||
    (l.details || '').toLowerCase().includes(filter.toLowerCase()) ||
    (l.patientId || '').toLowerCase().includes(filter.toLowerCase())
  );
  const totalPages = Math.ceil(filtered.length / PAGE_SIZE);
  const paginated = filtered.slice(page * PAGE_SIZE, (page + 1) * PAGE_SIZE);
  return (
    <div className="card" style={{ gridColumn: '1 / -1' }}>
      <div className="card-title">Audit Log</div>
      <p style={{ fontSize: 13, color: '#64748b', marginBottom: 16 }}>
        HIPAA compliance audit trail — recent activity across the system.
      </p>
      <div style={{ display: 'flex', gap: 12, marginBottom: 16, alignItems: 'center' }}>
        <input
          type="text"
          className="form-input"
          placeholder="Filter by action, entity, or details…"
```

```
          value={filter}
          onChange={e => { setFilter(e.target.value); setPage(0); }}
          style={{ maxWidth: 360 }}
        />
        <span style={{ fontSize: 13, color: '#64748b' }}>
          {filtered.length} {filtered.length === 1 ? 'entry' : 'entries'}
        </span>
      </div>
      {loading ? (
        <div style={{ padding: '32px 0', textAlign: 'center', color: '#64748b' }}>Loading audit log…</div>
      ) : paginated.length === 0 ? (
        <div style={{ padding: '32px 0', textAlign: 'center', color: '#64748b' }}>
          {filter ? 'No matching entries.' : 'No audit log entries found.'}
        </div>
      ) : (
        <div style={{ overflowX: 'auto' }}>
          <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 13 }}>
            <thead>
              <tr style={{ background: '#f8fafc', borderBottom: '2px solid #e2e8f0' }}>
                <th style={{ padding: '8px 12px', textAlign: 'left', fontWeight: 600, color: '#475569',
whiteSpace: 'nowrap' }}>Timestamp</th>
                <th style={{ padding: '8px 12px', textAlign: 'left', fontWeight: 600, color: '#475569'
}}>Action</th>
                <th style={{ padding: '8px 12px', textAlign: 'left', fontWeight: 600, color: '#475569'
}}>Entity</th>
                <th style={{ padding: '8px 12px', textAlign: 'left', fontWeight: 600, color: '#475569'
}}>Details</th>
                <th style={{ padding: '8px 12px', textAlign: 'left', fontWeight: 600, color: '#475569'
}}>Patient</th>
              </tr>
            </thead>
            <tbody>
              {paginated.map((log, i) => (
                <tr key={log.id} style={{ borderBottom: '1px solid #f1f5f9', background: i % 2 === 0 ? '#fff' : '
#f8fafc' }}>
                  <td style={{ padding: '8px 12px', color: '#64748b', whiteSpace: 'nowrap', fontFamily: '
monospace', fontSize: 12 }}>
                    {formatTimestamp(log.createdAt)}
                  </td>
                  <td style={{ padding: '8px 12px' }}>
                    <span style={{
                      display: 'inline-block',
                      padding: '2px 8px',
                      borderRadius: 4,
                      fontSize: 11,
                      fontWeight: 700,
                      color: '#fff',
                      background: getActionColor(log.action),
                      letterSpacing: '0.03em',
                    }}>
                      {log.action}
                    </span>
                  </td>
                  <td style={{ padding: '8px 12px', color: '#475569' }}>{log.entity ?? '—'}</td>
                  <td style={{ padding: '8px 12px', color: '#334155', maxWidth: 320 }}>
                    <span title={log.details ?? undefined} style={{ display: 'block', overflow: 'hidden',
textOverflow: 'ellipsis', whiteSpace: 'nowrap' }}>
                      {log.details ?? '—'}
                    </span>
                  </td>
                  <td style={{ padding: '8px 12px' }}>
                    {log.patient ? (
                      <Link href={`/patients/${log.patient.id}`} style={{ color: '#2563eb', textDecoration: '
underline', fontSize: 12 }}>
                        {log.patient.name || log.patient.patientId}
```

```
                    </Link>
                  ) : log.patientId ? (
                    <span style={{ color: '#64748b', fontSize: 12 }}>{log.patientId}</span>
                  ) : (
                    <span style={{ color: '#cbd5e1' }}>—</span>
                  )}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
    {totalPages > 1 && (
      <div style={{ display: 'flex', gap: 8, marginTop: 16, alignItems: 'center', justifyContent: 'flex-end' }}>
        <button
          className="btn btn-secondary"
          onClick={() => setPage(p => Math.max(0, p - 1))}
          disabled={page === 0}
          style={{ padding: '6px 14px', fontSize: 13 }}
        >
          ← Prev
        </button>
        <span style={{ fontSize: 13, color: '#64748b' }}>
          Page {page + 1} of {totalPages}
        </span>
        <button
          className="btn btn-secondary"
          onClick={() => setPage(p => Math.min(totalPages - 1, p + 1))}
          disabled={page >= totalPages - 1}
          style={{ padding: '6px 14px', fontSize: 13 }}
        >
          Next →
        </button>
      </div>
    )}
  </div>
);
}
export default function SettingsPage() {
  return (
    <>
      <div className="page-header">
        <div>
          <div className="page-title">Settings</div>
          <div className="page-subtitle">System configuration and administration</div>
        </div>
      </div>
      <div className="page-body">
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 24 }}>
          <div className="card">
            <div className="card-title">Clinic Information</div>
            <div className="flex flex-col gap-3">
              <div className="form-group">
                <label className="form-label">Clinic Name</label>
                <input type="text" className="form-input" defaultValue="Integrated Allergy Testing" />
              </div>
              <div className="form-group">
                <label className="form-label">Address</label>
                <input type="text" className="form-input" placeholder="123 Medical Drive" />
              </div>
              <div className="form-group">
                <label className="form-label">Phone</label>
                <input type="tel" className="form-input" placeholder="(555) 555-0100" />
```

```
              </div>
              <button className="btn" style={{ alignSelf: 'flex-start' }}>Save Changes</button>
            </div>
          </div>
          <div className="card">
            <div className="card-title">System Status</div>
            <div className="flex flex-col gap-3">
              {[
                { label: 'API Server', status: 'Operational', badge: 'badge-teal' },
                { label: 'Database', status: 'Operational', badge: 'badge-teal' },
                { label: 'Allergen Data', status: 'Loaded', badge: 'badge-green' },
                { label: 'Video Service', status: 'Operational', badge: 'badge-teal' },
              ].map(s => (
                <div key={s.label} className="flex justify-between items-center" style={{ padding: '8px 0',
borderBottom: '1px solid #f1f5f9' }}>
                  <span style={{ fontSize: 14 }}>{s.label}</span>
                  <span className={`badge ${s.badge}`}>{s.status}</span>
                </div>
              ))}
            </div>
          </div>
          <div className="card">
            <div className="card-title">Application Version</div>
            <div className="flex flex-col gap-2">
              {[
                { label: 'Version', value: '3.1.0' },
                { label: 'Environment', value: 'Production' },
                { label: 'Last Updated', value: new Date().toLocaleDateString() },
              ].map(r => (
                <div key={r.label} className="flex justify-between items-center" style={{ padding: '6px 0',
borderBottom: '1px solid #f1f5f9', fontSize: 14 }}>
                  <span style={{ color: '#64748b', fontWeight: 600 }}>{r.label}</span>
                  <span>{r.value}</span>
                </div>
              ))}
            </div>
          </div>
          <div className="card">
            <div className="card-title">Quick Links</div>
            <div className="flex flex-col gap-2">
              <Link href="/patients/new" className="btn btn-secondary" style={{ justifyContent: 'flex-start' }}>■
 Register New Patient</Link>
              <Link href="/testing" className="btn btn-secondary" style={{ justifyContent: 'flex-start' }}>■
Start Testing Session</Link>
              <Link href="/videos" className="btn btn-secondary" style={{ justifyContent: 'flex-start' }}>■
Manage Videos</Link>
            </div>
          </div>
          <AuditLogSection />
        </div>
      </div>
    </>
  );
}
```

■■ config & CI ■■

proxy.ts

proxy.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { jwtVerify } from 'jose'
const SECRET = new TextEncoder().encode(
```

```
  process.env.JWT_SECRET || 'iat-dev-secret-change-in-production-32c'
)
const SESSION_COOKIE = 'iat_session'
const PUBLIC_PATHS = ['/login', '/api/auth', '/consent', '/api/consent', '/api/health', '/kiosk', '
/kiosk/update-info', '/api/kiosk', '/_next', '/favicon']
// Note: /api/seed and /api/allergens/seed are intentionally excluded from PUBLIC_PATHS in production.
// Waiting room GET (staff) is auth-protected; only POST is kiosk-facing and handled via /api/waiting-room POST
with DB validation.
export async function proxy(req: NextRequest) {
  const { pathname } = req.nextUrl
  if (PUBLIC_PATHS.some(p => pathname.startsWith(p))) return NextResponse.next()
  const token = req.cookies.get(SESSION_COOKIE)?.value
  if (!token) {
    if (pathname.startsWith('/api/')) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }
    return NextResponse.redirect(new URL('/login', req.url))
  }
  try {
    const { payload } = await jwtVerify(token, SECRET)
    const headers = new Headers(req.headers)
    headers.set('x-user-id', payload.userId as string || '')
    headers.set('x-user-role', payload.role as string || '')
    return NextResponse.next({ request: { headers } })
  } catch {
    if (pathname.startsWith('/api/')) {
      return NextResponse.json({ error: 'Session expired' }, { status: 401 })
    }
    return NextResponse.redirect(new URL('/login', req.url))
  }
}
export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico|public/).*)'],
}
```

next.config.ts

next.config.ts

```
import { withSentryConfig } from '@sentry/nextjs';
import type { NextConfig } from 'next'
const nextConfig: NextConfig = {
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'Referrer-Policy', value: 'no-referrer' },
          { key: 'Strict-Transport-Security', value: 'max-age=63072000; includeSubDomains; preload' },
          { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' },
          { key: 'Content-Security-Policy', value: [
            "default-src 'self'",
            "script-src 'self' 'unsafe-inline' 'unsafe-eval'", // unsafe-inline needed for Next.js inline scripts
            "style-src 'self' 'unsafe-inline'",
            "img-src 'self' data: blob: https:",
            "font-src 'self' data:",
            "connect-src 'self' https://*.turso.io https://www.youtube.com https://player.vimeo.com",
            "frame-src https://www.youtube.com https://www.youtube-nocookie.com https://player.vimeo.com",
            "media-src 'self' https:",
            "object-src 'none'",
            "base-uri 'self'",
            "form-action 'self'",
            "frame-ancestors 'none'",
```

```
          ].join('; ') },
          { key: 'Access-Control-Allow-Origin', value: process.env.NODE_ENV === 'production' ? '
https://integrated-allergy-testing.vercel.app' : '*' },
        ],
      },
    ]
  },
}
// Only wrap with Sentry when DSN is configured (skips in CI without Sentry token)
const hasSentry = !!process.env.NEXT_PUBLIC_SENTRY_DSN || !!process.env.SENTRY_AUTH_TOKEN;
export default hasSentry
  ? withSentryConfig(nextConfig, {
      silent: true,
      org: 'tipinc',
      project: 'integrated-allergy-testing',
    })
  : nextConfig;
```

ci.yml

.github/workflows/ci.yml

```
name: CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'
      - name: Install dependencies
        run: npm ci
      - name: Security audit
        run: npm audit --audit-level=high
        continue-on-error: true # Warn but don't block on transitive dev-tool vulns
      - name: Generate Prisma client
        run: npx prisma generate
        env:
          DATABASE_URL: 'file:./dev.db'
      - name: Lint
        run: npx eslint app/ lib/ --ext .ts,.tsx --max-warnings=10
        env:
          DATABASE_URL: 'file:./dev.db'
        continue-on-error: true # Don't block CI on lint warnings
      - name: TypeScript check
        run: npx tsc --noEmit
      - name: Run tests
        run: npm test
        env:
          DATABASE_URL: 'file:./prisma/dev.db'
          JWT_SECRET: 'ci-test-secret-minimum-32-characters-long'
          NODE_ENV: 'test'
      - name: Build
        run: npm run build
        env:
```

```
      DATABASE_URL: 'file:./dev.db'
      DATABASE_AUTH_TOKEN: 'ci-placeholder-not-real'
      JWT_SECRET: 'ci-test-secret-minimum-32-characters-long'
      SENTRY_AUTH_TOKEN: ${{ secrets.SENTRY_AUTH_TOKEN }}
      NODE_ENV: 'test'
```